HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology





PROJECT REPORT

MEDORA: Integrated Clinic Management and Electronic Medical Records System

Course: System Design

Class ID: 175882

Supervisor: Ph.D Nguyen Huu Duc

Member	Student ID
Nguyen Tran Minh Tri	202400116
Le Tuan Hung	202416800
Nguyen Khac Hoang Phuc	202416818
Nguyen Hoang Quan	202416738
Nguyen Duc Dung	202416681

Ha Noi, 2026

Abstract

MEDORA is a proposed integrated clinic-management and electronic medical-record (EMR) system for outpatient clinics and small healthcare facilities. It addresses workflow fragmentation across patient registration, appointment scheduling, check-in, clinical documentation, prescriptions, service execution, billing and operational reporting. The proposed solution centers on a shared patient and encounter record, explicit workflow states, role-based access control and an audit trail for sensitive actions. This report develops the business model, domain concepts, business rules, functional and non-functional requirements, use-case model and traceability needed to guide the subsequent system design and evaluation. The scope is deliberately limited to internal clinic operations; clinical decision-making, insurer settlement, national interoperability and production compliance certification remain outside the current project boundary.

Keywords: electronic medical records, clinic management, outpatient workflow, role-based access control, system analysis and design.

Contents

List of Figures 5

List of Tables 7

List of Abbreviations 8

1 Introduction and Problem Definition 9
- 1.1 Background and Existing System . . . 9
- 1.2 Problem Statement and Motivation . . . 10
- 1.3 Project Objectives and Success Criteria . . . 10
- 1.4 System Scope, Assumptions, and Constraints . . . 11
- 1.5 Proposed Solution and System Overview . . . 12

2 Business Analysis and Domain Modeling 14
- 2.1 Business Context and Operational Overview . . . 14
- 2.2 Stakeholders and Responsibilities . . . 14
- 2.3 Business Workflow and Controls . . . 16
- 2.4 Business Rules . . . 19
- 2.5 Domain Model and Core Entities . . . 19

3 System Requirements Analysis 22
- 3.1 Functional Decomposition . . . 22
- 3.2 Actors and Overall Use Case Model . . . 22
- 3.3 Use Case Diagrams by Functional Group . . . 22
  - 3.3.1 Patient Management . . . 23
  - 3.3.2 Appointment, Check-in and Queue Management . . . 24
  - 3.3.3 Medical Examination and Encounter Management . . . 26
  - 3.3.4 Prescription, Billing and Governance . . . 29
- 3.4 Detailed Use Case Specifications . . . 31
  - 3.4.1 UC-021 Book Appointment . . . 31
  - 3.4.2 UC-027 Check In Patient . . . 32
  - 3.4.3 UC-037/UC-038 Sign and Amend a Clinical Note . . . 32
  - 3.4.4 UC-048 Validate and Issue Prescription . . . 33
  - 3.4.5 UC-039/UC-040 Service Order, Allocation and Result . . . 33
  - 3.4.6 UC-063/UC-065 Record Payment and Refund . . . 33
- 3.5 Functional Requirements . . . 34
- 3.6 Non-functional Requirements . . . 35
- 3.7 Role-based Access Control Matrix . . . 35
- 3.8 Requirements Traceability and Use Case Validation . . . 38

4 System Design and Structural Modeling 38
4.1 System Architecture . . . 39
4.1.1 Logical Architecture . . . 39
4.1.2 Deployment Architecture . . . 39
4.2 Layered Design and Design Patterns . . . 39
4.2.1 Presentation, Business Logic, and Data Access Layers . . . 41
4.2.2 API and Web Service Design . . . 41
4.3 Database Design and Data Integrity . . . 42
4.3.1 Entity–Relationship Diagrams . . . 42
4.3.2 Relational Schema and Normalization . . . 44
4.3.3 Persistent Classes and Database Mapping . . . 44
4.3.4 Keys, Constraints, Data Types, and Multiplicities . . . 45
4.4 Structural UML Diagrams . . . 45
4.4.1 Overall Class Diagram . . . 45
4.4.2 Design Class Views . . . 45
4.4.3 Class Diagrams by Use Case Group . . . 46
4.4.4 Package Diagram . . . 46
4.4.5 Component Diagram . . . 46
4.4.6 Deployment Diagram . . . 46
4.4.7 Object Diagram . . . 46
4.5 Use Case-oriented Design . . . 46
4.5.1 Participating Classes and Their Responsibilities . . . 46
4.5.2 Collaboration Between System Components . . . 47

5 Behavioral Modeling and System Implementation 47
5.1 Behavioral UML Diagrams . . . 47
5.1.1 Sequence Diagrams for Key Use Cases . . . 48
5.1.2 Activity Diagrams for Complex Workflows . . . 48
5.1.3 State Machine Diagrams for Lifecycle-driven Entities . . . 48
5.2 Source Code Organization and Module Structure . . . 51
5.3 Implementation of Core Functionalities . . . 53
5.4 Development and Deployment Environment . . . 53
5.5 Proposed User Interfaces and API Evidence . . . 54

6 System Testing and Evaluation 56
6.1 Testing Strategy and Test Environment . . . 56
6.2 Test Scenarios and Test Case Design . . . 56
6.3 Functional Testing and Use Case Coverage . . . 57
6.4 Exception Handling, Authorization, and Data Integrity Testing . . . 58
6.5 Test Execution Results and Evidence . . . 58
6.6 Evaluation Against Functional and Non-functional Requirements . . . 59
6.7 Limitations and Outstanding Issues . . . 59

7 Conclusion and Future Work **60**
7.1 Summary of Achievements 60
7.2 Limitations of the Current System 60
7.3 Future Improvements and Extensions 60

A Detailed Use Case Specifications **61**
A.1 UC-021 Book Appointment 61
A.2 UC-027 Check In Patient 62
A.3 UC-037/UC-038 Sign and Amend Clinical Note 62
A.4 UC-048 Validate and Issue Prescription 63
A.5 UC-039/UC-040 Schedule and Execute Service Order 64
A.6 UC-063/UC-065 Record Payment and Refund 65

B Supplementary UML Diagrams **66**

C API Specifications and Data Schemas **67**
C.1 Representative Appointment Data 67
C.2 Representative Encounter Data 67
C.3 Representative Audit Event Data 68

D Detailed Test Cases and Execution Results **68**

References **84**

List of Figures

1 Target business workflow for a standard outpatient journey . . . 17
2 Appointment and check-in workflow with alternative and exception paths . . . 18
3 Core clinical and financial conceptual domain model . . . 20
6 Patient-management use cases . . . 23
7 Appointment-management use cases . . . 24
8 Check-in and queue-management use cases . . . 25
9 Screening and consultation use cases . . . 26
10 Clinical-note lifecycle and encounter-closure use cases . . . 27
11 Service-order and execution use cases . . . 28
12 Prescription and medication-safety use cases . . . 29
13 Billing and payment use cases . . . 30
14 Administration, reporting and governance use cases . . . 31
4 Functional decomposition of MEDORA . . . 36
5 Overall use-case model of MEDORA . . . 37
15 Proposed BCE logical architecture for representative MEDORA clinic workflows 40
16 Proposed logical deployment baseline; physical infrastructure remains to be selected 40
17 Logical ERD for patient identity, appointments, and encounters . . . 43
18 Logical ERD for encounter notes and diagnoses . . . 43
35 Appointment lifecycle and terminal outcomes . . . 49
36 Encounter lifecycle from check-in to clinical completion . . . 49
37 Clinical-note version lifecycle with immutable signed versions . . . 50
38 Procedure-order lifecycle with allocation and execution outcomes . . . 50
39 Invoice lifecycle with payment and completed-refund transitions . . . 51
40 Front desk and queue wireframe . . . 55
41 Clinician encounter workspace wireframe . . . 55
42 Cashier invoice and payment wireframe . . . 55
19 Logical ERD for prescriptions, medication items, and service orders . . . 70
20 Logical ERD for invoices, payment events, and linked refunds . . . 71
21 Proposed boundary, controller, and entity responsibilities for patient registration and outpatient encounters . . . 72
22 Proposed billing responsibilities with an optional payment processor dependency 73
23 Domain class view for appointment booking and encounter creation . . . 74
24 Domain class view for invoice settlement and refund history . . . 74
25 Proposed MEDORA logical package decomposition . . . 75
26 Proposed application component dependencies . . . 75
27 System sequence for appointment booking and availability validation . . . 76
28 System sequence for check-in, encounter creation, and queue assignment . . . 77
29 System sequence for signing a clinical note and appending a correction . . . 78
30 System sequence for configured prescription safety checks and issue . . . 79
31 System sequence for resource allocation and service-order execution . . . 80

32 System sequence for payment and separately authorized refund events 81
33 Activity model for service-order allocation and execution 82
34 Activity model for encounter closure and independent financial settlement 83

List of Tables

2 MEDORA product definition and value proposition . . . 12
3 Target operating model by business area . . . 14
4 Stakeholder responsibilities and authority boundaries . . . 15
5 Role-to-activity mapping and interaction boundaries . . . 16
6 Major business processes, outcomes and controlled alternatives . . . 19
7 Core business rules . . . 19
8 Core entity glossary . . . 21
9 Core domain relationships . . . 21
10 Business-to-system traceability . . . 21
17 Functional requirements catalog . . . 34
18 Non-functional requirements catalog . . . 35
19 Target role-based access-control matrix . . . 36
20 Requirements traceability and planned verification . . . 38
21 Proposed architecture and design choices . . . 41
22 Proposed API responsibility groups; endpoint contract not yet specified . . . 42
23 Proposed domain-to-persistence mapping . . . 44
24 Proposed key multiplicities and integrity constraints . . . 45
25 Proposed class responsibilities by use-case group . . . 47
26 Lifecycle states by entity . . . 48
27 Implementation and deployment evidence available in the repository . . . 52
28 Proposed implementation slices and acceptance invariants . . . 53
29 Proposed development and deployment environment matrix . . . 54
30 Planned verification levels and present evidence boundary . . . 56
31 Planned test cases and current execution status . . . 57
32 Representative functional and quality-requirement test coverage . . . 58
33 Test execution summary at report time . . . 58
34 Evaluation criteria and evidence status . . . 59
35 UML source and rendered-figure inventory . . . 66
36 Detailed test procedures and expected outcomes . . . 68

List of Abbreviations

Abbreviation	Meaning
ADR	Architecture Decision Record
API	Application Programming Interface
BR	Business Rule (identifier prefix, for example BR-001)
DTO	Data Transfer Object
e2e	End-to-end (test level)
EMR	Electronic Medical Record
ERD	Entity–Relationship Diagram
FR	Functional Requirement (identifier prefix, for example FR-001)
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
LTS	Long-Term Support
MVP	Minimum Viable Product
NFR	Non-functional Requirement (identifier prefix, for example NFR-01)
OECD	Organisation for Economic Co-operation and Development
OpenAPI	OpenAPI Specification, a format for describing REST APIs
P0, P1, P2	Requirement priority: P0 is required for the MVP, P1 is a controlled extension, P2 depends on a later decision or integration contract
p95	95th percentile
PNG	Portable Network Graphics
RBAC	Role-Based Access Control
REST	Representational State Transfer
RPO	Recovery Point Objective
RTO	Recovery Time Objective
S3	Simple Storage Service, an object-storage interface
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
TC	Test Case (identifier prefix, for example TC-001)
TLS	Transport Layer Security
UC	Use Case (identifier prefix, for example UC-021)
UI	User Interface
UML	Unified Modeling Language
WHO	World Health Organization

1 Introduction and Problem Definition

1.1 Background and Existing System

Healthcare delivery is becoming a coordination problem as much as a clinical one. Across countries, providers are expected to offer safer, more continuous and more patient-centered care while dealing with rising demand, constrained staff capacity and increasingly complex information flows. The World Health Organization (WHO) estimates that approximately one in ten patients is harmed during health care and that more than half of this harm is preventable; medication-related harm accounts for a substantial share of the preventable cases [World Health Organization, 2023]. These figures do not imply that software can replace clinical judgment. They do, however, show why reliable identification, complete records, clear hand-offs and traceable actions are system-level concerns rather than administrative details.

Digital health is therefore moving from a collection of isolated applications toward an integrated operating capability. The WHO describes digital health as a means of aligning organizational, human and technological resources around better health outcomes, not simply as the deployment of another application [World Health Organization, 2021b]. Yet digitalization can reproduce the same fragmentation when applications are implemented as disconnected, supply-driven tools; WHO identifies integration over isolation, data governance and protection of privacy as essential conditions for useful digital transformation [World Health Organization, 2021a; World Health Organization Alliance for Health Policy and Systems Research, 2024]. In a clinic the difference is concrete: an appointment calendar, a patient database and a billing screen may each work in isolation while the overall journey still depends on repeated data entry, informal communication and manual reconciliation. The result is a fragile process in which a patient can be correctly registered in one place but remain invisible to the next person who must act.

The persistence of legacy workflows is not unique to one country. The OECD reports that 75% of global fax traffic is used for medical services and that evidence-based innovations can take as long as 17 years to become routine practice, illustrating the distance between available technology and dependable day-to-day adoption [Organisation for Economic Co-operation and Development, 2023]. For a small clinic, the implication is practical: the priority is not to buy the most sophisticated technology, but to design a coherent system that staff can use consistently and that can exchange trustworthy data when the organization grows.

The Vietnamese context makes this challenge particularly tangible. Viet Nam entered the aging phase in 2011; people aged 60 and over represented 11.9% of the population in 2019, and this share is projected to exceed 25% by 2050 [United Nations Population Fund Viet Nam, 2021]. An aging population increases the need for repeated visits, longitudinal clinical histories and coordination across services. At the point of care the friction is already measurable; the waiting-time studies are summarized in the next subsection. Long queues are more than a comfort issue: they consume staff time, hide workload, create openings for transcription errors and weaken patients' confidence in the service.

1.2 Problem Statement and Motivation

The need for an electronic medical-record system is visible in three connected indicators. First, outpatient flow is slow: a study of 7,931 visits at a Vietnamese national hospital reported a mean total waiting time of 104.1 minutes, while departments handled approximately 150–250 patients per day without proportional capacity growth [Nguyen et al., 2018]. A broader study reported average waits of 32.58 minutes at district hospitals and 11.58 minutes at commune health centers before patients met clinical staff [Nguyen et al., 2020]. Second, patient-safety risk remains material: the WHO estimates that one in ten patients is harmed in health care and that more than half of this harm is preventable [World Health Organization, 2023]. Third, medication errors alone are estimated to cost about US$42 billion globally each year [World Health Organization, 2017].

These figures do not prove that every clinic has the same performance. They establish why the workflow deserves system-level attention. When identity, appointments, clinical notes, orders and payment are split across paper forms, spreadsheets or isolated applications, the following problems become likely:

- duplicate patient records and repeated demographic entry;
- appointment, room or staff conflicts that are discovered too late;
- incomplete visibility of allergies, previous encounters, current medication or open orders;
- unclear responsibility when an order moves from prescription to execution;
- silent overwriting of signed information and weak evidence of who changed it; and
- limited management visibility into waiting, workload, service progress and payment status.

WHO identifies fragmented data governance and weak exchange mechanisms as barriers to timely, high-quality use of health data [World Health Organization, 2021a]. MEDORA addresses this coordination gap by providing one patient identity, one encounter context and explicit workflow states across authorized roles. Its "smart" behavior is deliberately bounded: the system applies rules, warnings, validation and audit trails at the right step, while diagnosis and treatment decisions remain with qualified clinicians.

The architecture is part of the motivation. Separating presentation, application rules, domain entities and persistence prevents a new screen from bypassing clinical or access-control rules. Explicit services and state transitions make double-booking prevention, signed-record amendment, authorization and auditability testable. The intended result is a system that is easier to audit and to extend than a set of screens that merely digitizes the existing fragmentation.

1.3 Project Objectives and Success Criteria

This project proposes MEDORA, an integrated clinic-management and electronic medical-record system for a clinic or small healthcare facility. Its objective is to turn a fragmented

sequence of tasks into a controlled, observable and reusable workflow. The project pursues five linked outcomes:

- one stable patient identity and one longitudinal record across visits;
- coordinated appointments, queues, rooms and staff;
- structured support for examination, diagnosis, prescriptions and service orders;
- auditable handling of sensitive changes and role-based access to health data; and
- operational dashboards and reports that support evidence-based management.

Success is specified using SMART criteria so that the prototype can be evaluated without overstating clinical impact. During the final test cycle, against a documented baseline workload, MEDORA will:

- reduce median registration-to-completion time by at least 25% for the representative outpatient scenario;
- accept zero double-bookings for the same doctor, room or constrained resource across 100% of the concurrency test cases;
- achieve at least 95% completion of configured mandatory encounter fields across the tested completed visits;
- record an actor, timestamp, action and affected entity for 100% of tested sensitive events, including clinical amendments, prescription changes, exports and permission changes; and
- demonstrate that 100% of negative authorization test cases are rejected at the API/server boundary, while the principal happy-path workflow completes successfully in the agreed test environment.

Each target is specific to a defined workflow, measurable through test evidence, achievable for the course-project scope, relevant to the stated pain points and time-bound to the final evaluation cycle. The figures are prototype acceptance targets. They are not claims of production performance; a real deployment would first establish a site-specific baseline and revalidate them with clinical and operational stakeholders.

1.4 System Scope, Assumptions, and Constraints

MEDORA is an internal web system for a clinic or small healthcare facility. It supports the outpatient journey from patient identification and appointment or walk-in reception to check-in, screening, consultation, prescription or service order, execution, encounter closure and payment status.

Receptionists manage identity, appointments and queues; nurses record screening; doctors own clinical decisions; technicians and medicine staff record actual execution; cashiers record payments; and authorized finance staff assess and execute permitted refunds. Managers, administrators and auditors supervise operations, configuration and accountability. The system preserves the difference between planned and completed work: an appointment is not an encounter, a prescription is not a medicine event, an order is not a result, and an invoice is not a payment. The MVP focuses on internal operations, basic billing, reporting and governance. Patient self-service, external diagnostic systems, insurer settlement and advanced inventory are later-stage extensions. Every sensitive action is constrained by role, scope and workflow state; signed notes are amended rather than overwritten, and concurrent booking must not create a conflict.

1.5 Proposed Solution and System Overview

MEDORA is a clinic operating system built around one shared patient and encounter record, not a collection of unrelated modules. Its central unit of work is the outpatient encounter. A Patient may have many Appointments and Encounters; an Encounter is the controlled context for clinical notes, vital signs, diagnoses, prescriptions and procedure orders; an order may consume a scheduled ResourceBooking and later receive an execution result; and an Invoice and Payment record the financial consequence without changing the clinical history. Attachments, notifications and AuditLogs are linked to the relevant business object rather than stored as anonymous files or messages.

Table 2 defines the product by capability. Each row states the function, its primary users and the operational value it is expected to create. This makes the product boundary concrete before the report describes its technical realization: MEDORA is an internal clinic-management and electronic medical-record product that coordinates people, information, resources and decisions throughout an outpatient episode.

Table 2: MEDORA product definition and value proposition

Function	Primary users	What the system does	Value delivered
Patient identity and record management	Reception, nurses, doctors	Search/register patients, flag duplicates and link history, allergies, documents and encounters.	One longitudinal record and safer identification.
Appointment, schedule and queue management	Reception, managers; patient portal (Phase 2)	Book, reschedule, cancel, check in and assign queues against current availability.	Visible workload and fewer coordination conflicts.
Clinical examination and EMR	Nurses, doctors	Capture screening, vital signs, consultation, diagnosis and follow-up in the Encounter.	Continuous, structured clinical context.

Function	Primary users	What the system does	Value delivered
Prescription and medication safety	Doctors, medicine staff	Issue prescriptions with directions and configured safety checks; retain amendment and medicine-event history.	Safer hand-offs between prescribing and dispensing.
Procedure, service and resource coordination	Doctors, nurses, technicians	Create orders, allocate constrained resources and record execution/result.	Prevents double-booking and links decision to outcome.
Billing and payment status	Cashiers, managers	Create invoices and retain payment/refund history separately from clinical content.	Clear financial accountability and reconciliation.
Administration and role-based access	Administrators, managers, auditors	Manage users, permissions, catalogs, schedules and scope.	Least privilege and adaptable operations.
Dashboards, reports and audit trail	Managers, auditors	Present operational indicators, controlled export and sensitive-event history.	Evidence for decisions and investigation.
Attachments and notifications	Assigned staff; patient portal (Phase 2)	Store controlled documents and send privacy-aware operational notifications.	Reliable follow-up with lower privacy exposure.

The solution uses four cooperating layers:

- **Presentation:** role-specific workspaces for reception, nursing, consultation, service execution, cashier operations and management.
- **Application:** use cases such as registration, booking, check-in, consultation, prescription, service scheduling, result recording, encounter closure and payment.
- **Domain:** rules for availability, permissions, state transitions, signed-record amendments and clinical warnings.
- **Data:** persistent patient, encounter, order, resource, invoice and audit entities with integrity constraints and history.

Each workflow step has an explicit state and accountable actor. Backend authorization, controlled attachments and audit history protect the record while the modular structure leaves a clear path for later portal, diagnostic and interoperability extensions.

2 Business Analysis and Domain Modeling

2.1 Business Context and Operational Overview

MEDORA is an internal web system for an outpatient clinic or small healthcare facility. It coordinates one outpatient episode from patient identification and appointment handling to consultation, service execution, payment status and operational reporting. Its central design principle is continuity: every authorized role works from the same patient and encounter context rather than from disconnected records.

Four distinctions protect that continuity. An Appointment reserves time, while an Encounter represents the visit that actually occurs. A prescription is a clinical decision; dispensing or administration is a later, accountable event. A service order is not an execution result. An Invoice records charges, whereas a Payment or refund records a financial transaction. These distinctions prevent the system from reporting planned work as completed work.

The MVP stops at internal operations, basic billing, reporting and governance. Patient self-service, laboratory and imaging interfaces and insurer settlement come later.

Business area	Target operating model
Patient identity	Search before registration; flag possible duplicates; preserve merge history.
Visit coordination	Treat appointment, check-in, encounter and queue placement as explicit states.
Clinical care	Keep screening, documentation, diagnosis, prescription and orders in one encounter context.
Execution and finance	Record allocation, service execution, invoice and payment as separate events.
Governance	Apply scoped access, controlled export and auditable sensitive actions.

Table 3: Target operating model by business area

2.2 Stakeholders and Responsibilities

Table 4 defines the roles that participate in the target workflow. Access is always restricted by role, facility, assignment and workflow state; managerial responsibility does not create unrestricted clinical-edit access.

Role	Primary responsibility	Authority boundary
Receptionist	Identify/register patients; manage appointments, arrivals and queues.	Administrative and operational data only.
Nurse	Record screening, vital signs and assigned care activity.	Cannot alter diagnosis or signed prescriptions.
Doctor	Consult, document, diagnose, prescribe, order services and close encounters.	Owns clinical decisions for assigned encounters.
Technician / Pharmacist	Execute assigned service work or record the medicine event.	Records execution; does not create the clinical decision.
Cashier	Create eligible invoices and record payments.	Cannot edit clinical content or approve refunds.
Authorized Finance Staff	Review refund requests and execute permitted refunds.	Requires the finance authority and approval rule defined for the request.
Authorized Registrar	Review flagged duplicates and perform approved record merges.	Merge authority only; cannot edit clinical content.
Manager	Monitor capacity, services, revenue and reports.	Uses scoped operational information.
Administrator / Auditor	Configure access/catalogs or review audit evidence.	Administration and audit are separate from clinical authority.
Patient (Phase 2)	Request appointments and view released documents.	Accesses only verified, explicitly released information.

Table 4: Stakeholder responsibilities and authority boundaries

Actor	Main system interaction	Boundary
Receptionist	Patient search, registration, appointment, check-in and queue.	No clinical decision editing.
Nurse / Doctor	Assigned encounter, clinical history, screening, documentation and orders.	Access is assignment- and role-scoped.
Technician / Pharmacist	Assigned service execution or medicine event.	Cannot create the originating clinical decision.
Cashier	Invoice, payment and payment status.	Cannot modify clinical records or approve refunds.
Authorized Finance Staff	Refund request, approval status and completed refund.	Cannot modify clinical records; authority is limited to approved finance actions.
Authorized Registrar	Merge request review and record merge, with merge history preserved.	Cannot edit clinical content; a Receptionist may request a merge but cannot perform it.
Manager / Administrator / Auditor	Reports, configuration or audit review.	Each role has a separate scope.
Patient (Phase 2)	Appointment request and released-document viewing.	Verified self-service only.

Table 5: Role-to-activity mapping and interaction boundaries

2.3 Business Workflow and Controls

Figure 1 shows the normal outpatient journey. Figure 2 isolates appointment and arrival exceptions, where invalid transitions are most likely to disrupt service flow.
The workflow is governed by five control points:

1. Search and duplicate checks occur before a new patient record is created.
2. Booking and rescheduling validate provider, room and required-resource availability before commitment.
3. Check-in creates one valid active Encounter and its queue state.
4. Orders, execution and medicine events are recorded as separate lifecycle events.
5. Encounter closure, payment and refund preserve their own accountable history.

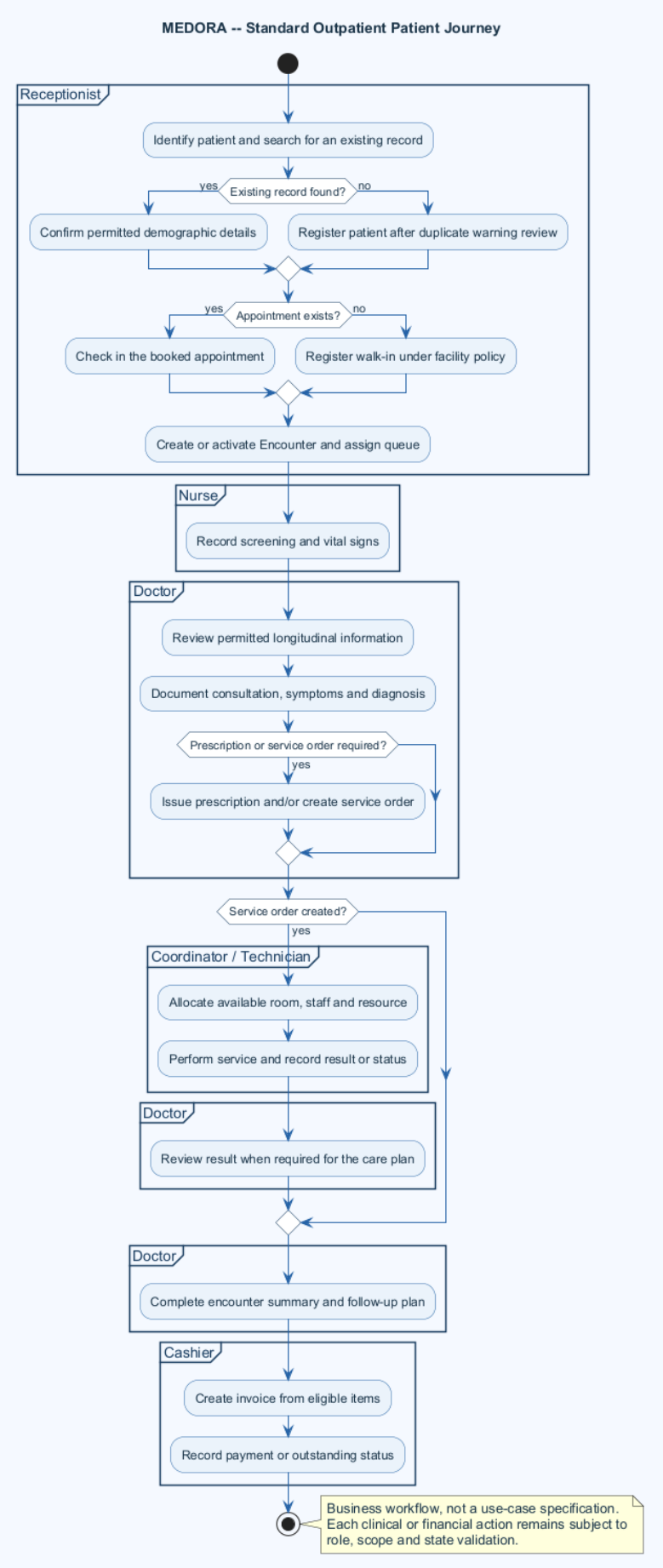


Figure 1: Target business workflow for a standard outpatient journey

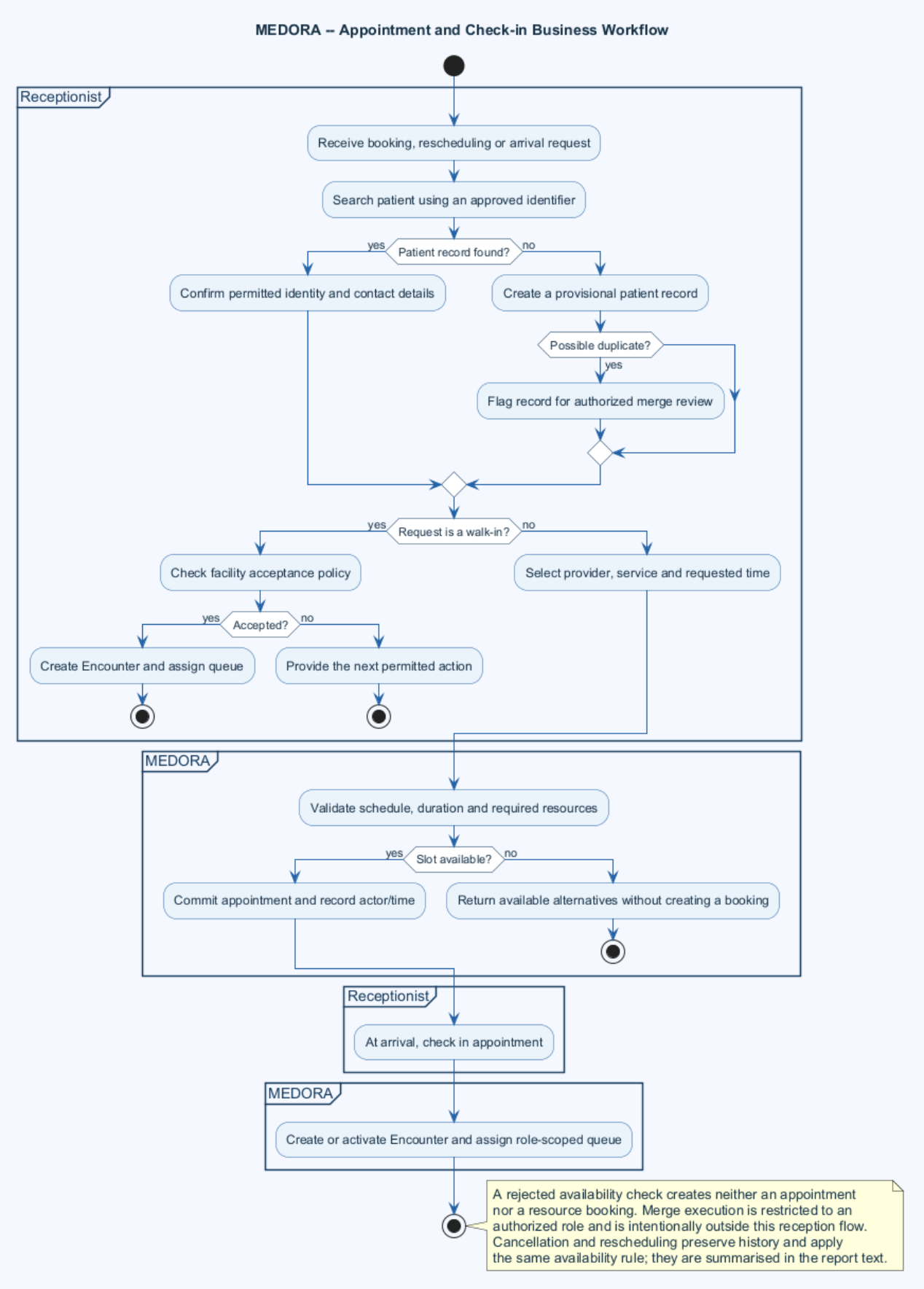


Figure 2: Appointment and check-in workflow with alternative and exception paths

Process	Completion state	Alternative or exception
Identity and booking	Confirmed patient and valid appointment.	Duplicate review or an alternative slot; no conflict is committed.
Arrival and queue	One active encounter with queue status.	Accepted walk-in or rejected duplicate active encounter.
Consultation and documentation	Structured clinical note and plan.	Draft is retained; signed content is amended, not overwritten.
Order and execution	Order, allocation and recorded result.	Unavailable resource, delay or cancellation preserves status history.
Closure and finance	Encounter summary plus invoice/payment status.	Incomplete work blocks closure; refund retains reason and history.

Table 6: Major business processes, outcomes and controlled alternatives

2.4 Business Rules

ID	Rule	Required behavior
BR-001	Patient identity	Use a stable identifier, flag possible duplicates and preserve merge history.
BR-002	Availability	Validate provider, room, staff and required resources before booking or allocation.
BR-003	Encounter integrity	Prevent duplicate active encounters and maintain role-scoped queues.
BR-004	Least-privilege action control	Enforce role, scope and state checks at the server boundary.
BR-005	Signed-note integrity	Preserve signed notes; record corrections as linked amendments with a reason.
BR-006	Bounded clinical warnings	Warnings inform clinicians without deciding treatment; an override needs a permitted role and a reason.
BR-007	Order versus execution	An order is not a result; resources are committed only for a valid allocation and execution is recorded separately.
BR-008	Prescription versus medicine event	Issuing a prescription creates no dispensing or administration event.
BR-009	Financial separation and history	Keep invoices, payments and refunds separate from clinical records; a refund is its own event that refers to the original payment and never edits it.
BR-010	Evidence of sensitive activity	Retain audit evidence of sensitive reads and writes, including merges, amendments, overrides and refunds.

Table 7: Core business rules

2.5 Domain Model and Core Entities

Figure 3 presents the clinical core and the financial chain of Invoice, Payment and Refund. The remaining operational and governance concepts extend this core without changing its central relationship: the Encounter contextualizes the clinical work performed for one Patient. A Refund refers to one original Payment and never edits it, and an Encounter has at most one Invoice.

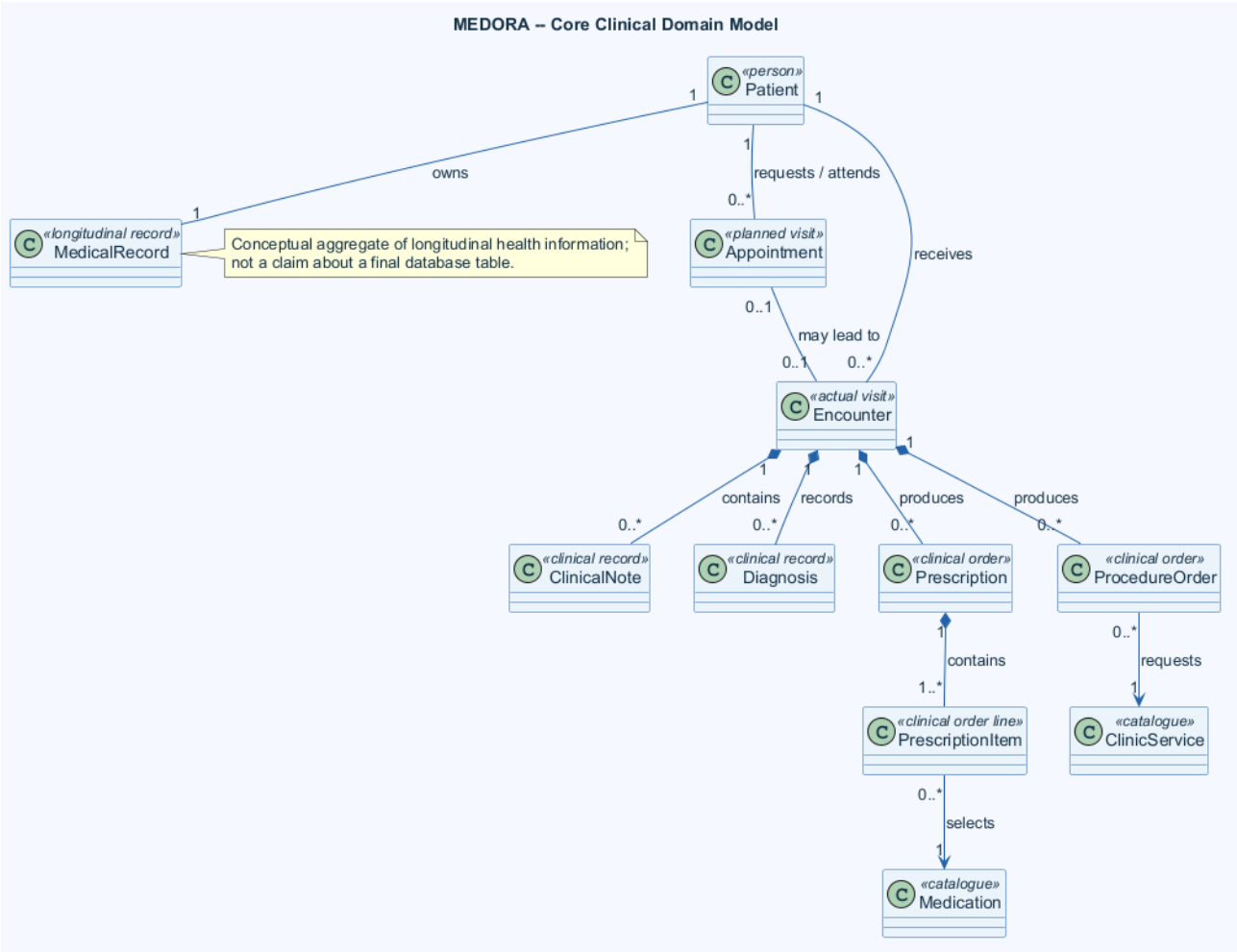


Figure 3: Core clinical and financial conceptual domain model

Entity group	Purpose
Patient and MedicalRecord	Stable identity and longitudinal clinical context.
Appointment and Encounter	Planned visit versus actual outpatient episode.
ClinicalNote, Diagnosis and Prescription	Authorized clinical documentation and decisions.
ProcedureOrder and ResourceBooking	Requested service, constrained allocation and execution context.
Invoice, Payment and Refund	Charges and financial transactions, each its own record, without clinical mutation.
UserAccount, Role, Attachment and AuditLog	Scoped access, controlled documents and accountability evidence.

Table 8: Core entity glossary

Relationship	Business meaning
Patient–Appointment–Encounter	A patient may have many appointments and encounters; an appointment may lead to one encounter, while a walk-in has none.
Encounter–clinical records	Notes, diagnoses, prescriptions and procedure orders belong to the encounter in which they are created.
Prescription–medicine event	A prescription contains medicine directions; dispensing or administration is a separate, accountable event.
ProcedureOrder–ResourceBooking–Result	An order may reserve resources and later obtain an execution result; neither is implied by the other.
Invoice–Payment	One invoice may have zero or many payment, refund or adjustment transactions.
UserAccount–Role–AuditLog	Role and scope govern access; audit evidence records the actor, action, target and time.

Table 9: Core domain relationships

Business need	System implication	Trace
Reliable identity	Search, duplicate control, merge history and longitudinal record.	BR-001; FR-003–FR-004
Conflict-free coordination	Transactional availability, queue state and resource allocation.	BR-002–BR-003; FR-005–FR-006, FR-015
Accountable care	Signed-note amendments, warnings and separate execution events.	BR-005–BR-008; FR-009–FR-018
Financial accountability	Invoice, payment and refund history separate from clinical data.	BR-009; FR-022–FR-023
Governance	Least privilege, audit trail, reports and controlled export.	BR-004, BR-010; FR-026–FR-030

Table 10: Business-to-system traceability

3 System Requirements Analysis

3.1 Functional Decomposition

Figure 4 groups MEDORA by business capability rather than user interface. It provides the functional boundary for the requirements and use-case model.
Identity and Access provides the stable patient and user context on which all other capabilities rely. Scheduling and Reception controls planned appointments, walk-ins, queues and resource availability. Clinical Care records the actual encounter; Orders and Execution preserves the distinction between a decision and the action carried out. Finance and Operations handles eligible charges and operational visibility, while Governance applies cross-cutting controls for attachments, audit evidence and exports. The figure shows grouping only; it draws no dependencies between the groups.

3.2 Actors and Overall Use Case Model

The overall use-case model in Figure 5 gives a bounded view of actor goals. Patient self-service is explicitly Phase 2; in the MVP, reception staff perform appointment and check-in work on the patient's behalf. No insurer, payment gateway, laboratory information system, imaging system or messaging provider is shown as an actor because no approved integration contract exists.

3.3 Use Case Diagrams by Functional Group

The focused diagrams below decompose the overview without changing the actor boundary. Solid associations identify participating external roles. A dashed `«include»` relationship is mandatory reusable behavior; `«extend»` is conditional behavior that leaves the base use case valid. UC-054 (operational notification) and UC-055 (released-document viewing) belong to P1 and Phase 2. The sources and mappings are maintained in the model inventory under `doc/asset/03-system-requirements/use-case-model`.

3.3.1 Patient Management

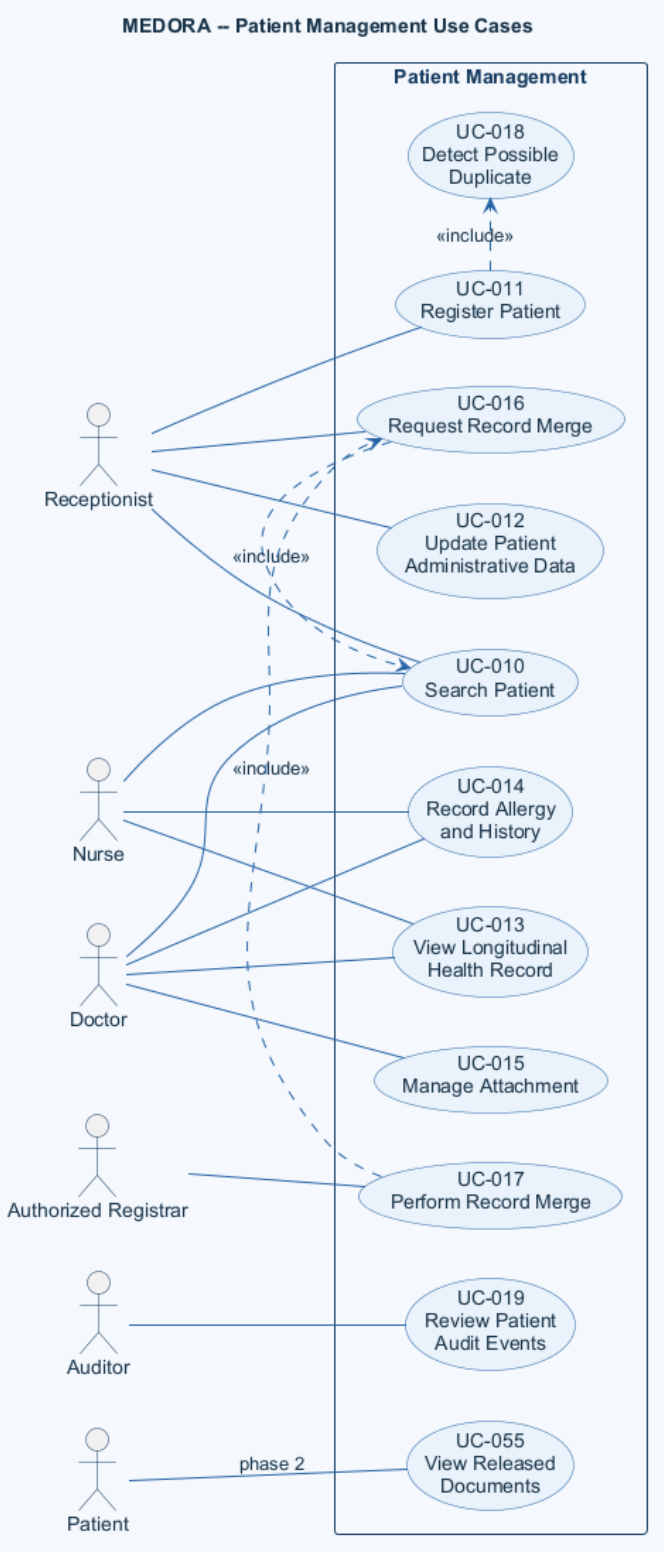


Figure 6: Patient-management use cases

3.3.2 Appointment, Check-in and Queue Management

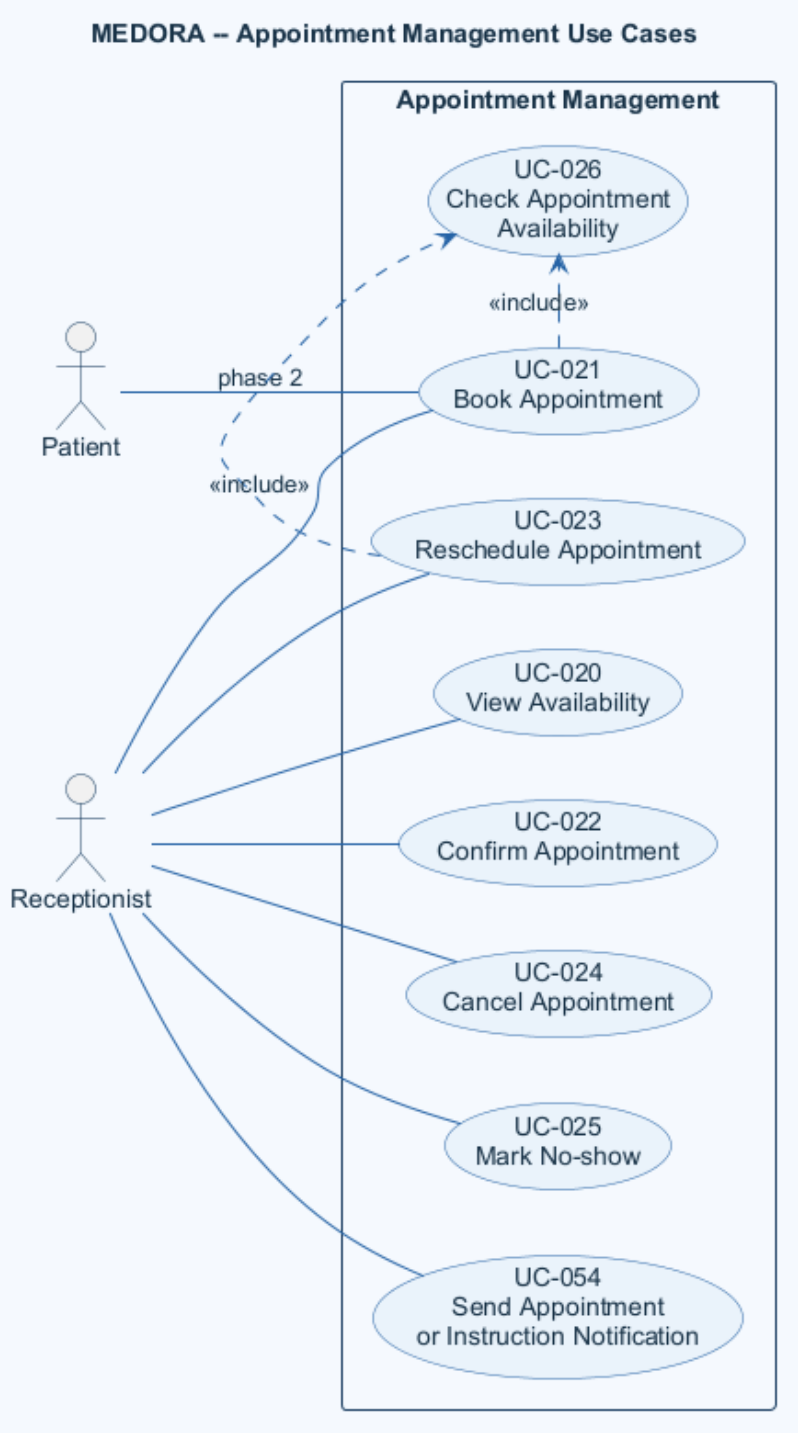


Figure 7: Appointment-management use cases

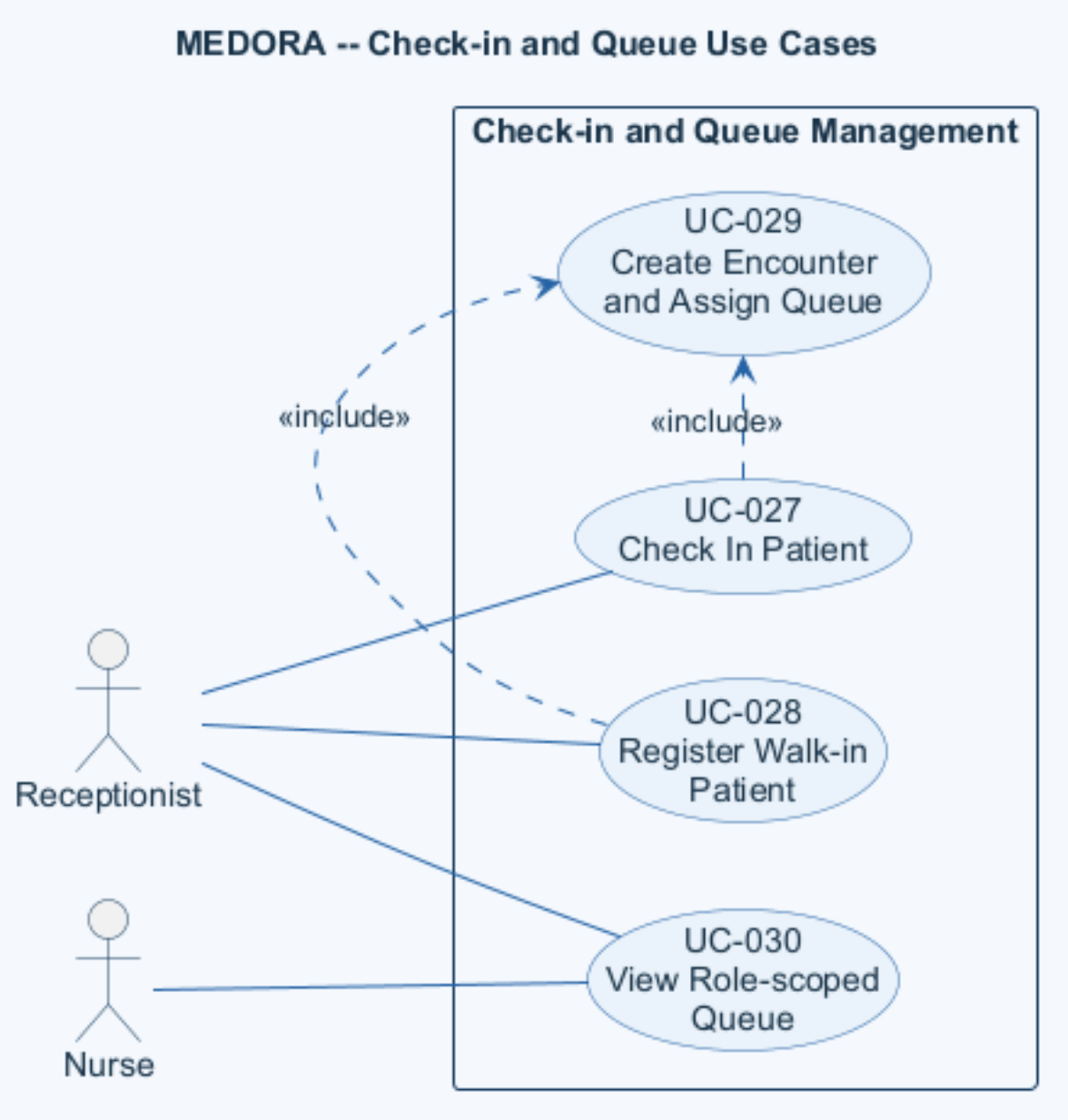


Figure 8: Check-in and queue-management use cases

3.3.3 Medical Examination and Encounter Management

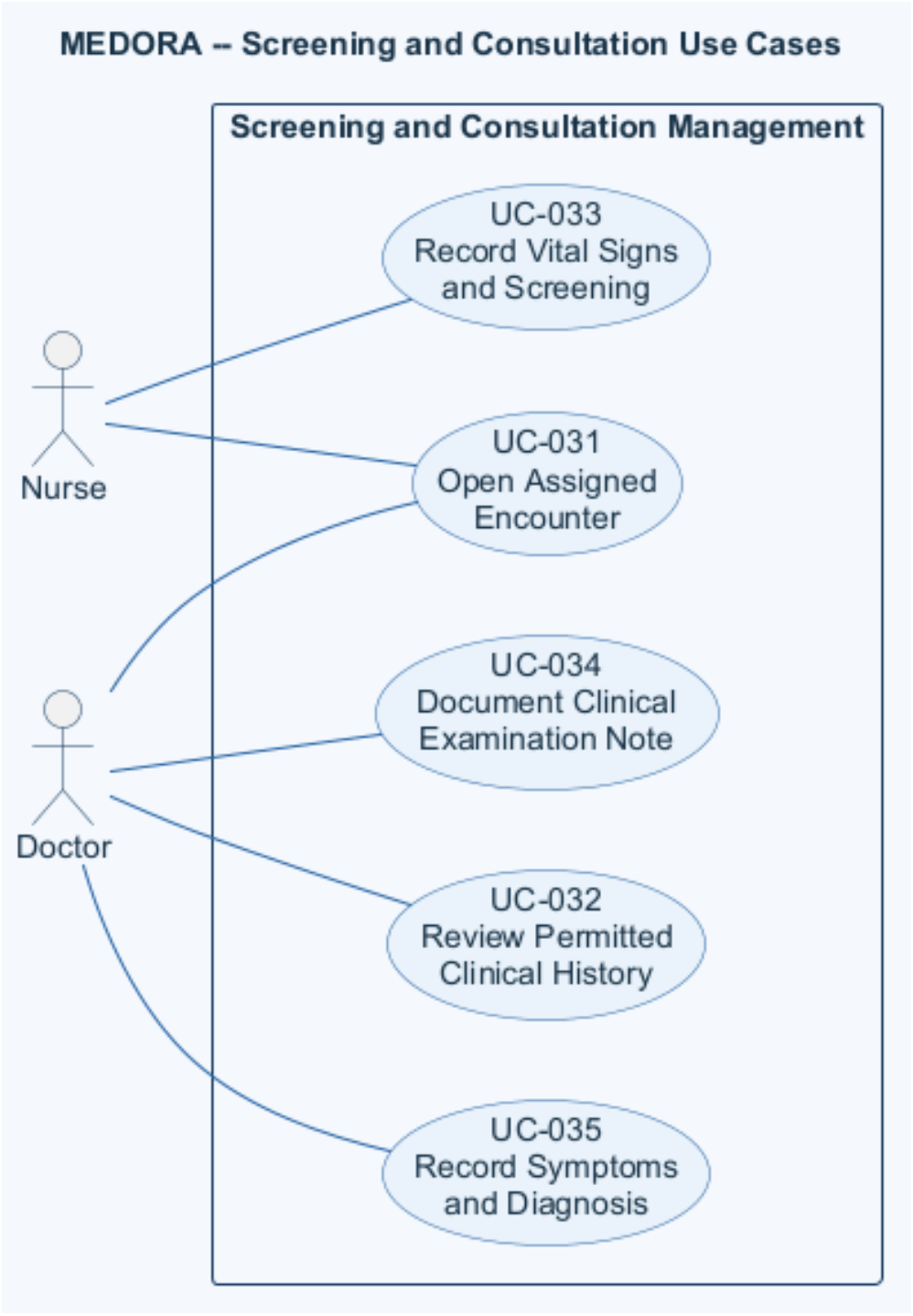


Figure 9: Screening and consultation use cases

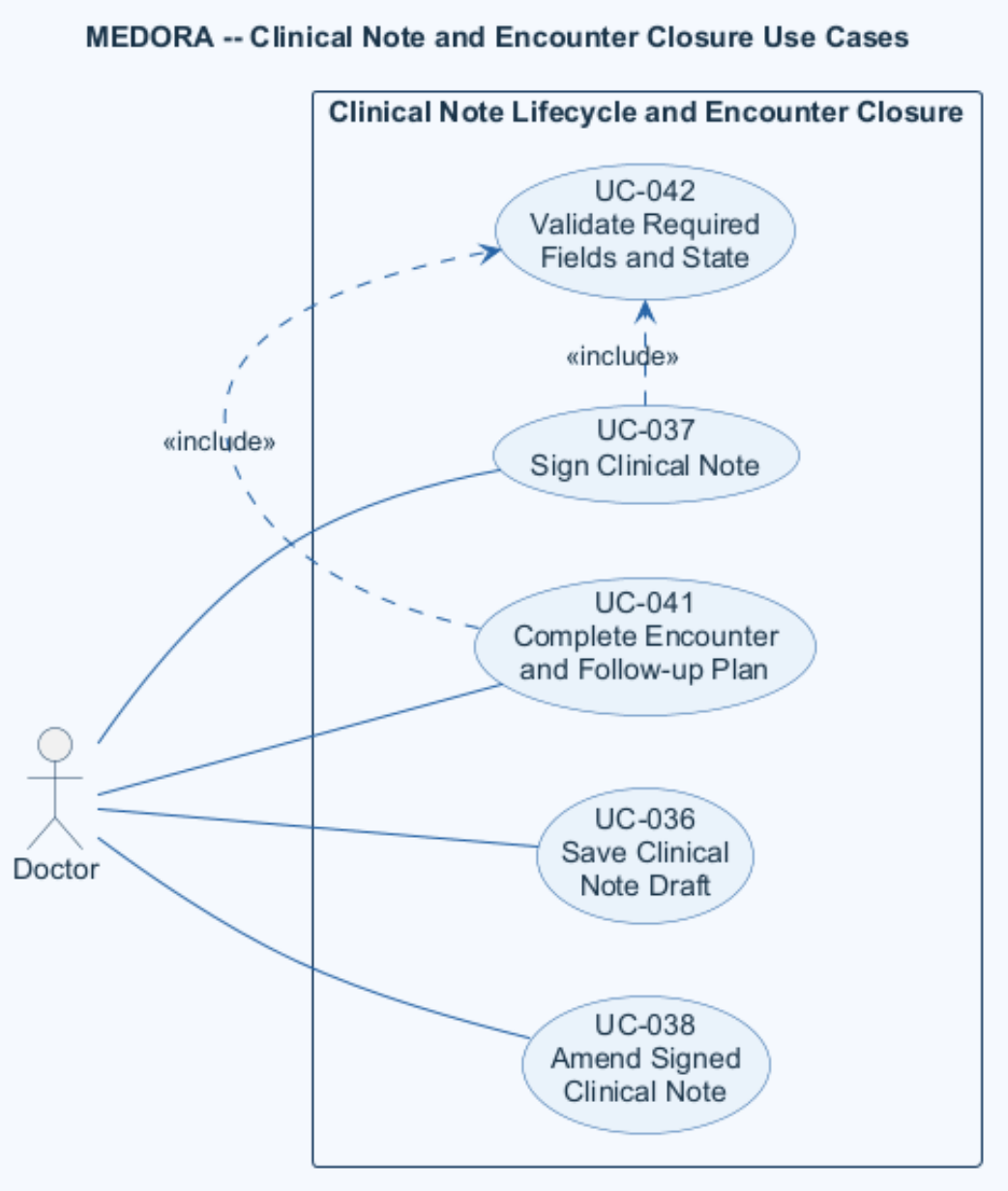


Figure 10: Clinical-note lifecycle and encounter-closure use cases

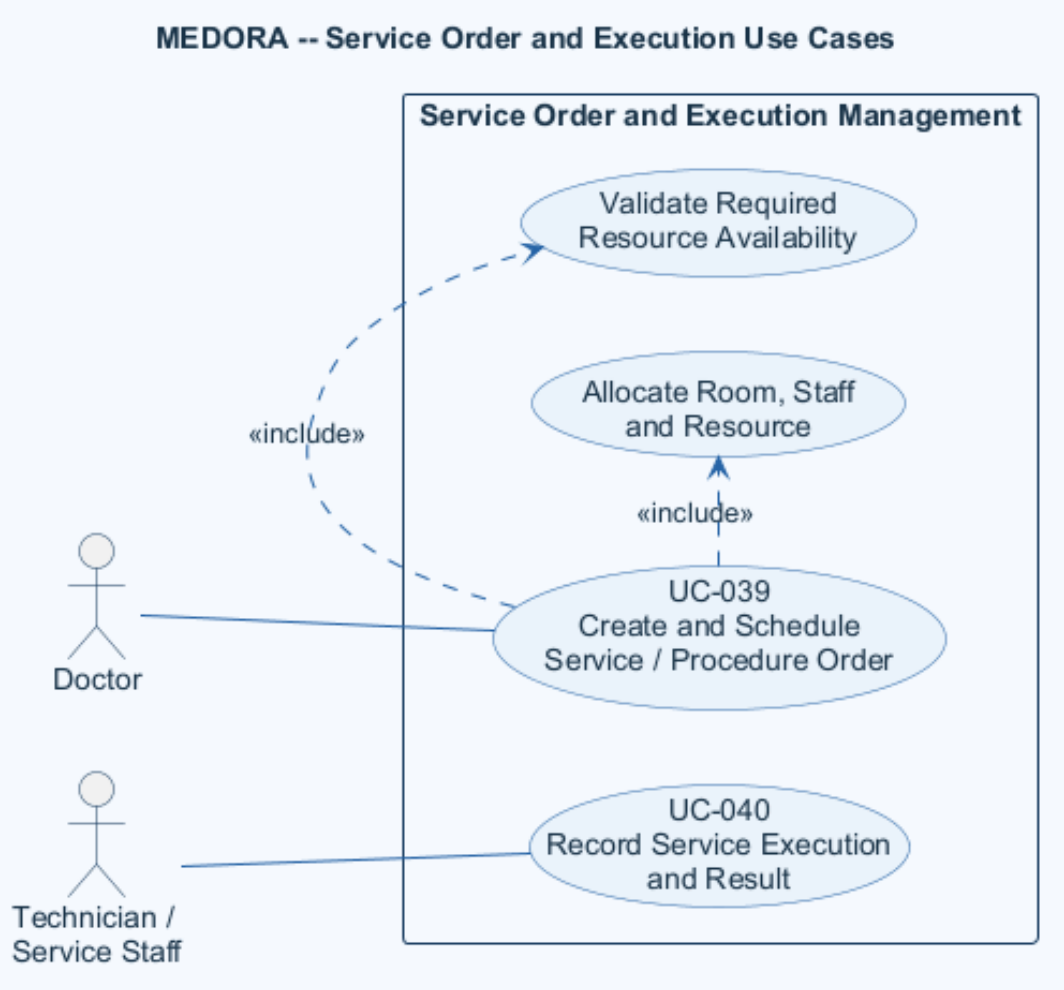


Figure 11: Service-order and execution use cases

3.3.4 Prescription, Billing and Governance

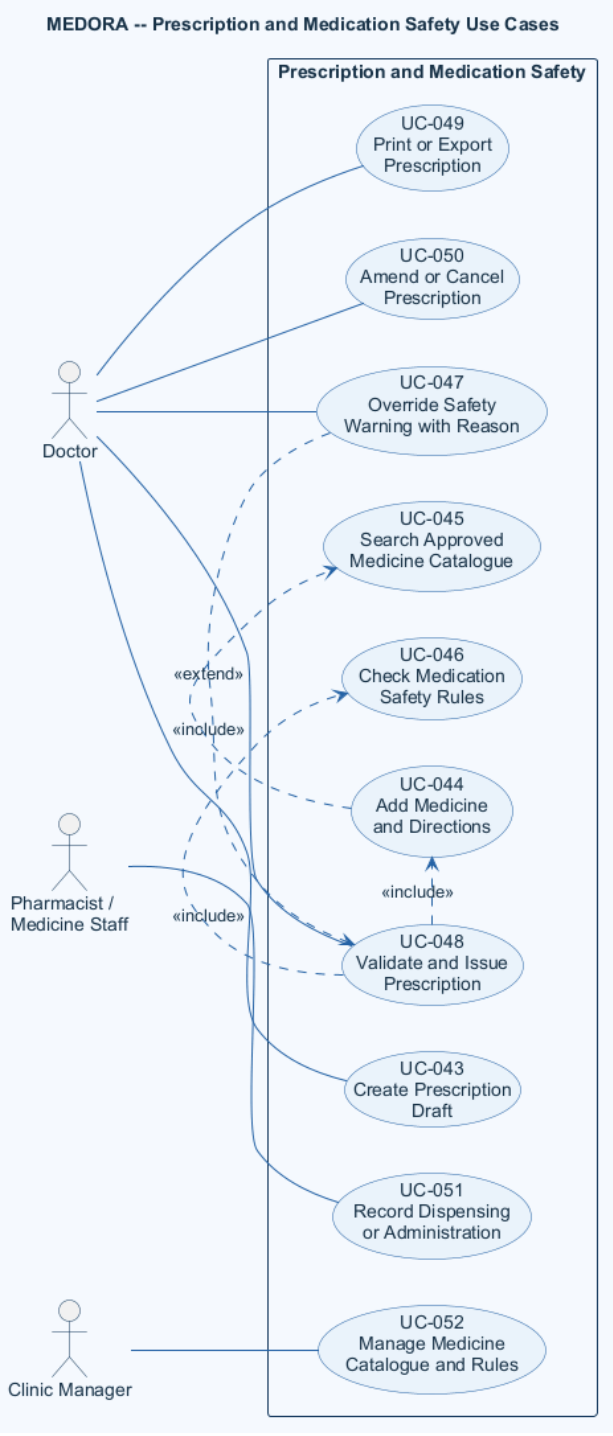


Figure 12: Prescription and medication-safety use cases

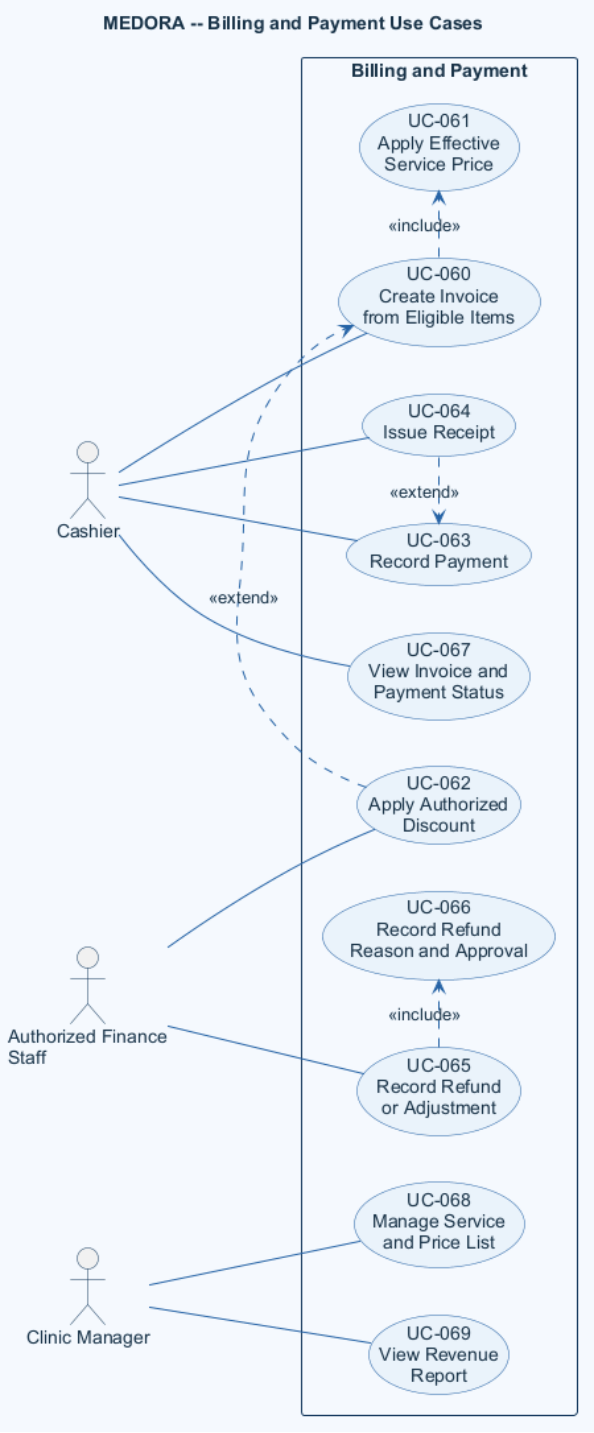


Figure 13: Billing and payment use cases

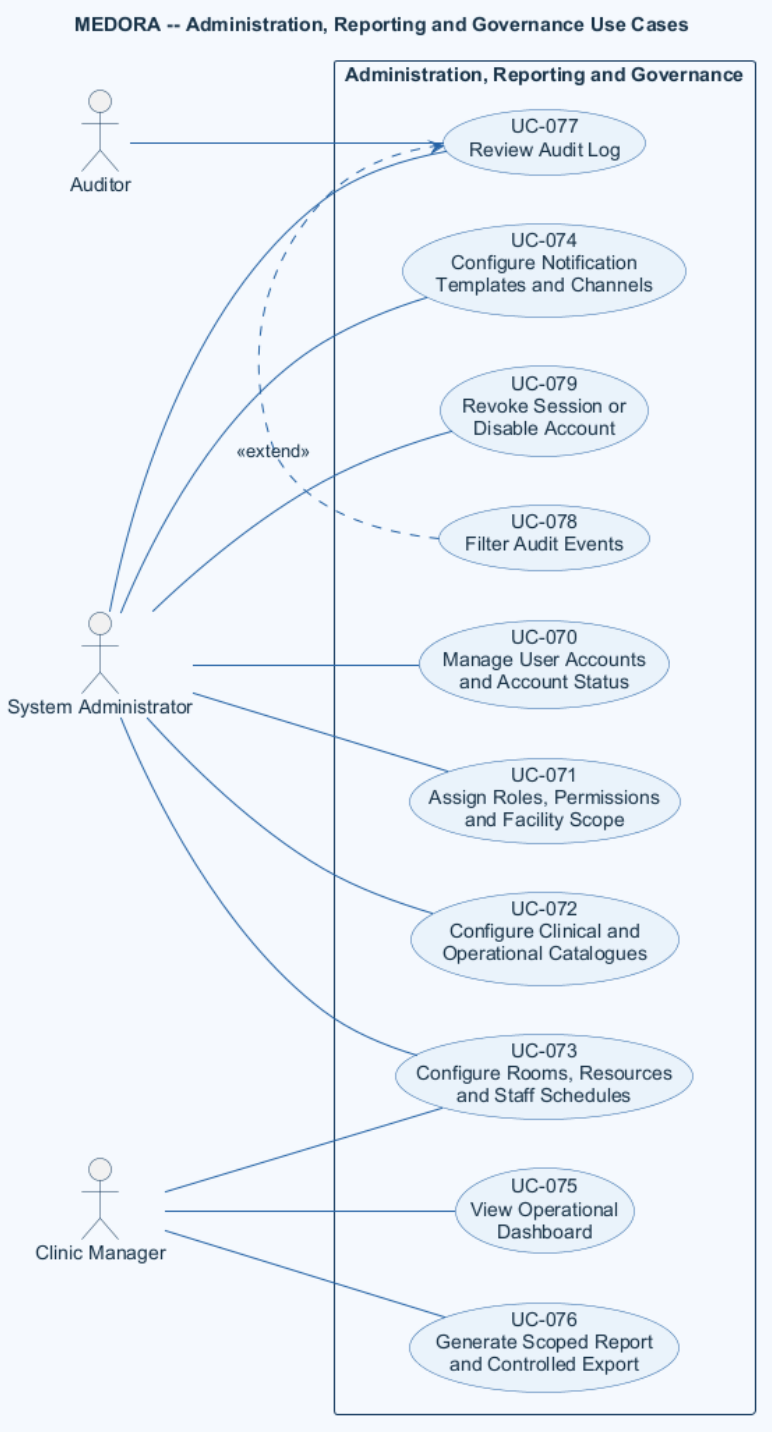


Figure 14: Administration, reporting and governance use cases

3.4 Detailed Use Case Specifications

The six use cases below carry the principal state, safety or financial risk. Each specification states the minimum successful path and the control that prevents an invalid state transition.

3.4.1 UC-021 Book Appointment

Item	Specification

Actor, trigger and preconditions	Receptionist; a patient and service/provider context are selected; the actor is authorized.
Success flow	Select slot; validate provider, room, duration and required resources; atomically create the Appointment and audit-relevant metadata.
Alternative and exception flow	Rescheduling repeats validation and preserves history. An unavailable slot, unresolved identity or invalid scope creates no appointment.
Postconditions and data	One valid Appointment exists; no Encounter is created by booking alone.
Rules and trace	BR-002, BR-004; FR-005, FR-017; UC-026; NFR-06, NFR-15.

3.4.2 UC-027 Check In Patient

Item	Specification
Actor, trigger and preconditions	Receptionist receives an identified booked patient or an accepted walk-in.
Success flow	Confirm arrival; check the active-encounter rule; create/activate the Encounter; assign the role-scoped queue state.
Alternative and exception flow	An accepted walk-in follows the same queue rule. A conflicting active encounter, rejected walk-in or invalid scope creates no duplicate encounter.
Postconditions and data	The relevant Appointment is updated and one active Encounter with queue status is available to authorized roles.
Rules and trace	BR-003, BR-004; FR-006; UC-029; NFR-06, NFR-15.

3.4.3 UC-037/UC-038 Sign and Amend a Clinical Note

Item	Specification
Actor, trigger and preconditions	Authorized Doctor; a draft note belongs to an assigned Encounter.
Success flow	Validate required content and state; sign the note with actor/time. A correction creates a linked amendment with a reason.
Alternative and exception flow	A draft may be saved for later. Missing fields, invalid state, invalid scope or a concurrent update prevents signing or requires reconciliation.
Postconditions and data	Signed content remains intact; each amendment retains author, time, reason and linkage.
Rules and trace	BR-004, BR-005; FR-009, FR-020, FR-032; UC-042; NFR-06, NFR-07, NFR-15.

3.4.4 UC-048 Validate and Issue Prescription

Item	Specification
Actor, trigger and preconditions	Authorized Doctor requests issue for a valid encounter with complete medicine items.
Success flow	Validate dose and directions; evaluate configured safety rules; address a warning or record a permitted override; issue the prescription.
Alternative and exception flow	The Doctor may retain a draft. Missing fields, an unresolved required warning or invalid scope blocks issue; unavailable source data does not create an invented warning.
Postconditions and data	An issued prescription and item history exist; no dispensing or administration event is inferred.
Rules and trace	BR-006, BR-008; FR-011–FR-013; UC-046–UC-047; NFR-06, NFR-07.

3.4.5 UC-039/UC-040 Service Order, Allocation and Result

Item	Specification
Actor, trigger and preconditions	Doctor creates an order for an active encounter; assigned Technician/Service Staff records execution.
Success flow	Record service, reason and priority; validate/allocate required room, staff and resource; record performer, time, status and result.
Alternative and exception flow	An order may be scheduled later or remain pending. Unavailable resources, unassigned staff, delay or cancellation preserve a valid non-completed state.
Postconditions and data	ProcedureOrder, optional ResourceBooking and execution/result history are linked to the Encounter.
Rules and trace	BR-002, BR-004, BR-007; FR-014–FR-018; NFR-06, NFR-15.

3.4.6 UC-063/UC-065 Record Payment and Refund

Item	Specification
Actor, trigger and preconditions	Cashier records payment for an eligible Invoice. Authorized Finance Staff submits a refund request for an identifiable original transaction, amount and reason.

Success flow	For payment, validate invoice state, amount, method and request key; record one payment event and return the transaction reference and invoice status. For a refund, validate the original transaction, amount, authority and policy; record a separate refund event only when the request is authorized and permitted.
Alternative and exception flow	Partial payment leaves an outstanding balance. An invalid payment/refund request, invalid scope or duplicate submission is rejected with no financial effect. A valid refund that requires approval returns a pending-approval status; it is neither rejected nor completed and no resulting balance is reported.
Postconditions and data	A successful payment or completed refund updates its own history and the resulting invoice status. The original transaction is retained and clinical content is unchanged.
Rules and trace	BR-004, BR-009, BR-010; FR-022–FR-023; UC-064, UC-066; NFR-06, NFR-07, NFR-15.

3.5 Functional Requirements

Table 17 groups related requirements without changing their identifiers. P0 is required for the MVP; P1 is a controlled extension; P2 depends on a later organizational decision or integration contract.

Table 17: Functional requirements catalog

ID	Capability	Observable system behavior	Priority
FR-001–FR-002	Authentication and access	Authenticate individual accounts; manage roles, facility scope and sensitive-session controls; enforce authorization at the server boundary.	P0
FR-003–FR-004	Patient identity	Create/search patient profiles, flag duplicates and preserve authorized merge evidence.	P0/P1
FR-005–FR-006	Appointment and reception	Manage appointment status, availability, walk-ins, check-in and role-scoped queues.	P0
FR-007–FR-010	Encounter documentation	Record screening, history, clinical notes, symptoms, diagnoses and follow-up in encounter context.	P0
FR-011–FR-013	Prescription and medicine event	Validate configured safety rules; issue prescriptions; record dispensing/administration separately.	P0/P1
FR-014–FR-018	Service order and execution	Create orders; allocate rooms, staff and resources; record execution status and result.	P0
FR-019	Diagnostic request	Support an approved manual request/result attachment or later adapter.	P1
FR-020–FR-021	Encounter conclusion	Record conclusion and follow-up; support configured leave/discharge review where applicable.	P0/P1
FR-022–FR-023	Catalog, invoice and payment	Maintain effective service/price data; create invoices and retain payment/refund history.	P0
FR-024–FR-025	Notification and patient portal	Send configured operational notifications; support verified portal self-service only when enabled.	P1 / Phase 2
FR-026–FR-027	Dashboard, reports and export	Present scoped operational indicators and log controlled report exports.	P0
FR-028–FR-031	Administration, attachment and search	Maintain reference catalogs, controlled attachments, audit evidence and scoped search.	P0
FR-032	Draft and recovery	Preserve drafts and unsaved-change state; report success only after server confirmation.	P1

ID	Capability	Observable system behavior	Priority
FR-033–FR-034	Facility and external integration	Apply approved facility scope and provide logged connectors only for approved contracts.	P2

3.6 Non-functional Requirements

The table defines the quality targets used for design and later verification.

Table 18: Non-functional requirements catalog

ID	Attribute	Requirement / acceptance basis	Status
NFR-01–NFR-03	Performance, availability and recovery	Target p95 ordinary screen $\leq$ 2 seconds, search $\leq$ 3 seconds; availability, RPO and RTO are still to be defined before deployment.	Target
NFR-04–NFR-07	Security and auditability	Use protected transport/storage, server-side RBAC, least privilege, time-synchronized sensitive-event evidence and retention policy.	Specified
NFR-08–NFR-09	Resilience and scale	Preserve confirmed data through dependency failure; use bounded queries, pagination and modular APIs.	Specified
NFR-10–NFR-11	Compatibility and accessibility	Support current Chrome/Edge desktop/tablet use with keyboard operation, labels, contrast and clear errors.	Target
NFR-12–NFR-13	Maintainability and observability	Use controlled migrations, documentation and structured operational metrics/logs.	Specified
NFR-14–NFR-15	Locale and integrity	Configure Vietnamese/local formats; use transactions, constraints and idempotency for critical operations.	Specified
NFR-16	Testability	Maintain unit, integration and end-to-end tests with non-production data.	Specified

3.7 Role-based Access Control Matrix

Table 19 records the intended minimum business permissions. It is a requirements baseline, not an implementation claim. **R**, **C**, **U**, **S**, **A** and **F** respectively mean read, create, update, sign/finalize, approve/override and finance transaction. A dash means no routine access. All non-dash permissions remain subject to facility, assignment and lifecycle state, and the server must enforce them at its boundary.

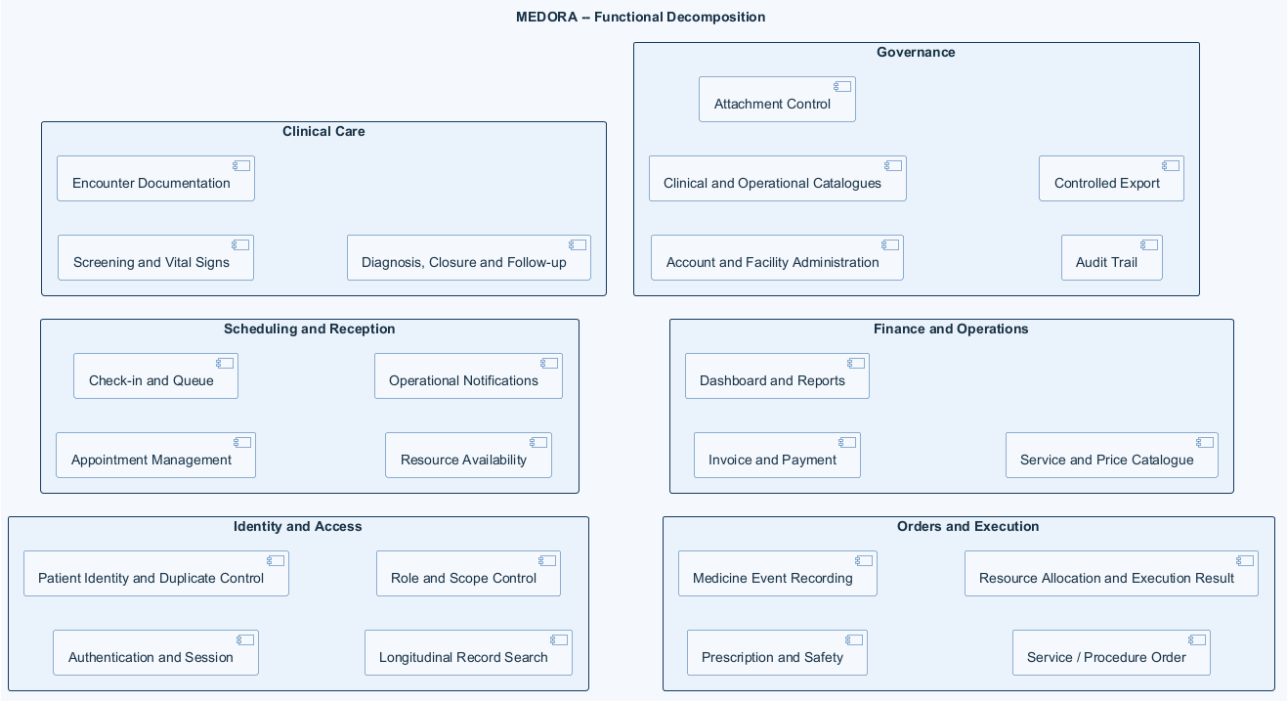


Figure 4: Functional decomposition of MEDORA

Clinical and front-desk roles

Capability / data domain	Rec.	Nurse	Doctor	Tech.	Pharm.
Patient administrative profile	R/C/U	R	R	R†	R†
Patient record merge[c]	C (request)	–	–	–	–
Appointment, check-in and queue	R/C/U	R	R	R†	–
Screening and vital signs	R	C/U	R	R†	–
Clinical note and diagnosis	R†	R†	R/C/U/S	R†	R†
Prescription and medicine event	R†	R†	R/C/U/S/A	–	R/C/U
Service order and execution result	R†	R/C/U†	R/C/U/S	R/C/U	–
Invoice, payment and refund	R†	–	R†	–	–
Dashboard and scoped reports	–	R†	R†	R†	–
User, role and catalog configuration	–	–	R†	–	R†
Audit log and controlled export	–	–	R†	–	–

Finance, governance and patient roles

Capability / data domain	Cash.	Finance	Manager	Admin	Auditor	Patient*
Patient administrative profile	R†	R†	R†	–	R†	R/U
Patient record merge[c]	–	–	–	–	R†	–
Appointment, check-in and queue	R†	–	R	–	R†	R/C/U
Screening and vital signs	–	–	R†	–	R†	–
Clinical note and diagnosis	–	–	R†	–	R†	R‡
Prescription and medicine event	–	–	R†	–	R†	R‡
Service order and execution result	–	–	R†	–	R†	R‡
Invoice, payment and refund	R/C/U/F[a]	R/C/A/F[b]	R/A†	–	R†	–
Dashboard and scoped reports	R†	R†	R	–	R†	–
User, role and catalog configuration	R†	–	R/U†	R/C/U/A	R†	–
Audit log and controlled export	R†	R†	R†	R†	R	–

Table 19: Target role-based access-control matrix

*Phase 2 only. †Only within assigned, facility or approved scope. ‡Only information expressly released through the portal. [a]Payment transactions only; Cashier cannot request, approve or execute a refund. [b]Refunds only after validation and any required approval; pending approval is not a completed refund, and the approver is a Manager or Authorized Finance Staff member other than the requester. [c]Merge is performed only by the Authorized Registrar (a role without its own column); other roles may not execute a merge. Administrative configuration does not confer clinical-edit authority; Auditor access is read-only and itself may be logged.

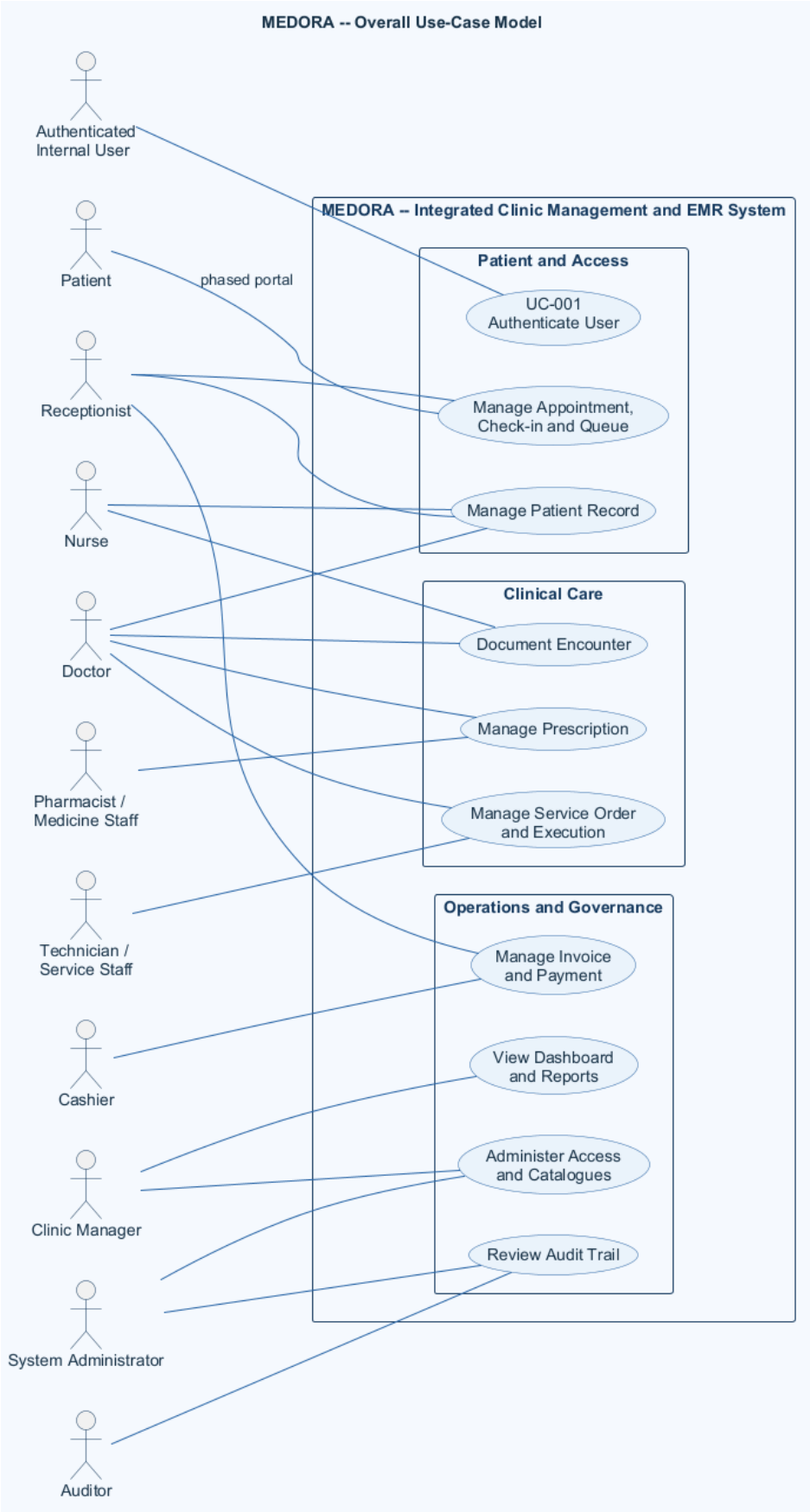


Figure 5: Overall use-case model of MEDORA

3.8 Requirements Traceability and Use Case Validation

Table 20 links the business rationale to requirements, use cases, entities and planned verification.

Table 20: Requirements traceability and planned verification

Business rule/need	Requirement	Use cases	Domain/design focus	Planned verification
BR-001 identity continuity	FR-003, FR-004, FR-031	UC-010–UC-018	Patient, MedicalRecord, AuditLog; unique identifier and merge history.	Duplicate-search, unauthorized-merge and merge-history test cases.
BR-002 conflict-free allocation	FR-005, FR-015–FR-017; NFR-15	UC-021–UC-026, UC-039	Appointment, ResourceBooking, schedule/room/staff constraints.	Concurrent booking/reschedule tests; zero accepted conflict target.
BR-003 controlled arrival	FR-006	UC-027–UC-030	Appointment–Encounter relationship and queue state.	Duplicate-active-encounter and role-scoped queue tests.
BR-005 signed-record integrity	FR-009, FR-032; NFR-07, NFR-15	UC-036–UC-038, UC-041–UC-042	ClinicalNote draft/signed/amendment lifecycle.	Missing-field, amendment and concurrent-edit tests.
BR-006 bounded warning	FR-011, FR-012	UC-043–UC-048	Prescription, PrescriptionItem, Medication and override evidence.	Rule-trigger, override-reason and no-autonomous-decision tests.
BR-007 accountable execution	FR-014–FR-018	UC-039–UC-040	ProcedureOrder, ResourceBooking and execution result.	Assignment, unavailable-resource, delay/cancel and result tests.
BR-008 separate medicine event	FR-012, FR-013	UC-048–UC-051	PrescriptionItem versus MedicationAdministration.	Verify issue alone creates no administration event.
BR-009 financial history	FR-022, FR-023; NFR-15	UC-060–UC-069	Invoice, Payment and refund/adjustment history.	Partial-payment, duplicate-submission and refund-reason tests.
BR-004 / BR-010 governance	FR-001, FR-002, FR-027, FR-030; NFR-06, NFR-07	UC-001, UC-070–UC-079	UserAccount, Role, AuditLog and controlled export.	Negative API authorization and audit-completeness tests.
Operational visibility	FR-026, FR-027	UC-075–UC-076	Scoped measures, report filters and export audit.	Role-scope/report-reconciliation tests.

Open decisions. The clinic still has to confirm clinical-warning sources, consent and retention policy, refund approval thresholds, branch scope and portal publication rules before deployment.

4 System Design and Structural Modeling

This section presents a proposed design for MEDORA, bounded to the outpatient clinic workflows and requirements in the WorkStation brief. The Smart Hospital Management System project informs the organization of the UML views and the Boundary–Control–Entity (BCE) separation; its inpatient wards, bed allocation, and hospital-wide departments are outside MEDORA's scope. The hospital UML example is used only to review how use-case, class,

and sequence views can support one another. These references do not establish MEDORA features or implementation status [saipraveen k, 2026; Rodina, 2024]. All architecture and data models below are design proposals; the hosting platform, physical schema, and running application have not been verified.

4.1 System Architecture

The logical baseline is a clinic-focused, modular application. Role-specific workspaces submit requests to application workflows; controllers validate identity, role, scope, and current business state; domain entities represent patient, appointment, encounter, clinical, and financial records; and repositories provide a controlled persistence boundary. This separation supports the report's requirements for server-side authorization, conflict-free scheduling, signed-record integrity, and accountable financial events. It does not assume a microservices deployment or an external payment, messaging, or clinical decision service.

4.1.1 Logical Architecture

Figure 15 summarizes representative patient-registration, consultation, and billing flows. The Boundary layer collects user input, the Control layer coordinates the workflow, and the Entity layer holds the affected domain records. The package view later in this section groups the broader functional areas, and the component view shows their application dependencies.

4.1.2 Deployment Architecture

The deployment view in Figure 16 is a logical baseline: a clinic workstation or tablet accesses the web application, which uses a separate persistence connection to a relational data store. HTTPS/TLS follows the transport-security requirement. The diagram deliberately leaves the hosting provider, server count, network zones, database engine, backup topology, and recovery configuration open; none is established by the available implementation evidence.

4.2 Layered Design and Design Patterns

The design applies BCE at the logical boundary and separates the application into presentation, workflow, domain, and data-access responsibilities. The main design choices are summarized in Table 21.

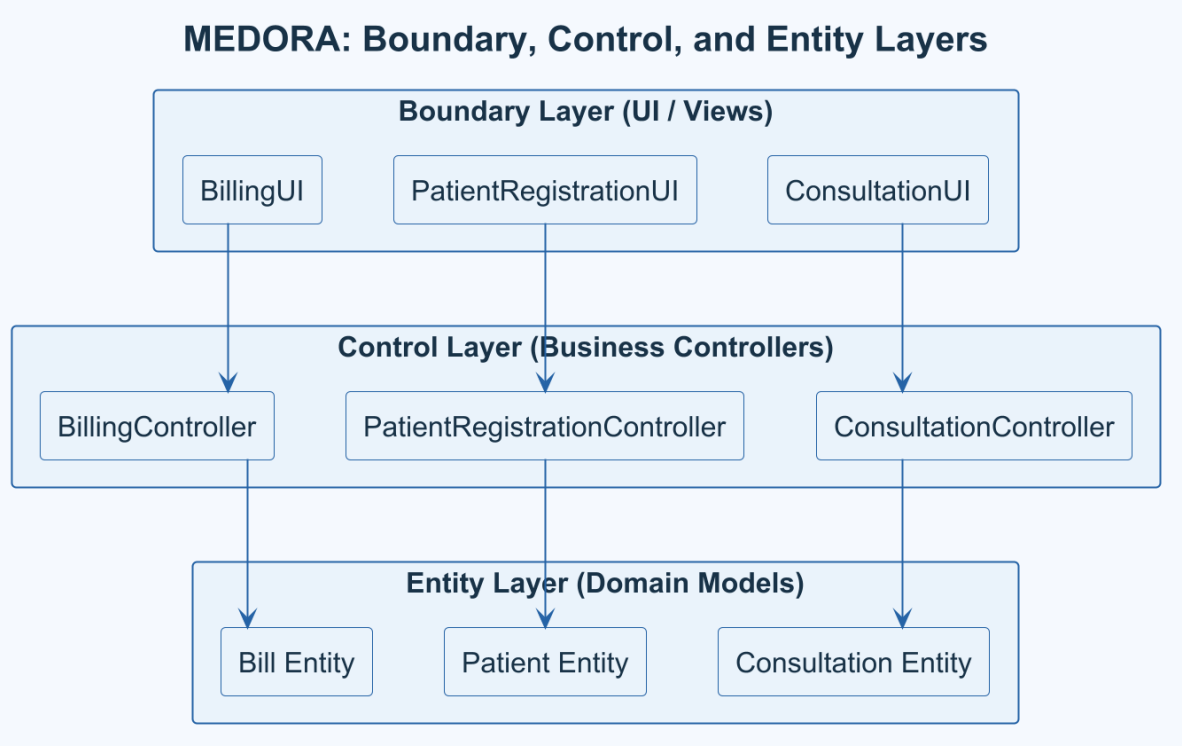


Figure 15: Proposed BCE logical architecture for representative MEDORA clinic workflows

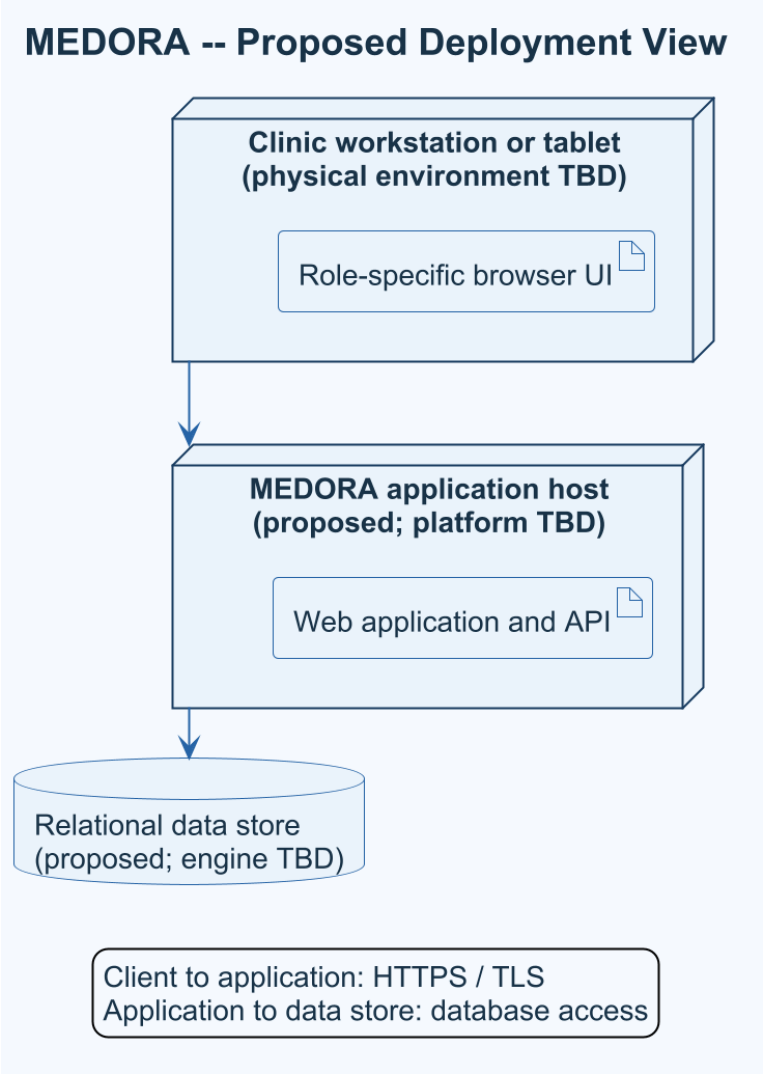


Figure 16: Proposed logical deployment baseline; physical infrastructure remains to be selected

Design choice	MEDORA responsibility and boundary
Boundary–Control–Entity	Screens and adapters collect requests; controllers coordinate use cases; entities represent the records and states those use cases change.
Modular application	Identity, scheduling, clinical care, orders, finance, and governance remain separable modules within one proposed application. No microservice boundary is assumed.
Service/controller layer	Enforces access scope and lifecycle rules, coordinates related changes, and returns a clear outcome to the caller.
Repository boundary	Keeps persistence queries and transactions out of screens and workflow-specific code. Repository interfaces are proposed design seams, not verified source modules.
Record integrity	Signed clinical-note amendments, payments, and refunds are appended as traceable changes rather than silently overwriting prior evidence.

Table 21: Proposed architecture and design choices

4.2.1 Presentation, Business Logic, and Data Access Layers

The presentation layer contains role-specific workspaces for reception, clinicians, finance, and administration. The authenticated application boundary checks identity and permissions on the server. Workflow controllers coordinate booking, check-in, encounter documentation, service orders, and financial operations. Domain rules validate appointment availability, encounter state, signed-note versioning, and payment/refund relationships. Repository interfaces then read or update persistent records through transactions. User interfaces do not access the data store directly.

The existing functional decomposition in Figure 4 supplies the module vocabulary. Figure 26 refines it into proposed components and their dependencies. The component boundaries are logical; they do not claim that corresponding software modules have already been implemented.

4.2.2 API and Web Service Design

The WorkStation brief calls for authenticated, versionable APIs, server-side authorization, bounded queries, and transaction-safe updates. The current report contains representative request and data examples in Appendix C; they are design examples rather than captured responses from a running endpoint. Table 22 therefore describes interface responsibilities without inventing URL paths, HTTP methods, or an API contract.

Interface area	Proposed responsibility and required control
Identity and patient records	Search and maintain identity under role and record scope; prevent unauthorized access and retain merge history.
Scheduling and reception	Check availability, create or change appointments, and record check-in and queue state without accepting conflicting allocations.
Encounter and documentation	Link an actual visit to its patient and optional appointment; validate note completion and preserve signed versions.
Orders and results	Link prescriptions and service orders to the encounter; represent execution and results separately from the request.
Invoice and payment	Issue an invoice from eligible charges, support separate payment events, and keep refunds linked to the original payment.
Governance and reporting	Apply scoped access, record sensitive actions, and control report and data exports.

Table 22: Proposed API responsibility groups; endpoint contract not yet specified

Every state-changing request should validate the caller's role and scope, re-check the current record state, and use a transaction for related updates. Duplicate-sensitive operations such as booking and payment should also use the idempotency and conflict controls specified by the requirements. Detailed payload fields, error codes, API versioning, and authentication/session protocols remain design decisions.

4.3 Database Design and Data Integrity

The WorkStation brief defines a conceptual data model and explicitly leaves the physical schema, key types, and storage choices for design. Figures 17–20 therefore show logical entities and candidate key relationships, not a deployed database schema. The model is split into patient and visit identity, encounter documentation and orders, and financial transactions to keep each relationship view readable.

4.3.1 Entity–Relationship Diagrams

The patient and visit ERD distinguishes a planned appointment from an actual encounter. A walk-in encounter may have no appointment, while every encounter remains linked to its patient. Each patient has one longitudinal MedicalRecord, represented in the persistent mapping table because it is outside this focused view.

The clinical record ERD keeps notes and diagnoses linked to the encounter. The companion orders ERD shows prescriptions and service orders, with prescription items linked to medication catalog entries and procedure orders linked to clinic services.

The financial ERD separates the invoice from its line items and payment events. An encounter may have at most one invoice; an invoice can receive multiple payments; each refund refers to its original payment. This supports partial settlement and preserves the transaction history.

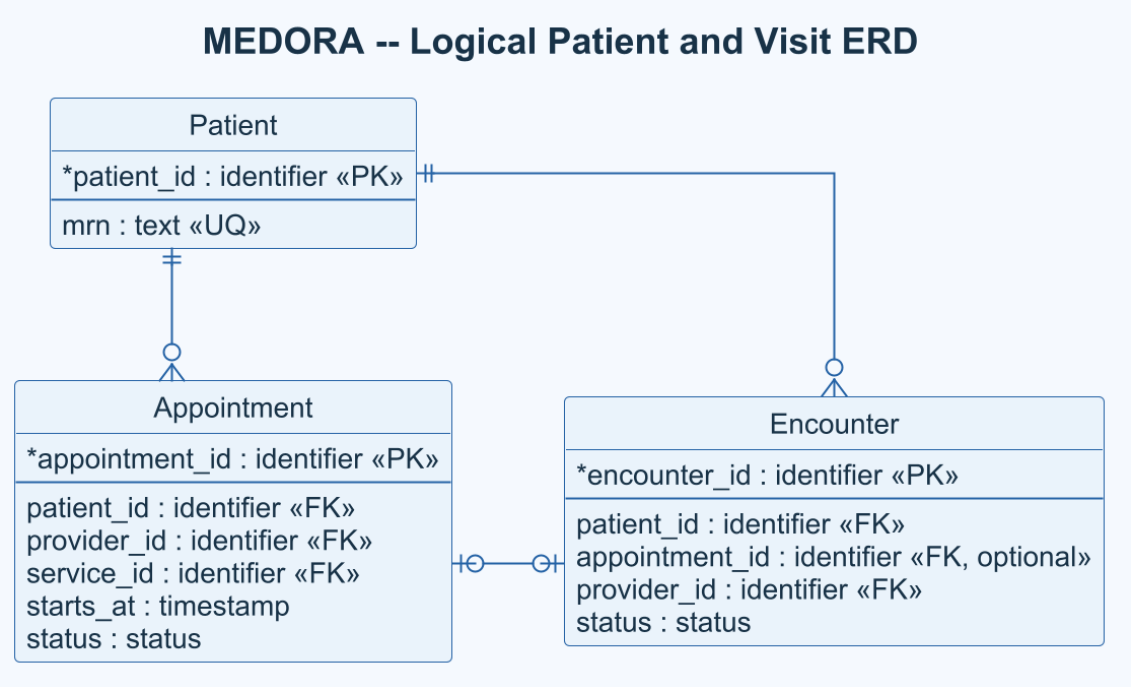


Figure 17: Logical ERD for patient identity, appointments, and encounters

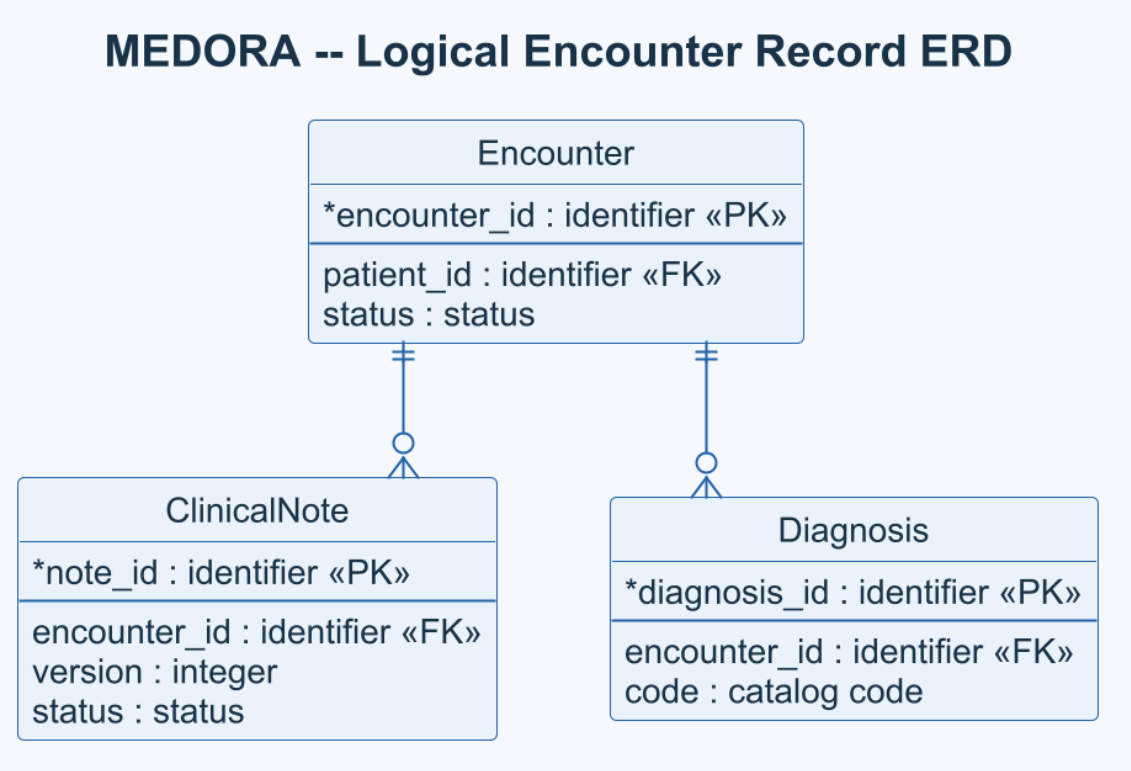


Figure 18: Logical ERD for encounter notes and diagnoses

4.3.2 Relational Schema and Normalization

A relational implementation should keep identity, scheduling, clinical documentation, orders, and financial transactions in separate relations joined by stable keys. Repeating groups such as diagnosis lists, prescription lines, invoice lines, and role assignments belong in child or join relations rather than duplicated columns. This supports a normalized design for updateable records. Historical snapshots that preserve the meaning of a signed note or an issued charge should be retained deliberately and linked to their source record.
The logical schema should prevent duplicate patient identifiers, overlapping provider/resource bookings, multiple active encounters for the same appointment, and duplicate payment submissions. Database transactions and uniqueness or exclusion constraints should reinforce the corresponding application checks. The final normal form and exact constraint syntax depend on the selected database engine and remain to be verified against a physical schema.

4.3.3 Persistent Classes and Database Mapping

Table 23 maps the main domain concepts to proposed persisted records. It distinguishes stable identity, planned work, completed care, and financial events so that a change in one lifecycle does not rewrite another.

Domain concepts	Proposed persisted records	Mapping rule
Patient and access	Patient, UserAccount, Role, Permission	Keep patient identity separate from login credentials; represent many-to-many role assignments with a join relation.
Appointment and visit	Appointment, Encounter, ResourceBooking	Preserve a planned appointment separately from a completed or walk-in encounter.
Clinical record	MedicalRecord, ClinicalNote, Diagnosis, VitalSign	Link encounter facts to patient and author; retain signed-note versions and their history.
Orders and results	Prescription, PrescriptionItem, Medication, ProcedureOrder, ExecutionResult	Store each order line or execution result separately and link it to its encounter and catalog item.
Finance	Invoice, InvoiceItem, Payment, Refund	Preserve issued line values and record payments/refunds as separate events.
Governance	AuditLog, Attachment, Notification	Store access metadata and controlled references; apply separate retention and access rules.

Table 23: Proposed domain-to-persistence mapping

4.3.4 Keys, Constraints, Data Types, and Multiplicities

Candidate primary and foreign keys are labeled in the logical ERDs. The final physical types have not been selected. Stable opaque identifiers are appropriate for primary keys; patient MRN requires a uniqueness constraint; exact money values should use a decimal representation; and event times should preserve an agreed time zone. Lifecycle status values should be constrained to the state transitions in Section 5. Key multiplicities and integrity rules are summarized in Table 24; the core domain relationships and business rules remain in Tables 7 and 9.

Relationship or key	Multiplicity / constraint	Integrity meaning
Patient–MedicalRecord	1 to 1	One longitudinal record per patient; patient identifier is unique.
Patient–Appointment	1 to 0..*	A patient can have multiple appointments; provider and resource conflicts are rejected.
Appointment–Encounter	0..1 to 0..1	An appointment may produce one encounter; walk-in encounters may have no appointment.
Encounter–ClinicalNote / Diagnosis / Order	1 to 0..*	Clinical facts and orders remain attached to their originating encounter.
Encounter–Invoice	1 to 0..1	An encounter may have at most one invoice.
Invoice–InvoiceItem / Payment	1 to 1..*; 1 to 0..*	An invoice contains charge lines and may be paid in several events.
Payment–Refund	1 to 0..*	Each refund references the original payment; prior payment evidence is retained.
UserAccount–Role	many to many	Access combines assigned role, action, and applicable record or facility scope.

Table 24: Proposed key multiplicities and integrity constraints

4.4 Structural UML Diagrams

4.4.1 Overall Class Diagram

Figure 3 remains the overall conceptual class view: it defines the central Patient, Appointment, Encounter, clinical-record, order, and financial concepts. It is a domain model, not a complete code-level class inventory. The focused diagrams below expand the two class groups that carry important lifecycle and integrity rules.

4.4.2 Design Class Views

The local SHMS `/design` catalog provides a useful Boundary–Control–Entity pattern: role-specific screens delegate to workflow controllers, which coordinate domain records. Figures 21 and 22 adapt selected outpatient class responsibilities to MEDORA's terminology. The billing controller refers to an optional `PaymentProcessor` port; no provider or working integration is

implied. These are proposed design views, while the following domain diagrams focus on data concepts and their relationships.

4.4.3 Class Diagrams by Use Case Group

Figure 23 models patient identity, appointments, and encounters. The optional appointment-to-encounter association preserves the supported walk-in path and keeps planned and actual visits distinct.
Figure 24 focuses on billing. Invoice lines belong to the invoice, but payment and refund records retain their own identities so partial settlement or a refund does not erase the original transaction.

4.4.4 Package Diagram

Figure 25 groups the system into identity and access, scheduling and reception, clinical care, orders and execution, finance, and governance. These are proposed logical boundaries; governance applies authorization and audit controls across the workflow. The package model adapts the SHMS repository's subsystem organization while excluding its inpatient-only packages. Runtime dependencies are shown separately in Figure 26.

4.4.5 Component Diagram

Figure 26 shows the proposed runtime-independent component boundary: the user interface calls an authenticated application boundary, workflow controllers coordinate domain rules, and repository interfaces access persistent records. It makes no claim about a particular framework, database, payment processor, or messaging provider.

4.4.6 Deployment Diagram

The deployment diagram is Figure 16. It is repeated by reference here rather than redrawn: logical nodes show the proposed client, application, and persistence boundary, while physical products and topology remain unselected.

4.4.7 Object Diagram

A separate object diagram is omitted because no concrete runtime snapshot is needed to explain the design, and no implemented dataset is available to ground one. The class and ERD views show the relevant structural relationships without inventing example patient records.

4.5 Use Case-oriented Design

4.5.1 Participating Classes and Their Responsibilities

Table 25 maps representative use cases to proposed boundary, control, and entity responsibilities. The names describe design roles; they are not claims about existing source-code classes.

Use case group	Boundary	Control responsibility	Principal entities
Patient registration and matching	Registration workspace	Validate required identity data, detect possible duplicates, and coordinate a controlled merge request.	Patient, UserAccount, MedicalRecord, AuditLog
Appointment and check-in	Reception and scheduling workspace	Check availability, save appointment changes, and create an encounter at check-in.	Appointment, Patient, Encounter, ResourceBooking
Encounter and documentation	Clinician workspace	Validate encounter state, save clinical facts, and append signed-note amendments.	Encounter, ClinicalNote, Diagnosis, Prescription, ProcedureOrder
Invoice and settlement	Billing workspace	Prepare eligible charges, issue an invoice, and record separate payment or refund events.	Invoice, InvoiceItem, Payment, Refund, AuditLog

Table 25: Proposed class responsibilities by use-case group

4.5.2 Collaboration Between System Components

The behavioral diagrams in Section 5 provide the interaction detail for these structural roles. Figures 27 and 28 show booking and reception interactions; Figure 29 shows signed-note handling; Figure 31 shows service scheduling and execution; and Figure 32 shows payment and separately authorized refund events. Together, the views trace a request from a role-specific boundary through validation and workflow coordination to the affected records. Protected reads and writes must enforce role and scope on the server and retain the audit evidence required by the business rules.

5 Behavioral Modeling and System Implementation

5.1 Behavioral UML Diagrams

The behavioral models in this section refine the business rules and use cases in Sections 2 and 3. They describe the observable behavior the system is designed to show; no implemented application has been observed. The six system-sequence diagrams treat MEDORA as one black box, so they imply no controller, repository, API or persistence technology, and every message carries a sequence number that the test cases can cite. The activity and state models focus on exception handling, lifecycle integrity, and separation of clinical and financial records.

5.1.1 Sequence Diagrams for Key Use Cases

The sequence views cover appointment booking (UC-021), patient check-in (UC-027), signing and amending a clinical note (UC-037/UC-038), prescription issue (UC-048), service scheduling and execution (UC-039/UC-040), and payment/refund (UC-063/UC-065). Rejected operations leave no partial business effect; successful transitions record the relevant accountable event. The request key in the payment view is an idempotency control derived from NFR-15; it is not an existing API contract.

5.1.2 Activity Diagrams for Complex Workflows

Figure 33 shows the service lifecycle from clinician order through resource validation and assigned execution. A conflict leaves the order pending without reserving resources; incomplete results do not mark execution complete. Figure 34 makes the boundary between signed clinical completion and invoice settlement explicit: an outstanding balance does not alter the clinical record. These diagrams complement the outpatient and appointment exception workflows in Figures 1 and 2.

5.1.3 State Machine Diagrams for Lifecycle-driven Entities

The state machines turn the lifecycle distinctions of Section 2 into explicit constraints for implementation and policy review. In particular, rescheduling preserves the prior appointment history, a clinical-note correction creates a linked version, and refunds remain separate financial transactions.

WorkStation names a status field for the Appointment only, so we fixed the state sets below as design decisions. Table 26 lists the values that the five figures use. A completed refund returns the invoice to Partially Paid or Issued according to the net paid amount; a refund still pending approval changes no invoice state.

Entity	States
Appointment	Draft, Booked, Confirmed, CheckedIn, NoShow, Canceled
Encounter	Waiting, In Consultation, Awaiting Service, Awaiting Closure, Completed
Clinical-note version	Draft, Signed, Amendment Draft, Amended Signed Version
Procedure order	Created, Pending Allocation, Scheduled, Delayed, In Progress, Completed, Not Performed, Canceled
Invoice	Draft, Issued, Partially Paid, Paid

Table 26: Lifecycle states by entity

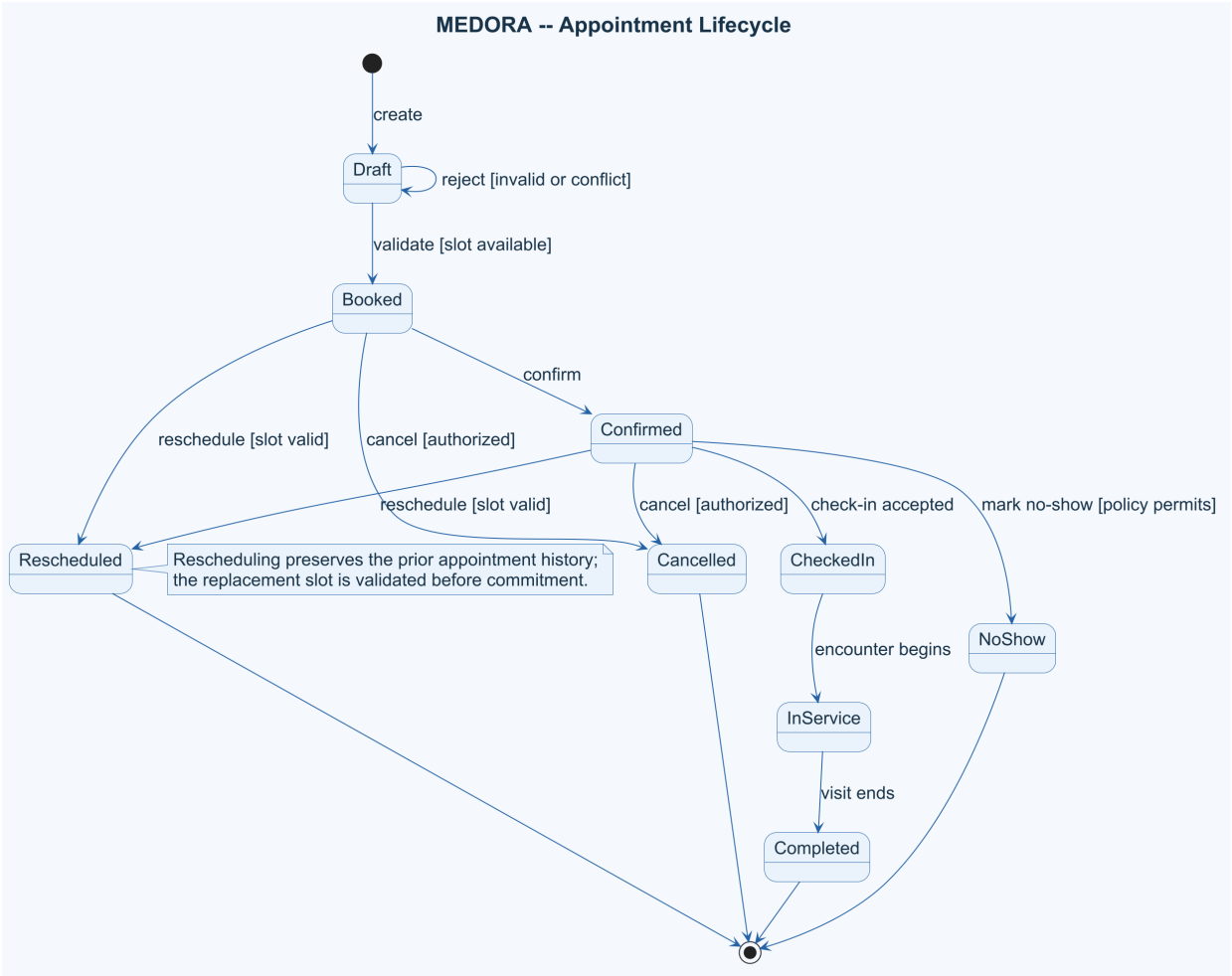


Figure 35: Appointment lifecycle and terminal outcomes

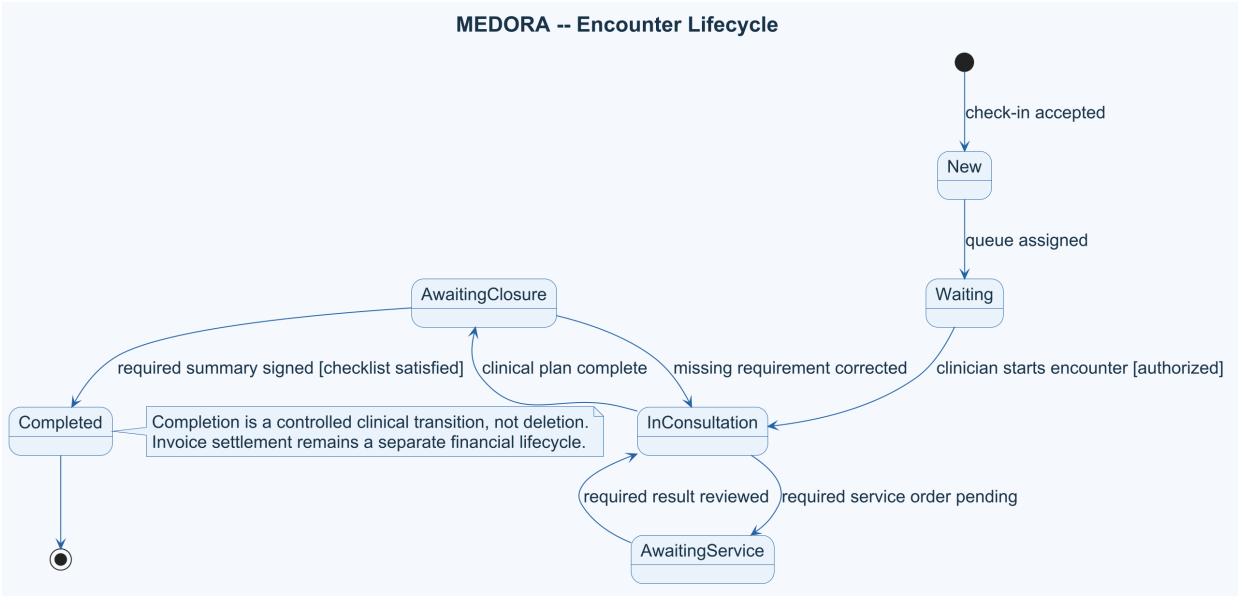


Figure 36: Encounter lifecycle from check-in to clinical completion

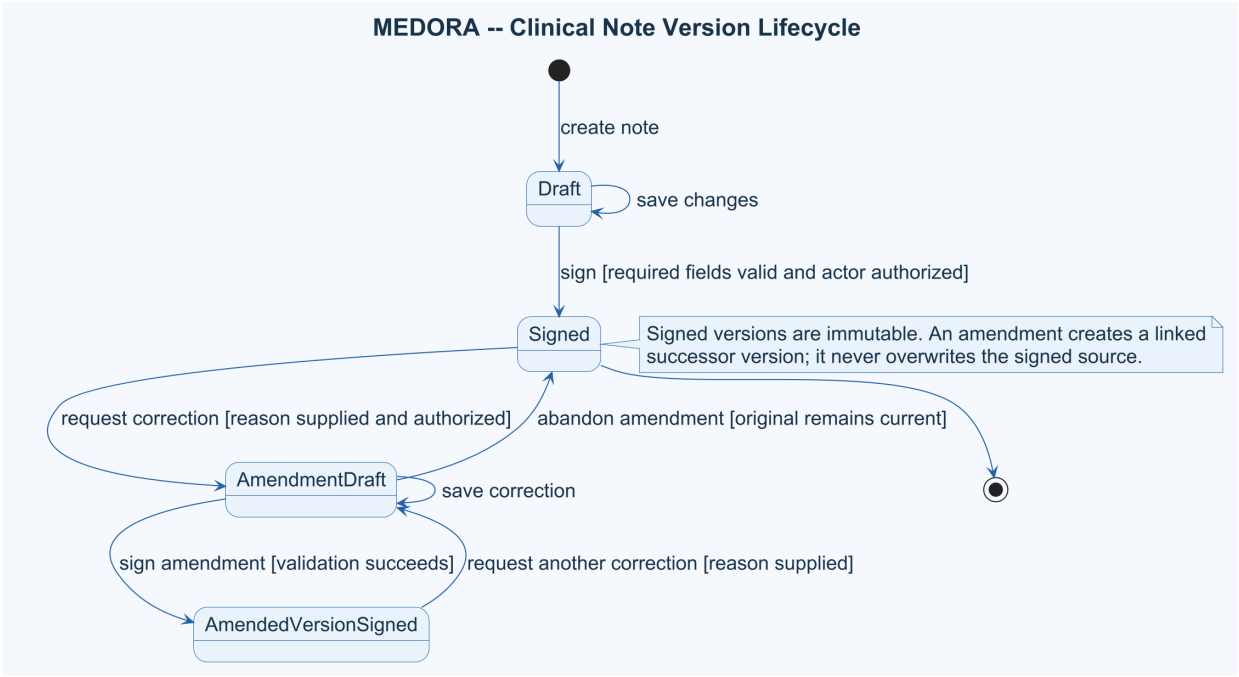


Figure 37: Clinical-note version lifecycle with immutable signed versions

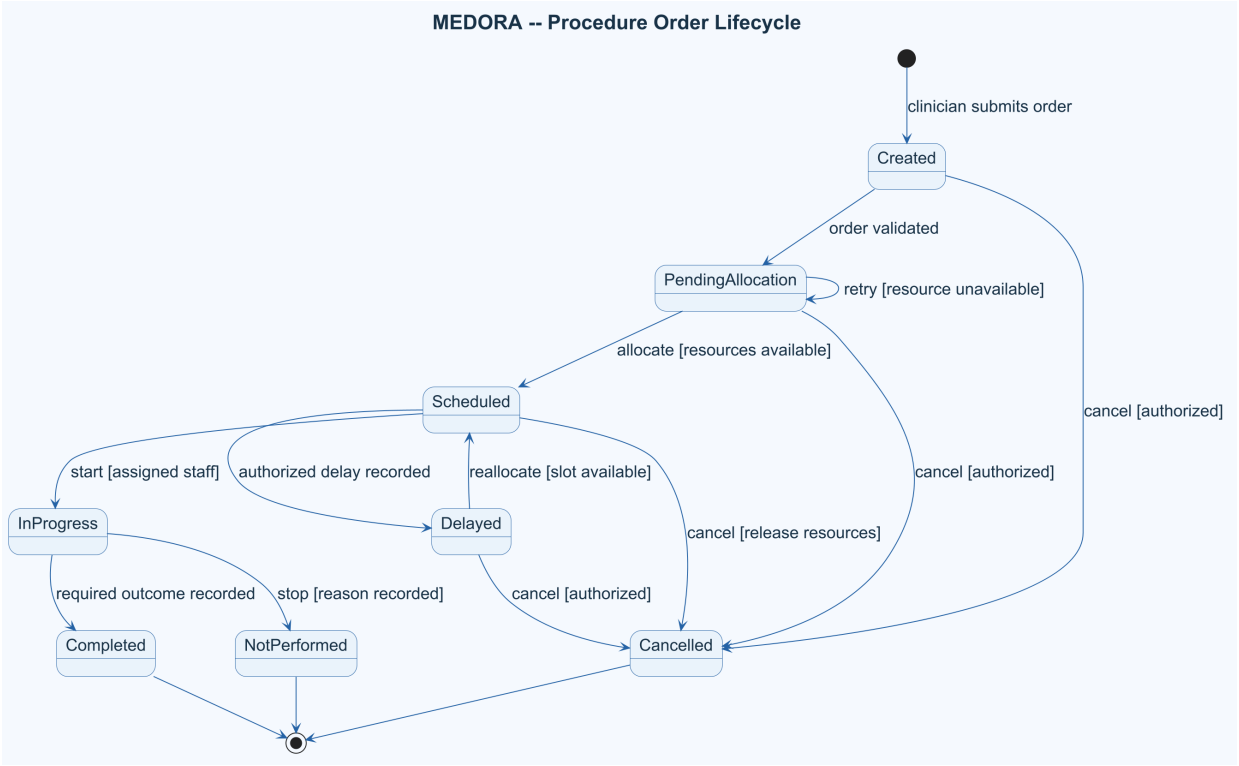


Figure 38: Procedure-order lifecycle with allocation and execution outcomes

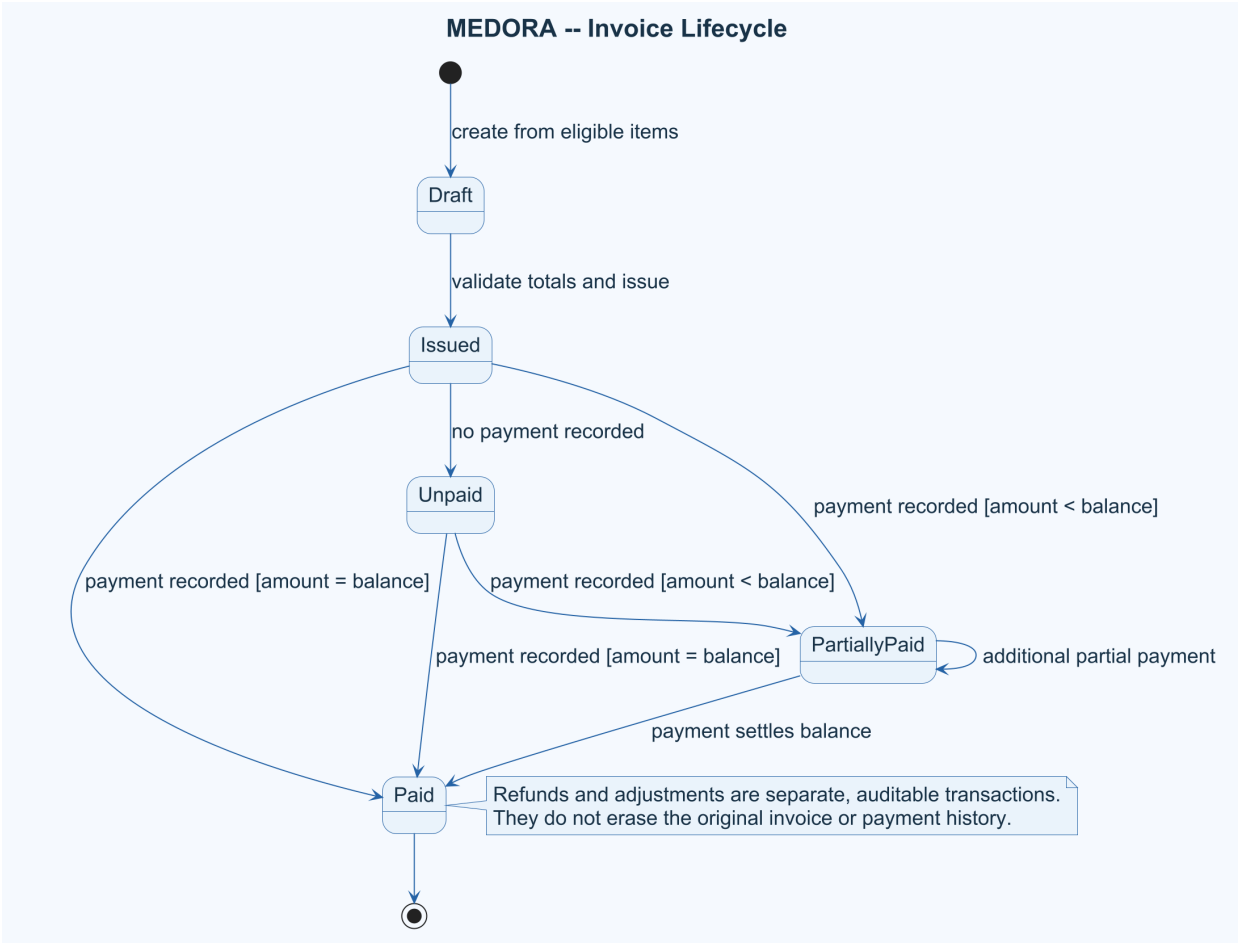


Figure 39: Invoice lifecycle with payment and completed-refund transitions

5.2 Source Code Organization and Module Structure

The repository contains no application code yet. To make the design actionable, this section proposes a TypeScript modular-monolith baseline: Next.js for the role-specific web client, a NestJS REST API, PostgreSQL persistence through Prisma migrations, private object storage for attachments and Docker Compose for local development. The stack is a recommendation, not an approved or implemented decision; the team should record approval as ADR-001 before scaffolding. The API stays the only authorization and state-transition boundary, and the browser never reaches persistence directly.

Listing 1: Proposed source layout for the first implementation

```
medora/
    apps/
        web/                          # Next.js role-specific browser
           UI
        api/src/modules/
            identity/ patients/ scheduling/
            encounters/ prescriptions/ orders/
            finance/ governance/ audit/
    packages/
```

```
        contracts/                    # versioned DTOs and OpenAPI
            contract
        persistence/                  # Prisma schema, repositories,
            migrations
    infra/
        compose.yaml                  # local-only services and test
            dependencies
        proxy/                        # non-production HTTPS
            configuration
    tests/
        unit/ integration/ e2e/       # synthetic data only
    doc/asset/                      # report and editable UML sources
```

Each API module owns its use cases and validation. Shared contracts contain request/response types, not business rules. Persistence migrations are reviewed and versioned; domain modules call repositories through application services. Authorization, transaction boundaries, audit emission, and idempotency belong in the API/application layer, not in UI visibility rules. A later implementation may reorganize this layout only with an updated module mapping and traceability review.

Evidence item	Repository observation	Status / implication
Application source	`src/README.md` is a placeholder asking for source code.	No application modules or implemented behavior are evidenced.
Build/dependency manifest	No application package manifest or runtime build configuration is present.	Next.js, NestJS and TypeScript are the recommended ADR-001 baseline, pending team approval.
Persistence and API	No database schema, migration, API contract, or service implementation is present.	PostgreSQL, Prisma and the versioned REST contract are proposals, not implementation facts.
Behavioral assets	PlantUML sources and rendered figures are stored under `doc/asset/05-behavioral-modeling-implementation/`.	These are behavioral models, not executable application code.
Deployment	No container, server, environment, or deployment configuration is present.	Production topology, hosting and operational controls are not yet determined.

Table 27: Implementation and deployment evidence available in the repository

The layout maps the logical layers and functional groups to a buildable project structure. No listed directory, package or module exists in the repository yet.

5.3 Implementation of Core Functionalities

The slices below turn the requirements into an ordered build plan. The invariant column states the minimum behavior each slice must keep before it is accepted. All six slices are currently Not Implemented.

Implementation slice	Trace	Acceptance invariant	Status
Identity and authorization	FR-001–FR-004; NFR-06	Server rejects out-of-scope reads/writes; patient duplicate review precedes merge.	Not Implemented
Scheduling and reception	FR-005–FR-006, FR-017	Concurrent conflicts do not commit; accepted check-in creates one active encounter.	Not Implemented
Encounter and clinical records	FR-007–FR-010, FR-020, FR-032	Signed versions are immutable; encounter close validates required work.	Not Implemented
Prescription and service orders	FR-011–FR-019	Configured safety checks are bounded; orders, bookings, and actual events remain distinct.	Not Implemented
Finance and reporting	FR-022–FR-027	Price is snapshotted; payment/refund are idempotent append-only events.	Not Implemented
Governance and attachments	FR-028–FR-031; NFR-07	Sensitive actions are scoped and audited; files are private and access-controlled.	Not Implemented

Table 28: Proposed implementation slices and acceptance invariants

5.4 Development and Deployment Environment

Table 29 defines a reproducible target environment. No such environment exists yet. Secrets, patient data and production records must never enter the local development stack. Production hosting, availability design, data residency, backup retention and incident ownership still need explicit approval.

Environment	Proposed services and configuration	Data / release controls
Local development	Node.js LTS toolchain; web and API; PostgreSQL; private S3-compatible storage emulator; Docker Compose.	Synthetic fixtures only; local secrets in ignored environment files; migrations applied from source.
Automated test	Ephemeral database and private file store; isolated test accounts; unit, integration, and Playwright end-to-end suites.	No real patient data; reset between runs; deterministic seeds and retained test logs.
Staging	One controlled application deployment behind HTTPS reverse proxy; restricted database and private attachment store; monitoring and backup/restore rehearsal.	De-identified/synthetic data only; role access reviewed; release build and migration version recorded.
Production target	Hosting provider and topology to be decided after capacity, privacy, recovery and cost review; separate app, database, object storage, and protected backup boundaries.	TLS, managed secrets, least-privilege service accounts, audit retention, restore exercises, and approved operational ownership.

Table 29: Proposed development and deployment environment matrix

The matrix fixes the intended trust boundaries without choosing a vendor topology. The LaTeX, PlantUML and Kroki tools that produce this report are documentation tooling and are not part of the MEDORA runtime.

5.5 Proposed User Interfaces and API Evidence

Figures 40–42 sketch the three screens with the highest workflow and financial risk. They follow the screen inventory in WorkStation Section 7 and use the canonical states and role boundaries of Sections 2 and 3: the Cashier screen offers no refund control, and Record payment stays unavailable until the invoice is issued. No running application exists, so no screen or API capture is available; Appendix C gives API and data examples that are not captured responses from an existing endpoint. A later evidence package must pair each redacted screen or API capture with the build ID, actor role, workflow state and test-case ID.

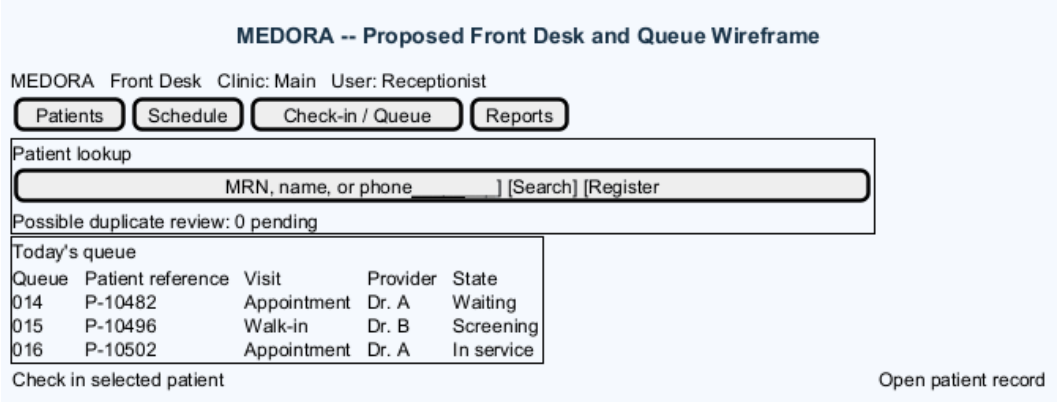


Figure 40: Front desk and queue wireframe

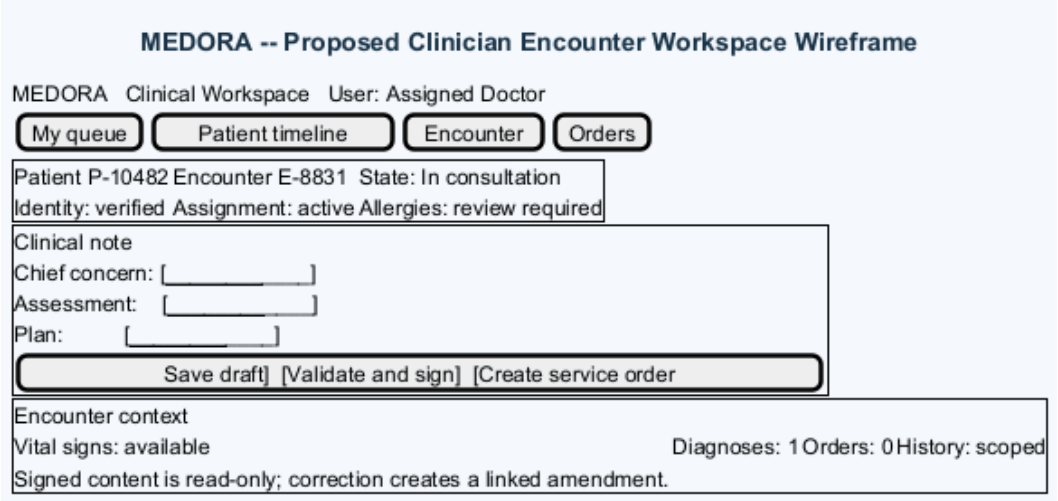


Figure 41: Clinician encounter workspace wireframe

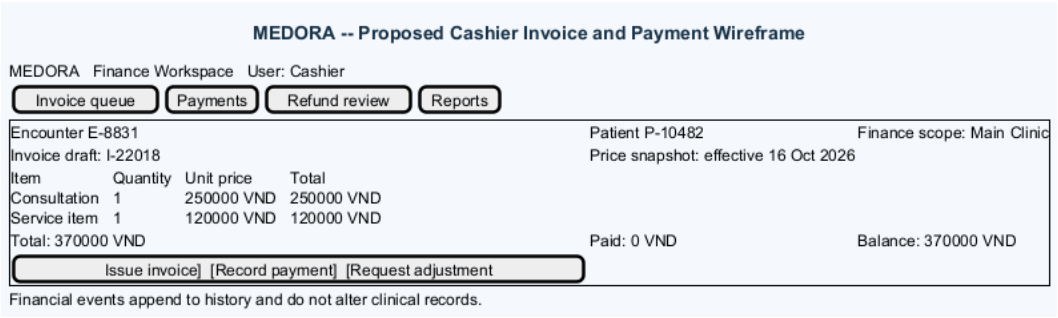


Figure 42: Cashier invoice and payment wireframe

Evidence required before implementation claims

After implementation, capture redacted UI, API, and workflow evidence from a running non-production build. Record build/version, actor role, workflow state, and linked test case. API evidence should include the approved endpoint/schema, authorization context, sanitized request/response, and observed result. Never include real patient data or secrets.

6 System Testing and Evaluation

6.1 Testing Strategy and Test Environment

Testing follows the business risks and acceptance criteria in WorkStation and Sections 2–3. It combines workflow, authorization, integrity, concurrency and quality-attribute checks. A rendered diagram or a successful report build is not evidence that the application works. The repository contains no application under test, database, API, automated test suite, test dataset or deployment environment, so execution is blocked. Once a build exists, testing should run in an isolated non-production environment with synthetic patient data, controlled roles, reproducible configuration and a recorded build identifier. Performance and recovery checks also need an agreed workload and an environment that represents the intended deployment.

Test level	Planned purpose	Current evidence
Unit / domain	Validate lifecycle guards, calculations, authorization decisions, and invariants.	Not available; no implementation or unit tests.
Integration	Verify transactions across encounter, order, resource, invoice, payment, and audit boundaries.	Not available; no database or service integrations.
End-to-end workflow	Exercise the reception-to-encounter, clinical, service, and financial paths by role.	Not available; no running application or test environment.
Security / negative access	Send unauthorized requests directly to protected server operations and verify safe rejection.	Not available; no API or identity implementation.
Quality attributes	Measure response time, accessibility, locale behavior, availability, and restore objectives.	Not available; targets remain proposed and unmeasured.

Table 30: Planned verification levels and present evidence boundary

6.2 Test Scenarios and Test Case Design

The following test cases translate high-risk requirements and use cases into executable checks. Expected outcomes are specifications for future tests. The status column records that none has been executed.

Table 31: Planned test cases and current execution status

ID	Scenario and expected result	Trace	Status
TC-001	Call a protected patient/clinical operation without the required role or scope; reject the request without returning protected data or applying a state change.	FR-001–FR-002; NFR-06; UC-001	Not Run
TC-002	Register a patient matching an existing identity; flag a possible duplicate and prevent an unauthorized merge.	FR-003–FR-004; BR-001; UC-010–UC-018	Not Run
TC-003	Submit concurrent bookings for the same constrained provider, room, or resource; accept no conflicting allocation and preserve rejected requests without a booking.	FR-005, FR-015, FR-017; NFR-15; UC-021–UC-026	Not Run
TC-004	Check in an accepted walk-in and attempt a second active encounter for the same patient; create one valid encounter/queue and reject the duplicate.	FR-006; BR-003; UC-027–UC-030	Not Run
TC-005	Sign a note with missing required content, then amend a signed note; block incomplete signing and retain the original signed version with a linked, reasoned amendment.	FR-009, FR-020, FR-032; NFR-07; UC-037–UC-042	Not Run
TC-006	Issue a prescription with a configured safety warning; require the permitted response/override evidence and verify issue alone creates no dispensing or administration event.	FR-011–FR-013; BR-006, BR-008; UC-043–UC-052	Not Run
TC-007	Allocate a service order with unavailable or conflicting resources, then complete a valid execution; do not commit a conflict and do not mark complete without required results.	FR-014–FR-018; NFR-15; UC-039–UC-040	Not Run
TC-008	Record partial and repeated payment submissions; maintain the correct balance and prevent a duplicate request from applying a second financial effect.	FR-022–FR-023; NFR-15; UC-060–UC-069	Not Run
TC-009	Submit an invalid refund request (authority, original transaction, amount or reason) and verify rejection with no financial effect; submit a valid request requiring approval and verify a pending status with no completed refund/balance; then verify that only an authorized, permitted request creates one separate refund event.	FR-023, FR-030; BR-009; UC-065–UC-066	Not Run
TC-010	Export a scoped report and review its audit evidence; enforce the actor's scope and record the controlled export event.	FR-026–FR-027, FR-030; NFR-06–NFR-07; UC-076–UC-079	Not Run
TC-011	Measure ordinary-screen and search response times under an agreed representative workload; compare p95 values with the provisional targets.	NFR-01; WorkStation Section 10	Not Run
TC-012	Restore a protected backup and measure data loss and recovery time against approved RPO/RTO; a backup log alone is insufficient evidence.	NFR-02–NFR-03; WorkStation Section 10	Not Run
TC-013	Verify keyboard operation, labels, contrast, Vietnamese text, configured date/time, and currency formats on supported browsers.	NFR-10–NFR-11, NFR-14	Not Run

6.3 Functional Testing and Use Case Coverage

The planned cases cover representative paths across identity/access, scheduling/reception, clinical documentation, prescribing, service execution, finance, and governance. They are a first execution set and do not cover FR-001–FR-034 exhaustively. Before implementation acceptance, each requirement and detailed use case must be mapped to one or more test cases, including negative authorization and exception cases.

Requirement group	Representative tests	Coverage status
FR-001–FR-004, FR-028–FR-031	TC-001–TC-002, TC-010	Specified; not yet implemented or run.
FR-005–FR-006, FR-017	TC-003–TC-004	Specified; not yet implemented or run.
FR-007–FR-010, FR-020–FR-021, FR-032	TC-004–TC-005	Specified; not yet implemented or run.
FR-011–FR-019	TC-006–TC-007	Specified; not yet implemented or run.
FR-022–FR-027, FR-030	TC-008–TC-010	Specified; not yet implemented or run.
NFR-01–NFR-16	TC-001, TC-003, TC-005–TC-013	Verification methods proposed; no measurements or results.

Table 32: Representative functional and quality-requirement test coverage

6.4 Exception Handling, Authorization, and Data Integrity Testing

The highest-risk checks are rejection paths that must leave no invalid partial state: unauthorized access (TC-001), duplicate identity/encounter handling (TC-002 and TC-004), concurrent resource allocation (TC-003 and TC-007), incomplete signing and append-only amendments (TC-005), bounded prescription overrides (TC-006), and idempotent payment/refund handling (TC-008–TC-009). TC-009 deliberately distinguishes rejection, pending approval and completed refund because these outcomes have different state, evidence and follow-up. The expected behavior is specified; the server-side controls and transactional boundaries cannot be verified until they exist in code.

6.5 Test Execution Results and Evidence

Execution status: Not Run

This revision has no runnable MEDORA application, test harness, database or test environment. All 13 cases in Table 31 are therefore **Not Run**. No case is reported as Pass or Fail, no coverage percentage is claimed, and no execution log exists.

Status	Count	Evidence
Pass	0	No executed results.
Fail	0	No executed results.
Blocked	0	The suite has not started; the prerequisite gap is recorded below.
Not Run	13	No system under test or execution environment.

Table 33: Test execution summary at report time

6.6 Evaluation Against Functional and Non-functional Requirements

Functional requirements are specified and traced to use cases. None is verified as implemented. The quality targets below come from WorkStation and are proposed acceptance criteria that still need owner confirmation and measurement.

Evaluation measure	Proposed acceptance basis	Current evaluation
Registration-to-completion time	Reduce median by at least 25% against a documented baseline for the representative scenario.	Not measured; no baseline or executable workflow.
Conflicting allocations	Zero accepted conflicts in all defined concurrency cases.	Not measured; no scheduling implementation.
Encounter completeness	At least 95% of configured mandatory fields for tested completed visits.	Not measured; no configured forms or visits.
Sensitive-event traceability	100% of tested sensitive events have actor, time, action, and target evidence.	Not measured; no audit implementation.
Authorization	100% of negative authorization cases rejected at the server/API boundary.	Not measured; no API or access-control implementation.
Response time	p95 ordinary screens $\leq$ 2 seconds and search $\leq$ 3 seconds under an agreed workload.	Proposed only; no deployment or load test.
Availability and recovery	Provisional 99.5% monthly availability, RPO $\leq$ 15 minutes, and RTO $\leq$ 4 hours.	Proposed only; no operational environment or restore exercise.

Table 34: Evaluation criteria and evidence status

6.7 Limitations and Outstanding Issues

The main limitation is that no system has been implemented, so the proposed test cases cannot establish behavior, performance, security, accessibility, recovery or user acceptance. The clinic must also confirm clinical templates and warning sources, retention and consent rules, merge, refund and override approvals, branch scope and the patient-portal release policy before these behaviors become firm acceptance requirements. No legal or certification compliance is claimed.

7 Conclusion and Future Work

7.1 Summary of Achievements

This report delivers a clinic-focused business and requirements baseline, a traceable use-case model and behavioral models for appointment booking, check-in, clinical-note integrity, prescription safety, service execution, encounter closure and payment history. It also provides a test plan that states each expected outcome and its execution status. These are analysis and design results; MEDORA itself has not been implemented, deployed or tested.

7.2 Limitations of the Current System

MEDORA is a specified system, not an operational product: the repository holds no application code, schema, API, user interface, deployment topology, test evidence or user feedback. The use-case, activity, sequence and state diagrams express intended behavior and will need reconciling with code once it exists. Performance, availability, recovery and success-rate values are targets. Clinical rule sources, data retention, consent, approval thresholds, multi-branch operation and portal publication remain open decisions.

7.3 Future Improvements and Extensions

Future work should proceed in evidence-producing stages:

1. Confirm clinic policies for authorization, clinical safety, record correction, finance, retention, and lifecycle transitions.
2. Decide and document the architecture, persistence model, deployment assumptions, and mapping from approved components to source modules.
3. Implement a testable MVP for identity, appointment/check-in, encounter documentation, role/scope enforcement, and audit evidence.
4. Extend the MVP with clinician-approved prescription and service workflows, then basic finance with separate, immutable transaction history.
5. Establish automated and negative-authorization tests; measure quality targets using synthetic data and a recorded environment; capture privacy-redacted UI/API evidence and usability/recovery results.
6. Add the patient portal or external integrations only after scope, contracts, security controls, and operational ownership are approved.

This sequence keeps the report aligned with the project's clinic scope and makes each future claim auditable against source code, test results, or operational evidence.

A Detailed Use Case Specifications

The following specifications expand the highest-risk use cases from Section 3. They are the behavior baseline for implementation and verification. Facility policies, clinical rules, approval thresholds and retention requirements must be approved before the matching behavior is final.

A.1 UC-021 Book Appointment

Goal Reserve a valid clinic service slot for an identified patient.

Primary actor
Receptionist. Patient self-service is Phase 2.

Trigger A patient requests an appointment or a receptionist schedules follow-up.

Preconditions
Actor is authenticated and facility-scoped; patient identity and requested service are resolved; scheduling data is available.

Main success flow
1. Receptionist selects patient, service, provider, and requested time.
2. System validates request fields, actor permissions, facility, and patient scope.
3. System checks provider schedule, duration, room, and any configured constrained resource.
4. System repeats the availability check at transaction commit and atomically creates the Appointment and required ResourceBooking records.
5. System records actor/time and returns appointment ID and status.

Alternative flows
If a slot is unavailable, the system offers permitted alternatives; the receptionist selects one and the system repeats validation. A draft request may be retained without reserving resources if the clinic enables drafts.

Exception flows
Invalid identity, unauthorized scope, unavailable resource, or a concurrent conflict rejects the request without partial appointment/resource records. Repeating a successful request with the same idempotency key returns the original result.

Postconditions and data
Success creates one Appointment and optional resource allocation; it does not create an Encounter. Failure leaves the prior schedule unchanged.

Authorization
Receptionist in facility scope; any patient portal actor is separately authenticated and limited to their own identity.

Trace BR-002; FR-005, FR-015, FR-017; UC-020–UC-026; TC-003; Figures 27 and 35.

A.2 UC-027 Check In Patient

Goal Start one valid outpatient Encounter and place it in the configured queue.

Primary actor
Receptionist.

Trigger An identified booked patient arrives or a walk-in is accepted by facility policy.

Preconditions
Receptionist is authenticated and scoped; patient identity is resolved; walk-in policy is known where applicable.

Main success flow
1. Receptionist selects an Appointment or accepted walk-in intake.
2. System validates patient identity, facility scope, and current Appointment state.
3. System checks whether the patient already has an active Encounter under the approved rule.
4. System creates/activates the Encounter and assigns the configured queue state atomically.
5. System returns Encounter and queue references.

Alternative flows
Accepted walk-ins use the same identity and duplicate-Encounter checks. A rejected walk-in remains unregistered as an active Encounter and receives the permitted next step.

Exception flows
Unresolved identity, unauthorized facility, invalid Appointment state, or a duplicate active Encounter rejects the transition without creating a second Encounter.

Postconditions and data
Success links Encounter to one Patient and optionally one Appointment. A booking without check-in remains an Appointment only.

Authorization
Receptionist may create/check in; Nurse may read only the queue assigned by role/facility policy.

Trace BR-003; FR-006; UC-027–UC-030; TC-004; Figures 28 and 36.

A.3 UC-037/UC-038 Sign and Amend Clinical Note

Goal Finalize a clinically attributable note while preserving any later correction history.

Primary actor

Doctor assigned to the active Encounter.

Trigger Doctor completes documentation or identifies an error in signed content.

Preconditions

Doctor is authenticated, authorized for the Encounter, and the encounter is in a state that permits documentation.

Main success flow

1. Doctor saves a draft with the required clinical sections.
2. System validates required data, assignment, and expected current version.
3. Doctor requests signing.
4. System records an immutable signed version with actor and timestamp and emits audit evidence.
5. System returns the signed-version reference.

Alternative flows

An incomplete note remains a Draft. A correction creates a successor amendment linked to the signed predecessor; the Doctor supplies a reason and signs the amendment separately.

Exception flows

Missing fields, stale version, invalid assignment, or unauthorized amendment rejects the operation. The signed source and all previous versions remain unchanged.

Postconditions and data

NoteVersion history is append-only; no direct update of signed content is allowed.

Authorization

Assigned Doctor may sign/amend; all reads/writes remain role-, facility-, and encounter-scoped.

Trace BR-005; FR-009, FR-020, FR-032; UC-037/UC-038; TC-005; Figures 29 and 37.

A.4 UC-048 Validate and Issue Prescription

Goal Issue an authorized, complete prescription after configured safety checks.

Primary actor

Doctor assigned to the active Encounter.

Trigger Doctor submits a prescription draft for issue.

Preconditions

Patient and Encounter are verified; Doctor is assigned and authorized; at least one medicine item is present.

Main success flow 1. Doctor submits medicine items and directions.

2. System validates required fields and configured rule inputs that are available.
3. System presents any supported warning and its reason.
4. Doctor resolves the warning or performs a permitted override with reason.
5. System issues the Prescription, records actor/time/audit evidence, and returns the issue reference.

Alternative flows

Doctor may keep the Prescription in Draft. If no blocking warning is raised, the Doctor may issue after required-field validation.

Exception flows

Missing data, a blocking warning, unavailable required policy data, unauthorized override, or stale Encounter state prevents issue and creates no administration event.

Postconditions and data

Issued Prescription and PrescriptionItems are distinct from MedicationAdministration; issuing alone never asserts medicine was dispensed/administered.

Authorization

Assigned Doctor issues; medicine staff record only the separately authorized administration/dispensing event.

Trace BR-006/BR-008; FR-011–FR-013; UC-043–UC-052; TC-006; Figure 30.

A.5 UC-039/UC-040 Schedule and Execute Service Order

Goal Coordinate a requested service with valid resources and record its actual outcome.

Primary actors

Doctor creates the order; assigned Service Staff records execution.

Trigger Doctor adds a service/procedure to an active Encounter.

Preconditions

The Encounter is active; service catalog and resource requirements are configured; actors have relevant scope.

Main success flow 1. Doctor records service, reason, priority, and required preparation.

2. System validates Encounter, order fields, and actor permissions.
3. Scheduler requests a time/resource allocation.
4. System validates room, staff, and constrained-resource availability at commit.
5. Assigned staff starts the order and records actor/time.

6. Staff records required outcome; system validates and marks execution complete.

Alternative flows

If no resource slot is available, the order remains pending without a booking and may be rescheduled after a fresh availability check. A delayed order remains non-complete.

Exception flows

Conflict, missing assignment, invalid state, or incomplete outcome rejects the transition and preserves the order's last valid state. Cancellation records a reason and releases resources according to policy.

Postconditions and data

ProcedureOrder, ResourceBooking, and execution outcome are linked but distinct records.

Authorization

Doctor creates order; assigned Service Staff updates execution; scheduling authority follows clinic policy.

Trace BR-002/BR-007; FR-014–FR-018; UC-039/UC-040; TC-007; Figures 31 and 33.

A.6 UC-063/UC-065 Record Payment and Refund

Goal Record settlement or refund while preserving a complete financial history.

Primary actors

Cashier posts payment; authorized Finance Staff posts refund/adjustment.

Trigger Eligible Invoice is issued or a prior PaymentTransaction requires refund/adjustment.

Preconditions

Actor is authenticated and facility/finance-scoped; invoice and original transaction state are known.

Main success flow

1. Cashier selects invoice and enters amount, currency, and method.
2. System validates balance, state, scope, and idempotency key.
3. System appends PaymentTransaction and returns the resulting invoice balance.
4. For a refund, Finance Staff references the original transaction, supplies a reason, and obtains configured approval.
5. System appends a refund/adjustment transaction and audit event; the original payment remains intact.

Alternative flows

Partial payment leaves an outstanding balance. Same-key retries return the original result.

Exception flows

Invalid amount/state, conflicting idempotency key, insufficient authority, or missing reason/approval has no financial effect.

Postconditions and data

Invoice and PaymentTransaction history is preserved; clinical records are unchanged.

Authorization

Cashier posts payments; a separately authorized finance role handles refunds and approvals.

Trace BR-009; FR-022/FR-023; UC-063/UC-065; TC-008/TC-009; Figure 32.

B Supplementary UML Diagrams

Table 35 catalogs the editable sources, rendered figures and traceability inventory. The diagrams are models, not implementation evidence. Section 4 is reserved for the architecture, component, class and ERD diagrams; workflow and lifecycle diagrams are in Section 5.

Table 35: UML source and rendered-figure inventory

Source group	Types / count	Purpose	Status
Section 2 business analysis	Activity 2; domain 1	Outpatient journey, appointment/check-in exceptions, and clinical domain concepts.	Rendered
Section 3 requirements	Functional 1; overall use case 1; focused use case 9	Capability groups, system boundary, actors, and detailed actor goals.	Rendered
Section 4 architecture	Reserved for Chapter 4 owner	This contribution restores the original Chapter 4 outline and claims no Chapter 4 UML.	Pending
Section 4 persistence	Reserved for Chapter 4 owner	ERD and schema artifacts remain with the Chapter 4 contributor.	Pending
Section 4 classes	Reserved for Chapter 4 owner	Structural class artifacts remain with the Chapter 4 contributor.	Pending

Source group	Types / count	Purpose	Status
Section 5 interaction	System sequence 6	Actor-visible success, alternative, and rejection outcomes for high-risk use cases.	Rendered
Section 5 process/lifecycle	Activity 2; state machine 5	Service execution, closure/finance separation, and entity transitions.	Rendered
Section 5 UI wireframes	Wireframe 3	Front desk and queue, clinician encounter, and cashier invoice and payment screens. Captures of a running application come after implementation.	Rendered

The PlantUML source is stored in the matching `uml_code/` folder and the rendered PNG in `figure/`. Rendering checks syntax and supports visual review; it does not establish implementation, clinical approval or user acceptance.

C API Specifications and Data Schemas

This appendix and Chapter 5 propose API request and response examples and a common error schema. The examples below show representative persisted data shapes for implementation planning; all IDs and values are synthetic.

C.1 Representative Appointment Data

Listing 2: Proposed Appointment resource shape

```
{
  "appointmentId": "appt-demo-1001",
  "patientId": "pat-demo-001",
  "serviceId": "svc-demo-012",
  "providerId": "emp-demo-004",
  "facilityId": "facility-demo-main",
  "startAt": "2026-10-20T09:30:00+07:00",
  "endAt": "2026-10-20T10:00:00+07:00",
  "status": "BOOKED",
  "createdAt": "2026-10-16T09:00:00+07:00"
}
```

C.2 Representative Encounter Data

Listing 3: Proposed Encounter resource shape

```
{
  "encounterId": "enc-demo-2001",
  "patientId": "pat-demo-001",
  "appointmentId": "appt-demo-1001",
  "status": "WAITING",
  "facilityId": "facility-demo-main",
  "assignedClinicianId": "emp-demo-004",
  "startedAt": "2026-10-20T09:25:00+07:00"
}
```


C.3 Representative Audit Event Data

Listing 4: Proposed audit event shape with no clinical payload duplication


```
{
  "auditEventId": "audit-demo-9001",
  "actorId": "user-demo-001",
  "action": "CLINICAL_NOTE_SIGNED",
  "targetType": "ClinicalNoteVersion",
  "targetId": "note-version-demo-002",
  "facilityId": "facility-demo-main",
  "occurredAt": "2026-10-20T10:02:00+07:00",
  "reason": null,
  "correlationId": "req-demo-001"
}
```


These shapes are illustrative contract baselines. The approved OpenAPI schema, database types, retention rules, field-level access policy and migration definitions still have to be written and reviewed before implementation.

D Detailed Test Cases and Execution Results

Detailed test procedures and expected outcomes are listed in Table 36. The actual-result, tester, timestamp, build and evidence fields are left empty. All 13 cases remain **Not Run** because no application or test environment exists, and no pass or fail result is claimed.

Table 36: Detailed test procedures and expected outcomes

ID	Setup	Procedure	Expected result / evidence to capture	Status
TC-001	Synthetic patient; authenticated user without clinical permission.	Call protected patient/clinical read and write endpoints directly, bypassing the UI.	Reject both requests; disclose no protected fields; create no mutation; retain sanitized response and audit-policy evidence.	Not Run
TC-002	One synthetic patient; registrar and ordinary receptionist accounts.	Register a matching identity, then request merge as receptionist.	Flag possible duplicate; deny unauthorized merge; preserve source records and history.	Not Run

ID	Setup	Procedure	Expected result / evidence to capture	Status
TC-003	One provider/room/resource slot; two concurrent clients.	Submit conflicting bookings with separate idempotency keys at the same time.	At most one booking commits; losing request receives conflict/alternatives; no orphan resource reservation remains.	Not Run
TC-004	Patient with no active encounter and an accepted appointment or walk-in.	Check in once; repeat check-in while the first encounter remains active.	First call creates one encounter/queue assignment; second is rejected without duplicate state.	Not Run
TC-005	Assigned doctor; incomplete draft and then corrected note.	Attempt signing while required content is missing; correct, sign, then append an amendment with reason.	Incomplete signing is blocked; signed source remains immutable; amendment links predecessor and records actor/time/reason.	Not Run
TC-006	Synthetic, clinician-approved safety-rule fixture; doctor and medicine staff roles.	Issue with a warning; test dismissal and permitted override; inspect medication events.	Apply configured warning behavior; require authorized override reason; prescription issue alone creates no administration event.	Not Run
TC-007	Valid order and two requests competing for the same resource.	Attempt conflicting allocations; submit incomplete execution result and then a valid result.	Conflict does not commit; incomplete result cannot complete order; valid result records performer/time and releases resources by policy.	Not Run
TC-008	Issued invoice with known total; cashier in authorized facility.	Post partial payment; retry same key; attempt another payment with a new key.	Same key returns original event; permitted new payment updates balance exactly once; reconciliation matches transaction history.	Not Run
TC-009	Paid synthetic invoice; cashier and authorized finance role.	Request refund with unauthorized role, invalid amount, missing reason, then valid approval.	Invalid requests have no financial effect; valid refund appends event and preserves original payment.	Not Run
TC-010	Two facility scopes with synthetic report data.	Query and export through auditor/manager roles across authorized and unauthorized scopes.	Enforce scope; audit actor, time, filter, and target; do not duplicate protected clinical content in logs.	Not Run
TC-011	Staging-like environment, agreed synthetic dataset, representative concurrent load.	Measure ordinary screens and search; record sample size, p95, build, and configuration.	Compare results with owner-approved targets; retain sanitized measurements; do not infer production capacity.	Not Run
TC-012	Encrypted synthetic backup and clean recovery environment.	Restore database and private files; verify checksums and measure data loss and recovery time.	Compare measured RPO/RTO with approved objectives; retain restore log and timestamps.	Not Run
TC-013	Supported browsers and synthetic Vietnamese names, dates, currencies; keyboard-only user.	Complete core workflows using keyboard; review labels, errors, contrast, date/time, and currency formats.	Record browser/version and findings; no blocking accessibility or localization defect remains at acceptance.	Not Run

For an executed run, add the actual result, tester, timestamp, build ID, environment, evidence link and, where relevant, defect ID. The test may then be marked Pass, Fail or Blocked. Until real execution evidence supports those fields, the status stays Not Run.

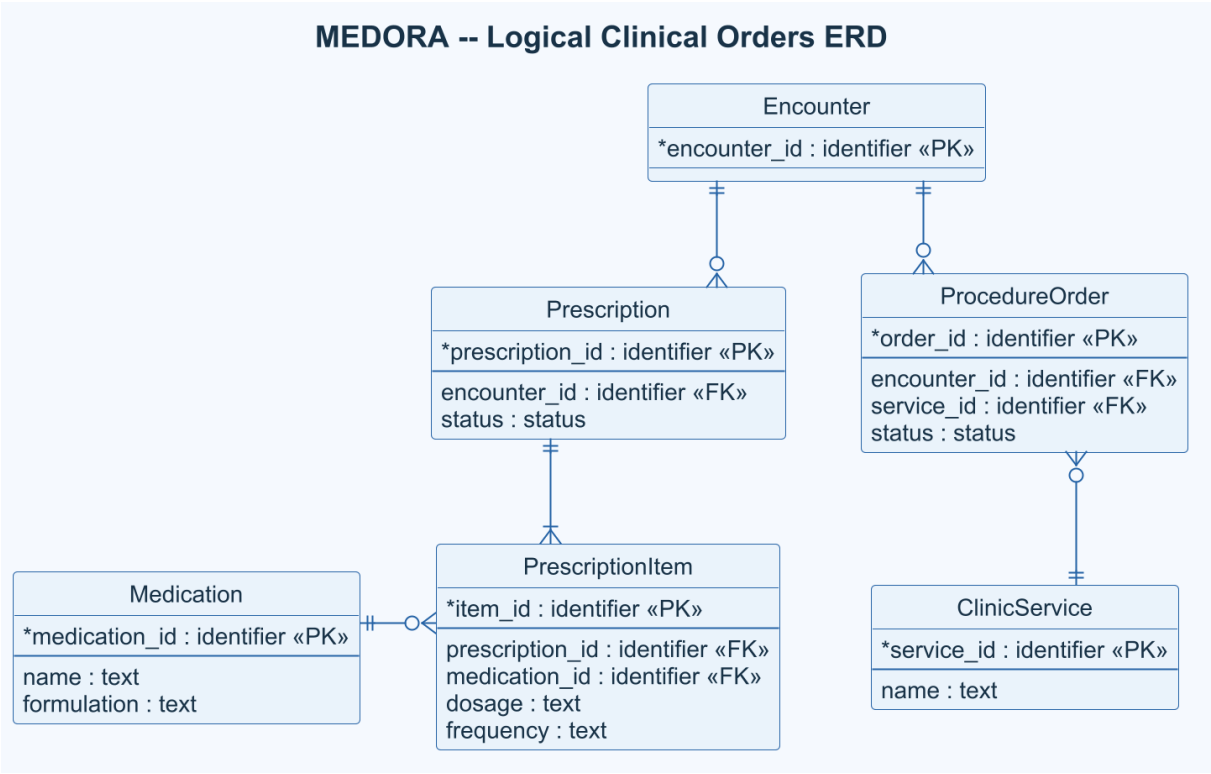


Figure 19: Logical ERD for prescriptions, medication items, and service orders

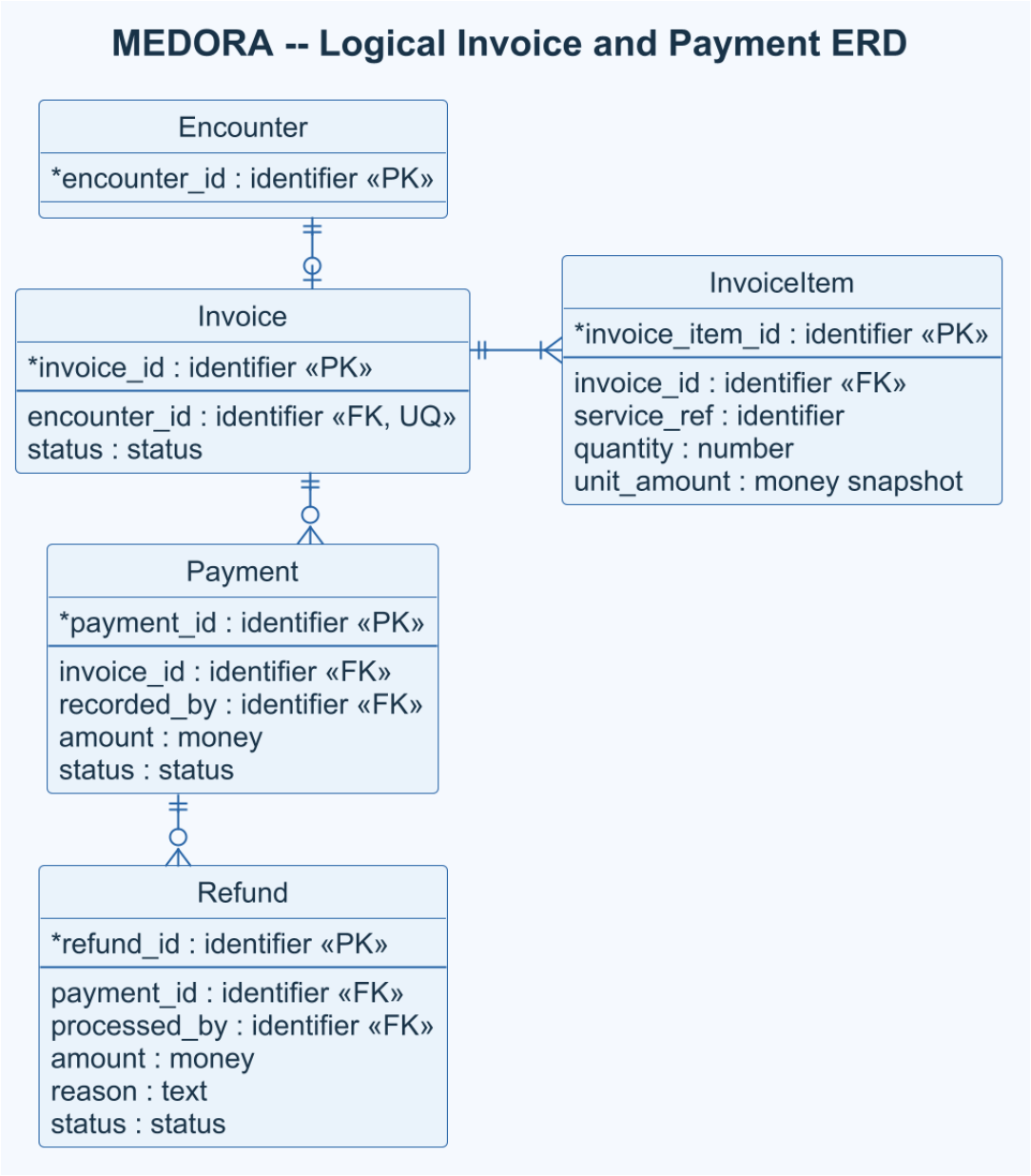


Figure 20: Logical ERD for invoices, payment events, and linked refunds

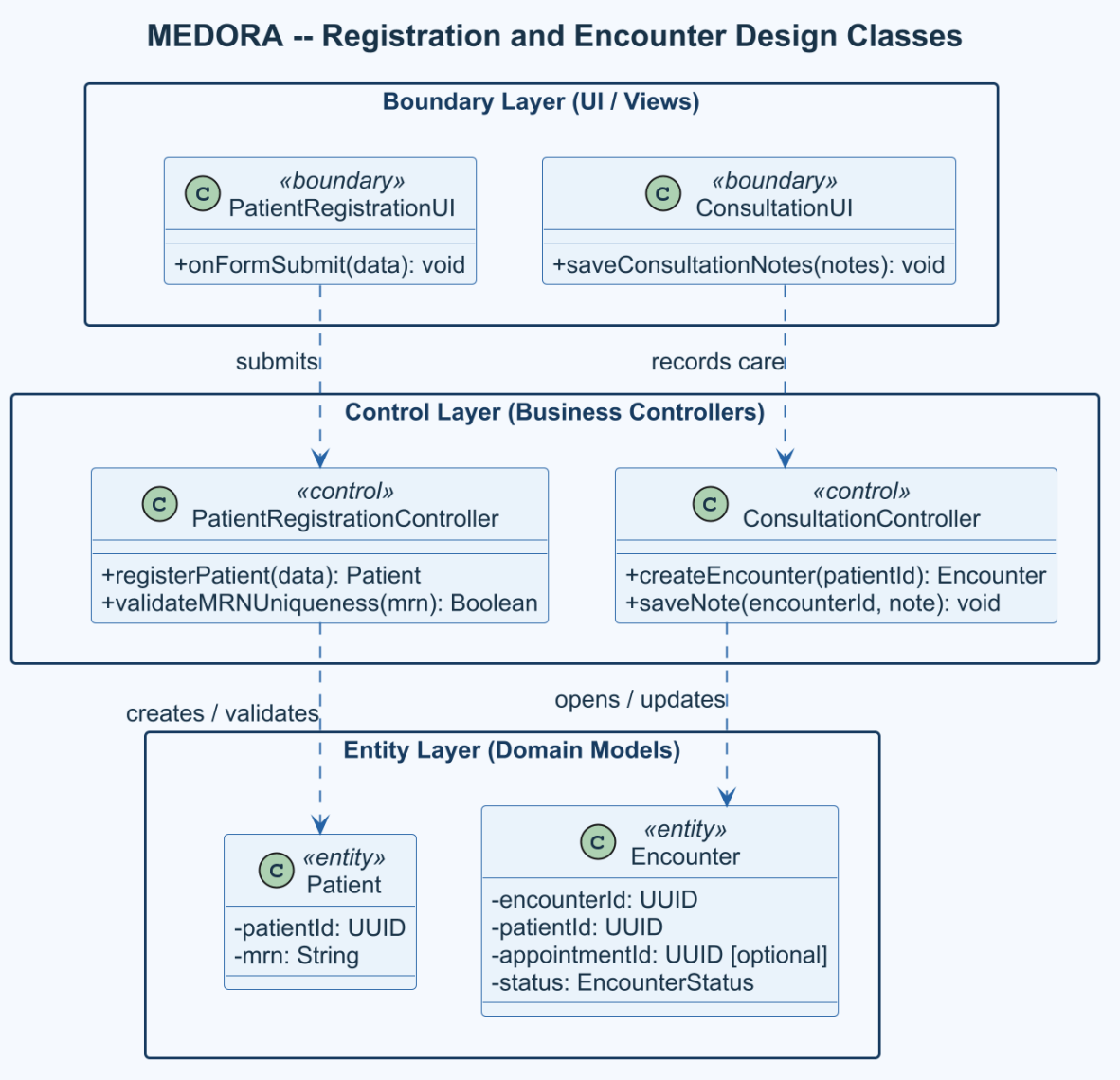


Figure 21: Proposed boundary, controller, and entity responsibilities for patient registration and outpatient encounters

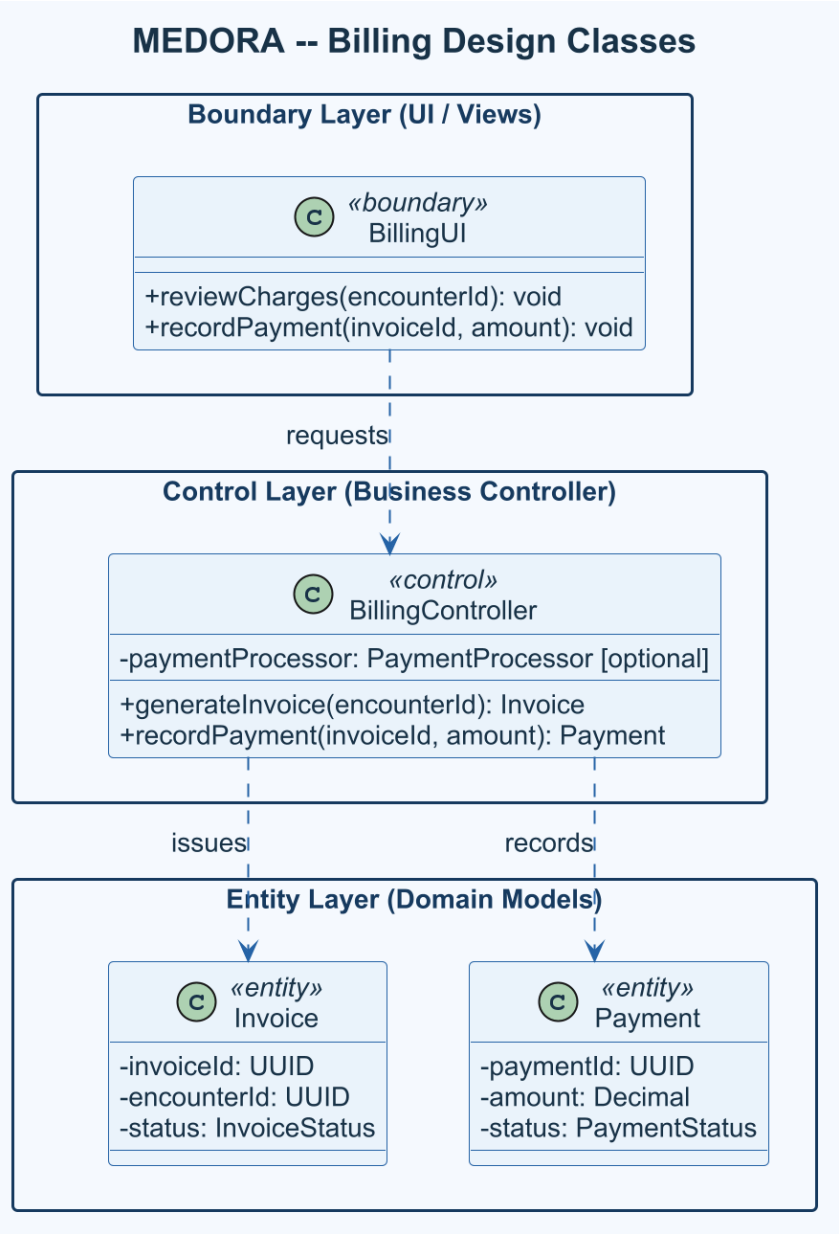


Figure 22: Proposed billing responsibilities with an optional payment processor dependency

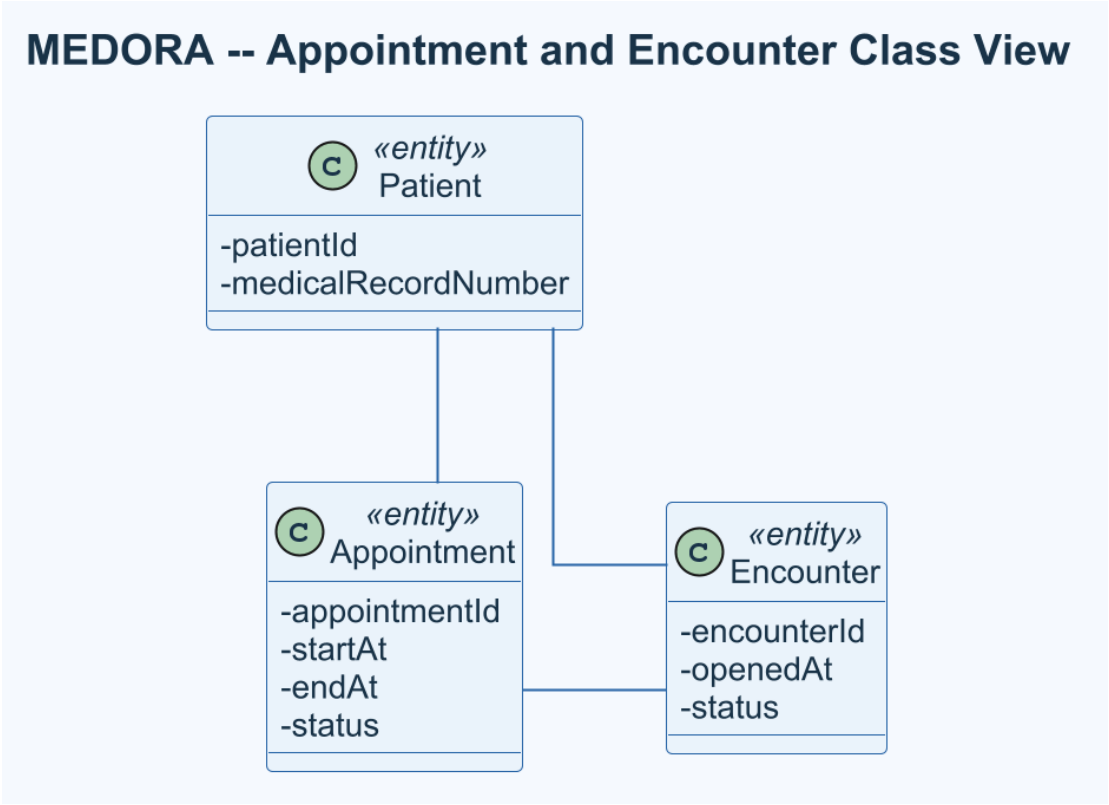


Figure 23: Domain class view for appointment booking and encounter creation

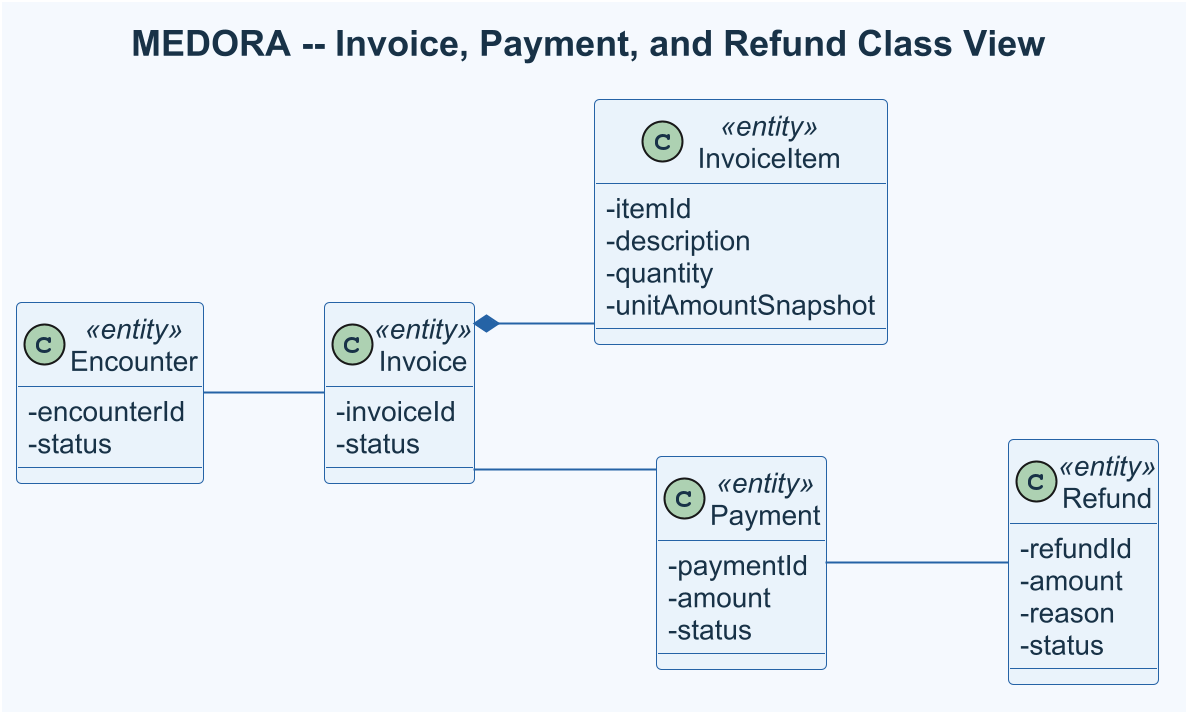


Figure 24: Domain class view for invoice settlement and refund history

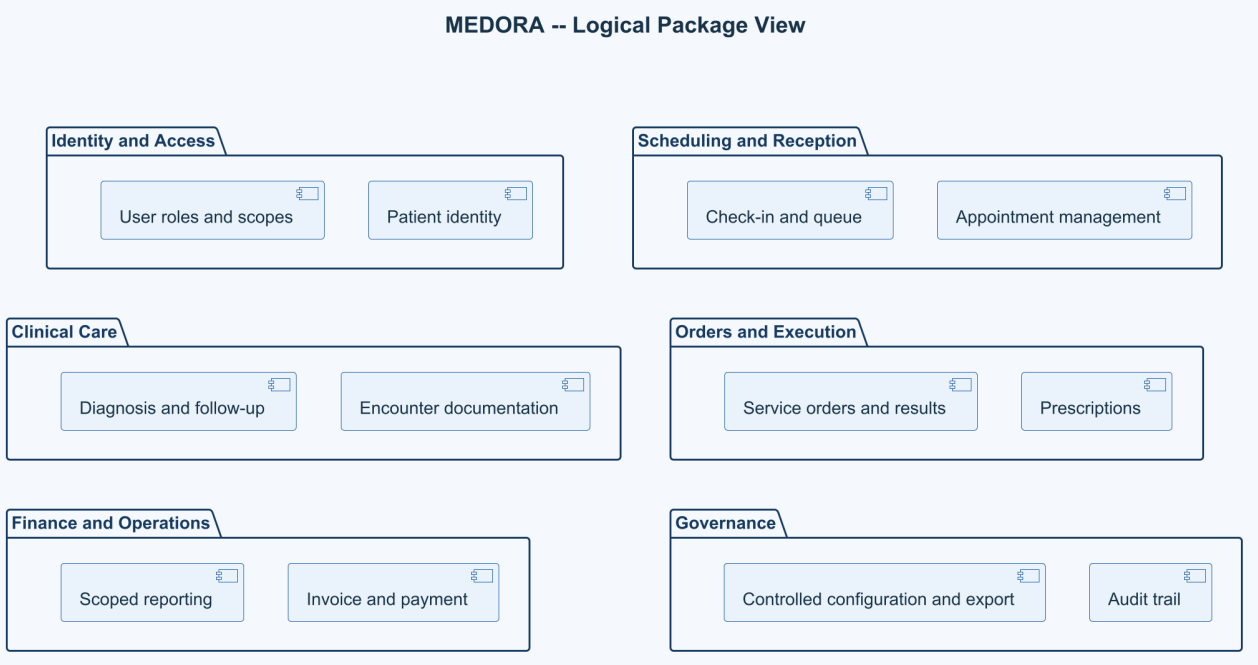


Figure 25: Proposed MEDORA logical package decomposition

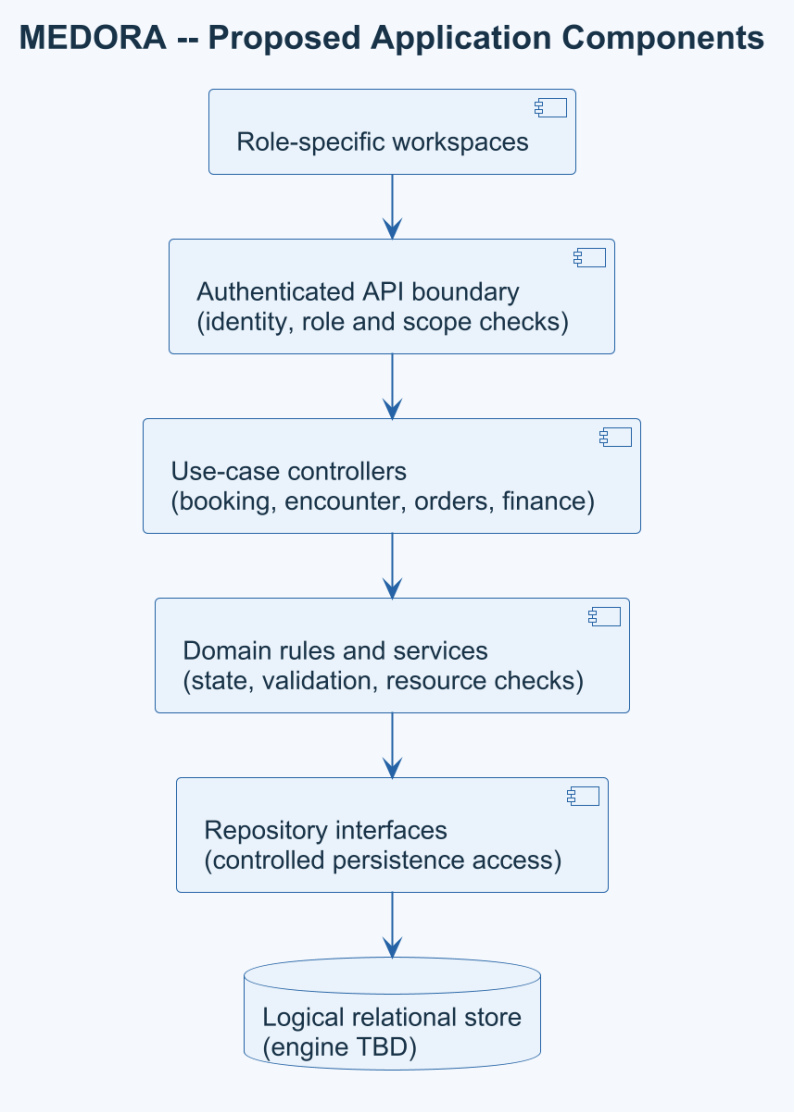


Figure 26: Proposed application component dependencies

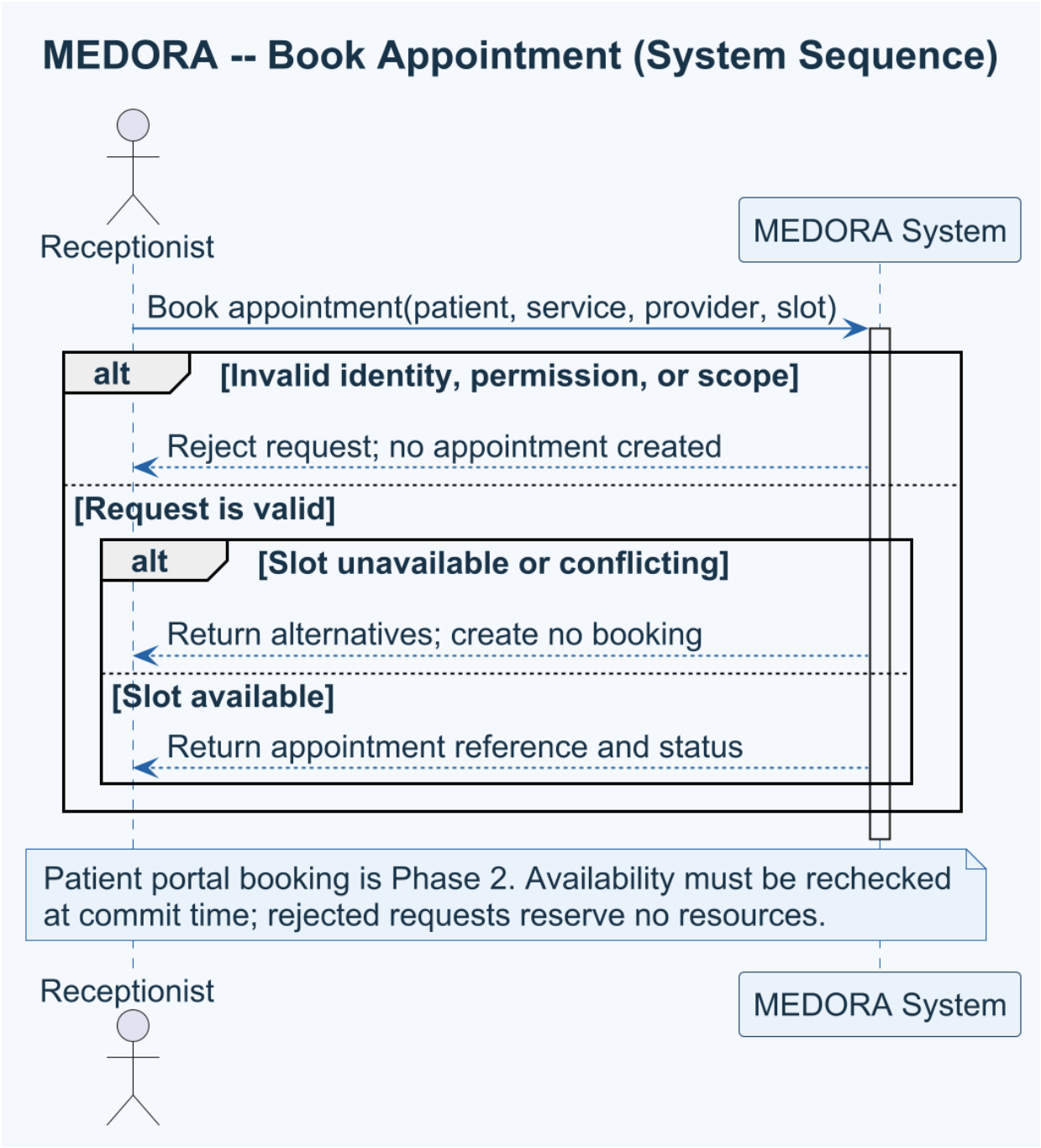


Figure 27: System sequence for appointment booking and availability validation

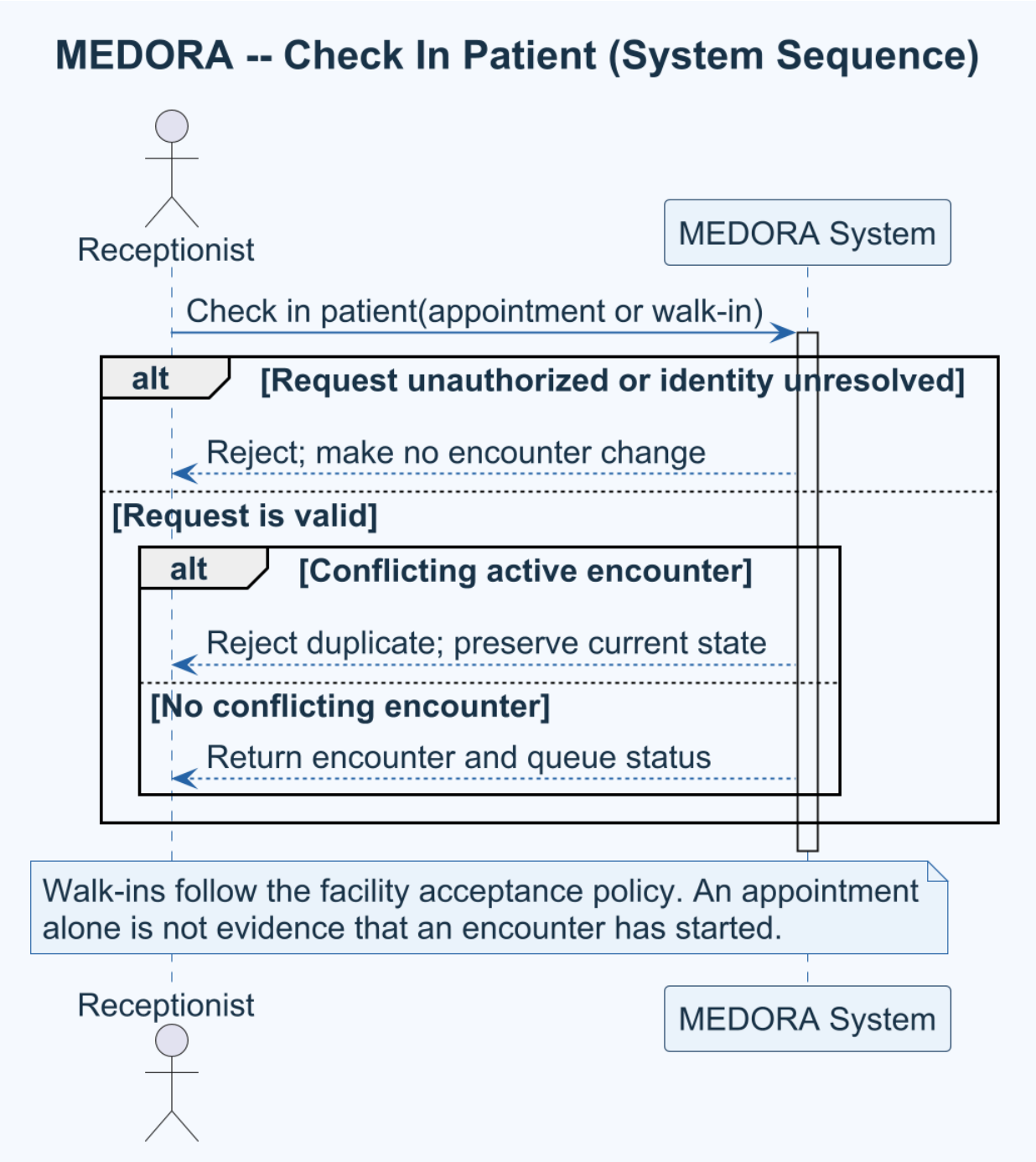


Figure 28: System sequence for check-in, encounter creation, and queue assignment

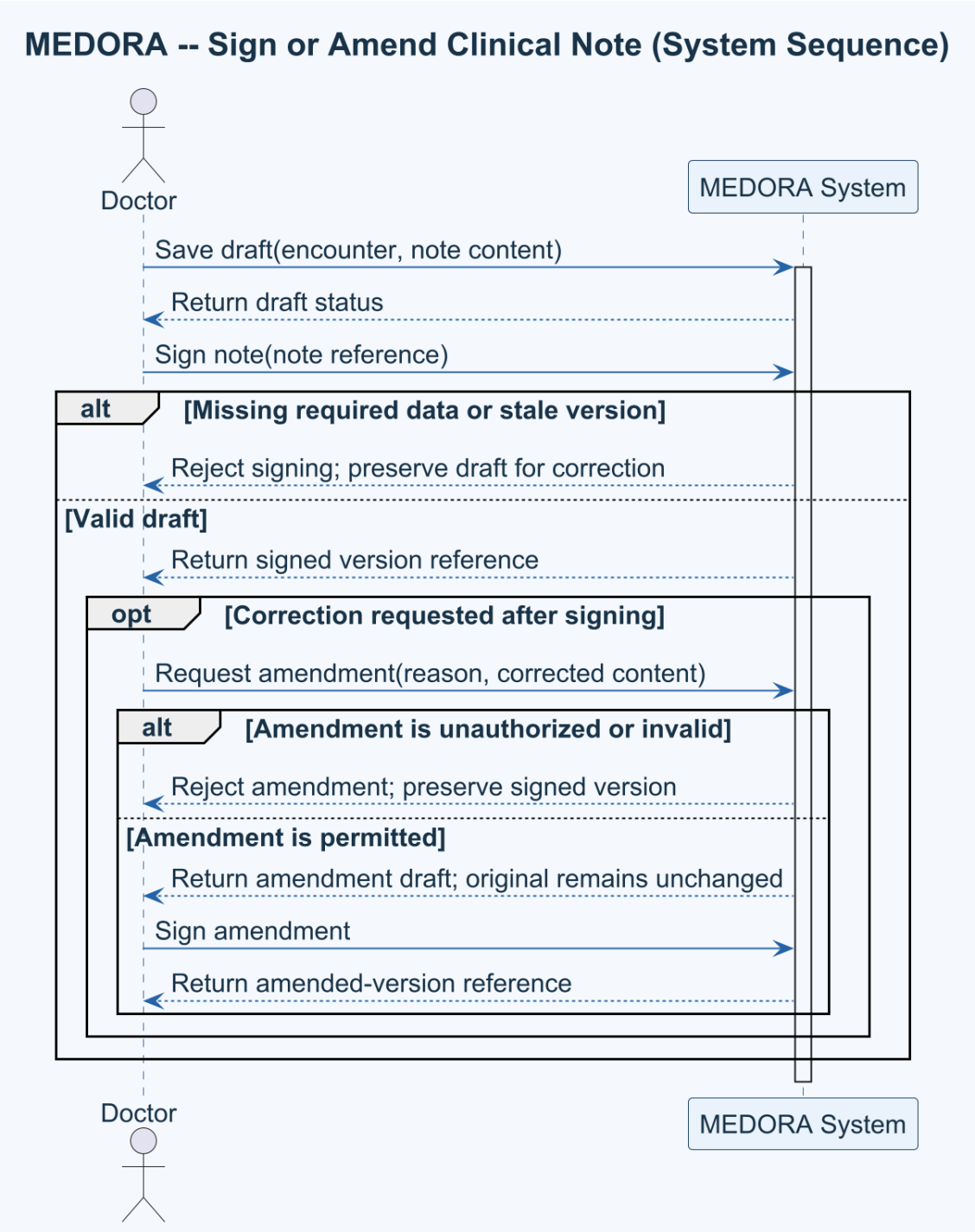


Figure 29: System sequence for signing a clinical note and appending a correction

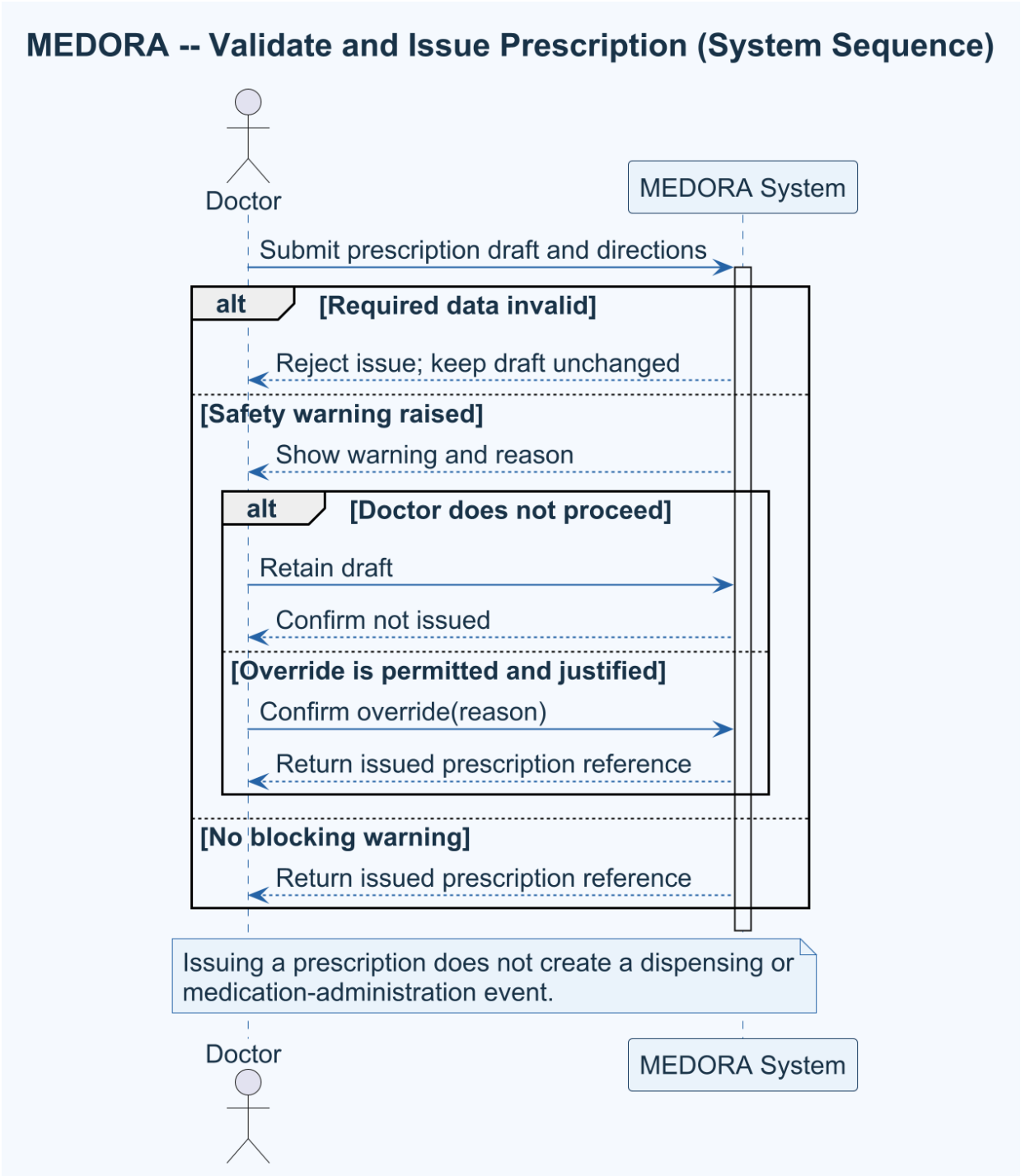


Figure 30: System sequence for configured prescription safety checks and issue

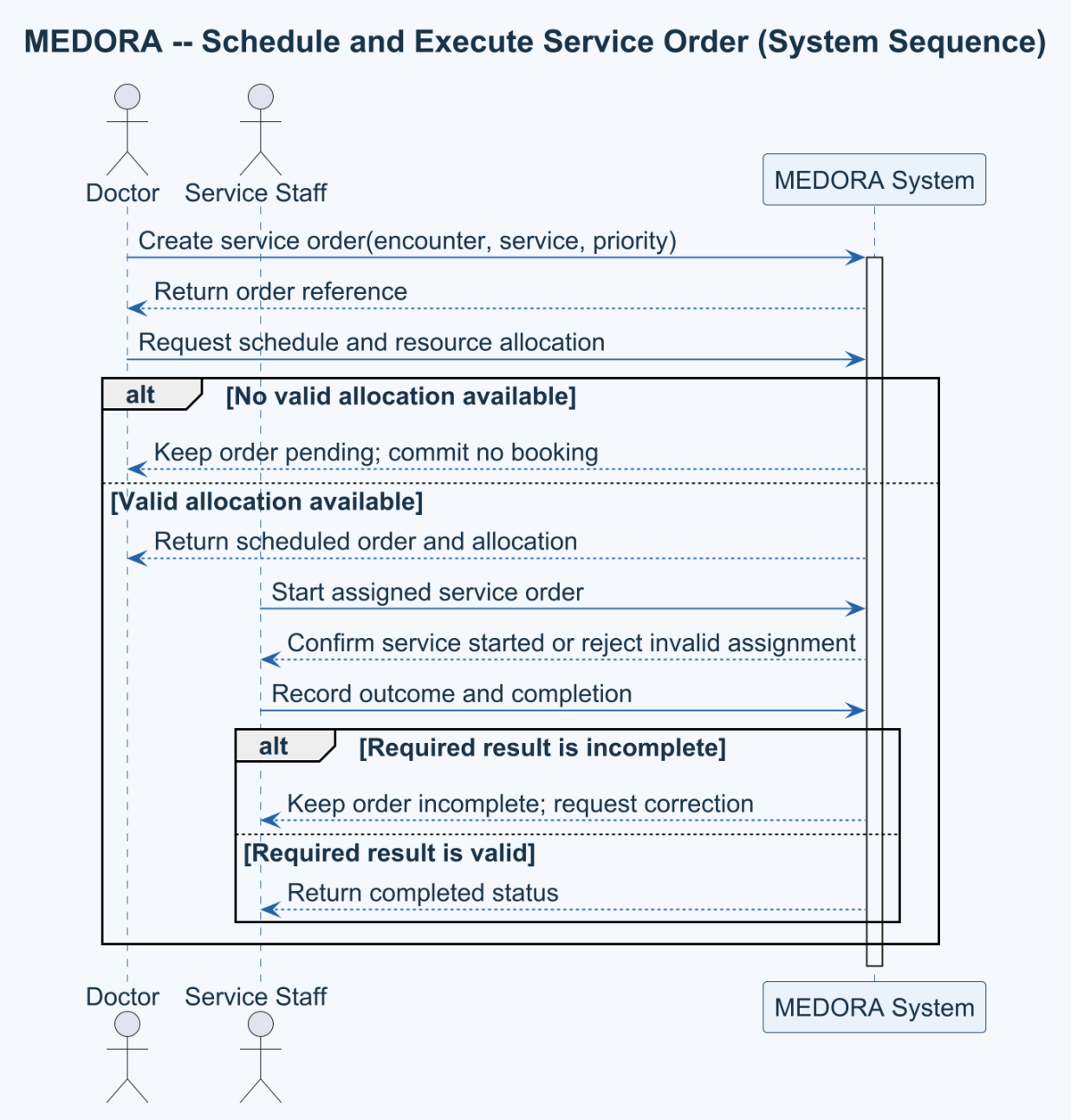


Figure 31: System sequence for resource allocation and service-order execution

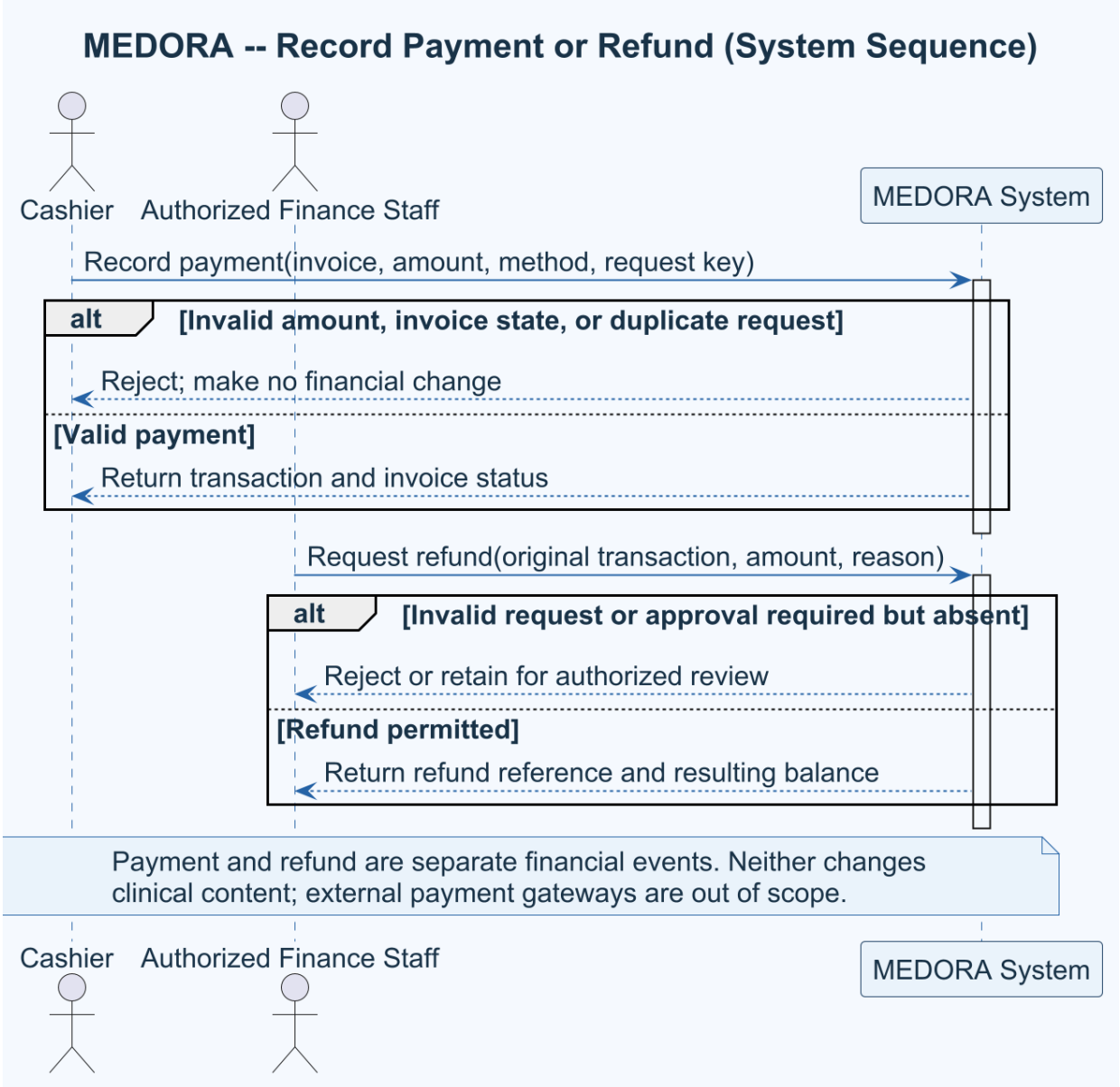


Figure 32: System sequence for payment and separately authorized refund events

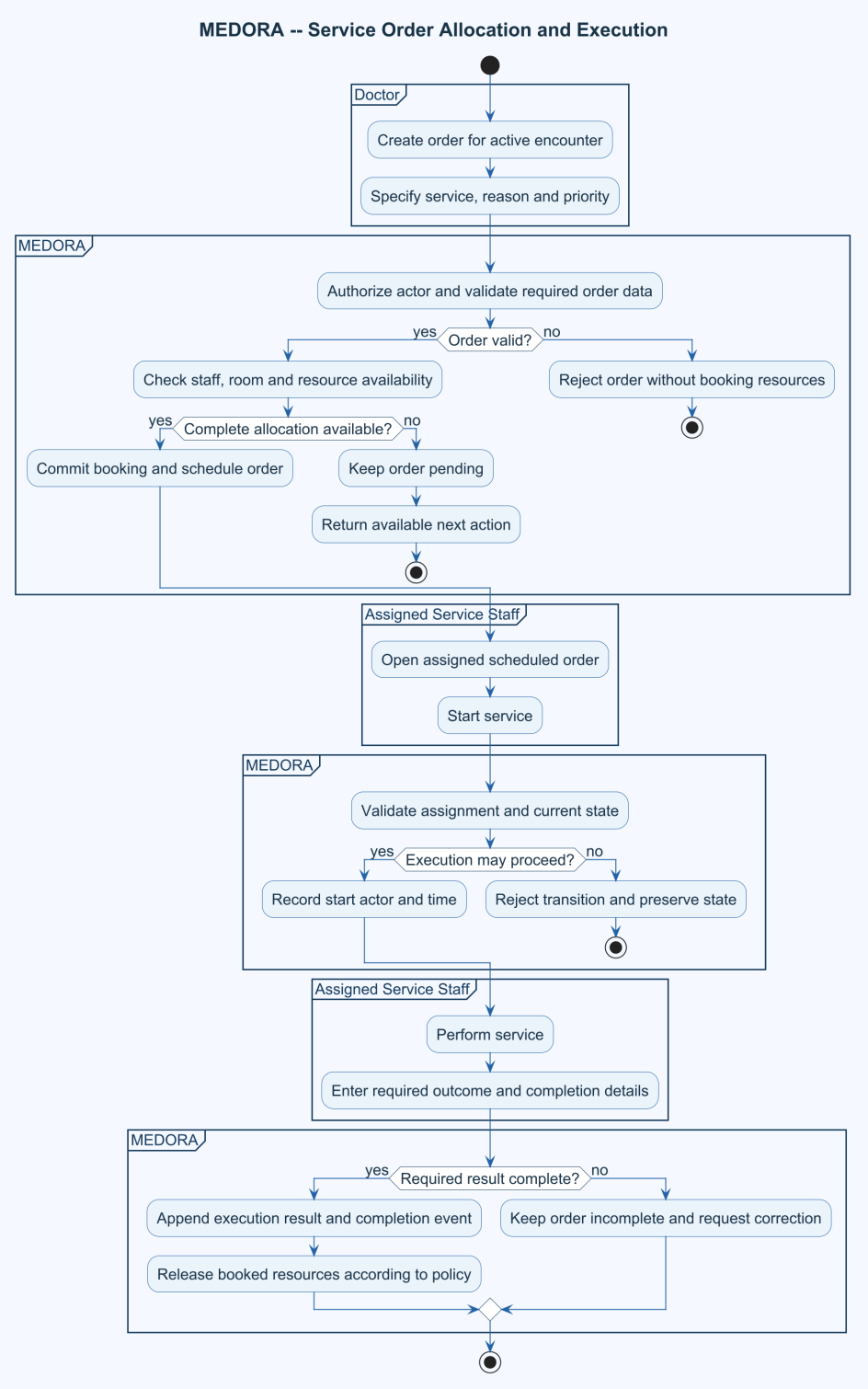


Figure 33: Activity model for service-order allocation and execution

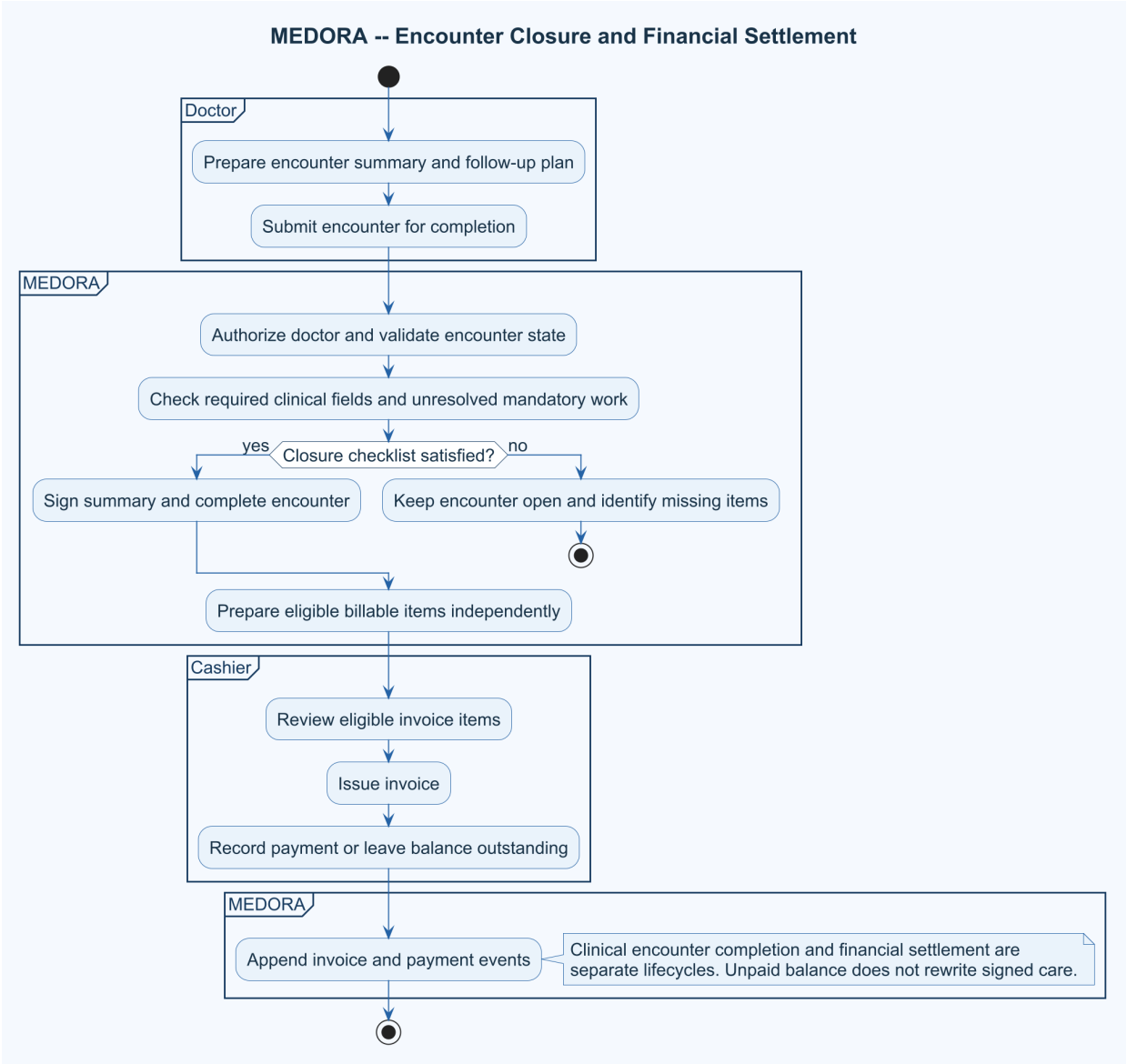


Figure 34: Activity model for encounter closure and independent financial settlement

References

Thi Thanh Thuy Nguyen et al. Waiting time in the outpatient clinic at a national hospital in vietnam. *BMC Health Services Research*, 18:1–9, 2018. doi: 10.1186/s12913-018-3285-2. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC5995730/.

Thi Thanh Thuy Nguyen et al. Outpatient waiting time at vietnam health facilities: policy implications for medical examination procedure. *Healthcare*, 8(1):63, 2020. doi: 10.3390/healthcare8010063. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC7151016/.

Organisation for Economic Co-operation and Development. *Health at a Glance 2023: OECD Indicators*. OECD Publishing, Paris, 2023. doi: 10.1787/7a7afb35-en. URL https://www.oecd.org/en/publications/health-at-a-glance-2023_7a7afb35-en/full-report/digital-health-at-a-glance_86518984.html.

Dusan Rodina. Hospital management system (UML diagrams). Software Ideas Modeler, 2024. URL https://www.softwareideas.net/uml-diagram-hospital-management-system. Accessed 14 October 2026.

saipraveen k. Smart hospital management system — UML. GitHub repository, 2026. URL https://github.com/saipraveen-k/Smart-Hospital-Management-System---UML. Accessed 14 October 2026.

United Nations Population Fund Viet Nam. Ageing in viet nam. UNFPA Viet Nam thematic brief, 2021. URL https://vietnam.unfpa.org/en/topics/ageing-0. Accessed 14 October 2026.

World Health Organization. Who launches global effort to halve medication-related errors in 5 years. WHO news release, 2017. URL https://www.who.int/news/item/29-03-2017-who-launches-global-effort-to-halve-medication-related-errors-in-5-years. Accessed 14 October 2026.

World Health Organization. Health data as a global public good: A call for health data governance. WHO policy article, 2021a. URL https://www.who.int/news-room/articles-detail/health-data-as-a-global-public-good-a-call-for-health-data-governance-30-september. Accessed 14 October 2026.

World Health Organization. *Global Strategy on Digital Health 2020–2025*. World Health Organization, Geneva, 2021b. ISBN 978-92-4-002092-4. URL https://www.who.int/publications/i/item/9789240020924.

World Health Organization. Patient safety. WHO Fact Sheet, 2023. URL https://www.who.int/news-room/fact-sheets/detail/patient-safety. Accessed 14 October 2026.

World Health Organization Alliance for Health Policy and Systems Research. Harnessing digital transformation to improve health. Thematic programme overview, 2024. URL https://ahpsr.who.int/what-we-do/thematic-areas-of-focus/harnessing-digital-transformation-to-improve-health. Accessed 14 October 2026.